Integrating Graph Learning and LLM Reasoning for Personalized Recommendation

BY

Mingdai Yang
B.Eng., The Chinese University of Hong Kong, Shenzhen, 2019
M.S., The University of Chicago, 2020

DISSERTATION

Submitted as partial fulfillment of the requirements
for the degree of Doctor of Philosophy in Computer Science
in the Graduate College of the
University of Illinois Chicago, 2026

Chicago, Illinois

Defense Committee:
Philip S. Yu, Chair and Advisor (Department of Computer Science)
Xinhua Zhang (Department of Computer Science)
Sourav Medya (Department of Computer Science)
Zhiwei Liu (Microsoft AI)
Liangwei Yang (Salesforce Research)

Accessibility Statement

An accessible EPUB version of this document is available at `https://github.com/mdyfrank/thesis_epub/blob/main/docs/thesis.epub`.

Acknowledgments

My deepest thanks go to my advisor, Prof. Philip S. Yu. Across my years at the University of Illinois Chicago he gave me the freedom to choose my own problems and the honesty to tell me when a direction was not working. Much of what I understand about finding a question worth years of effort, and about recognizing when an answer is not yet ready, I learned from him.

I thank Prof. Xinhua Zhang, Prof. Sourav Medya, Dr. Zhiwei Liu, and Dr. Liangwei Yang for serving on my committee. The hours they set aside for this work, and the questions they raised about it, made its argument clearer than I could have made it alone.

My two internships at Amazon showed me what recommendation research looks like at industrial scale. For that I thank my managers, Dr. Edward W. Huang and Dr. Yan Gao, my mentors, Dr. Jiangshu Du, Dr. Nurendra Choudhary, and Dr. Fan Yang, and the Amazon Scholars I worked with there, Prof. Danai Koutra and Prof. Jia Liu. They took my half-formed ideas seriously and were generous with their own.

I have been fortunate in the people I worked alongside. Dr. Chen Wang, Dr. Xiaolong Liu, Weizhi Zhang, Yibo Wang, and Prof. Hao Peng shared their thinking freely, pushed back on mine when it needed pushing, and made the slow stretches of this work far better company.

Finally, I thank my parents, Hong Fu and Lin Yang, who trusted my decisions long before there was any evidence that they would work out, and my grandmother, Zhen Tang, the very first teacher in my life. She raised me from childhood, and the habits of study she settled in me then are the ones I still work by.

MY

Contribution of Authors

Chapter 2 presents a published paper [91], for which I am the primary author. Dr. Xiaolong Liu, Dr. Chen Wang and Prof. Hao Peng contributed through idea discussions. Dr. Liangwei Yang assisted with experiments. Dr. Zhiwei Liu and Prof. Philip S. Yu provided constructive feedback and contributed to the revision of the manuscript.

Chapter 3 presents a published paper [86], in which I am the primary author. Dr. Liangwei Yang, Dr. Xiaolong Liu, Dr. Chen Wang and Prof. Hao Peng contributed through idea discussions. Dr. Zhiwei Liu and Prof. Philip S. Yu provided detailed instruction and helped on paper revision.

Chapter 4 presents a published paper [90], where I am the primary author. Dr. Zhiwei Liu, Dr. Liangwei Yang, Dr. Xiaolong Liu, Dr. Chen Wang and Prof. Hao Peng contributed through idea discussions. Prof. Philip S. Yu provided detailed guidance and assisted with the manuscript revision.

Chapter 5 is based on a manuscript being prepared for submission, for which I am the primary author. Dr. Zhiwei Liu and Prof. Li Sun contributed to idea discussions. Prof. Philip S. Yu helped on paper revision.

Chapter 6 presents a published paper [84], for which I am the primary author. Dr. Nurendra Choudhary, Dr. Jiangshu Du, Dr. Edward W Huang and Dr. Karthik Subbian contributed through idea discussions. Prof. Danai Koutra and Prof. Philip S. Yu provided detailed guidance and assisted with the manuscript revision.

Contents

Accessibility Statement ii

Acknowledgments iii

Contribution of Authors iv

List of Tables vii

List of Figures ix

List of Abbreviations x

Summary xii

Chapter 1. Introduction 1
1. Motivation and Dissertation Overview 1
2. Hypergraph Learning for Relation Modeling in Recommendation 5
3. Hypergraph Pretraining Guided by PLM-Encoded Textual Prompts 6
4. Training Large Recommendation Models via Graph-Language Token Alignment 6
5. LLM-Inferred Substitute and Complement Signals for Recommendation 7
6. Dynamic Recommendation with Item-Item Relations via LLM-based Agents 8

Chapter 2. Hypergraph Learning for Relation Modeling in Recommendation 9
1. Introduction 9
2. Related Work 12
3. Preliminaries 13
4. Proposed Framework: UPRTH 14
5. Experiment 21
6. Chapter Summary 27

Chapter 3. Hypergraph Pretraining Guided by PLM-Encoded Textual Prompts 29
1. Introduction 29
2. Related Work 33
3. Preliminaries 33
4. Proposed Framework: IHP 34
5. Experiment 41
6. Chapter Summary 48

Chapter 4. Training Large Recommendation Models via Graph-Language Token Alignment 50
1. Introduction 50
2. Related Work 52

3. Preliminaries 53
4. Proposed Framework: GLTA 53
5. Experiment 55
6. Chapter Summary 59

Chapter 5. LLM-Inferred Substitute and Complement Signals for Recommendation 60
1. Introduction 60
2. Related Work 62
3. Preliminaries 63
4. Proposed Framework: ERLSC 63
5. Experiment 67
6. Chapter Summary 70

Chapter 6. Dynamic Recommendation with Item-Item Relations via LLM-based Agents 71
1. Introduction 71
2. Related Work 75
3. Preliminaries 76
4. Proposed Framework: AgentDR 76
5. Experiment 87
6. Chapter Summary 97

Chapter 7. Conclusion 98
1. Summary of Contributions 98
2. Cross-Chapter Synthesis and Findings 99
3. Future Directions 101

Appendix A. ACM Author's Reuse Rights 104

Appendix B. Use of Generative AI and Language Models 105

BIBLIOGRAPHY 106

Vita 115

List of Tables

1.1 Relations modeled and semantic sources used by the five works presented in this dissertation. 4

1.2 Role of the language model, its coupling with the graph model, and the stage at which it is used. 4

2.1 Statistics of datasets for UPRTH. 21

2.2 Overall performance comparison of UPRTH with baselines. 22

2.3 Performance of recommendation models with and without embeddings from UPRTH. 23

2.4 Ablation study on the proposed TA Layer in UPRTH. 25

2.5 Performance of UPRTH w.r.t. objective functions in pretraining and finetuning. 26

3.1 Statistics of datasets for IHP. 41

3.2 Overall performance comparison of IHP with baselines. 41

3.3 The total training time of IHP. 43

3.4 Performance of IHP with different text encoders. 44

3.5 Performance of IHP on inductive user nodes. 47

4.1 Statistics of the datasets for GLTA 55

4.2 Overall performance comparison of GLTA with baselines. 56

4.3 Ablation study on profile and prediction tokens in GLTA. 58

5.1 Statistics of datasets for ERLSC. 67

5.2 Overall performance comparison of ERLSC with baselines. 68

5.3 Performance of ERLSC on each single relation type. 70

6.1 Comparison of AgentDR and other recommendation methods. 75

6.2 Prompts in AgentDR for $\text{LLM}_{\text{prof}}(\cdot)$. 84

6.3 Prompts in AgentDR for $\text{LLM}^{sub}_{\text{gen}}(\cdot)$. 84

6.4 Prompts in AgentDR for $\text{LLM}_{\text{cpr}}(\cdot)$. 85

6.5 Prompts in AgentDR for $\text{LLM}_{\text{dcm}}(\cdot)$. 85

6.6 Prompts in AgentDR for $\text{LLM}^{reg}_{\text{dcm}}(\cdot)$. 86

6.7 Prompts in AgentDR for $\text{LLM}^{reg}_{\text{rank}}(\cdot)$. 86

6.8 Prompts in AgentDR for $\text{LLM}^{sub}_{\text{rank}}(\cdot)$ and $\text{LLM}^{com}_{\text{rank}}(\cdot)$. 87

6.9 Statistics of datasets for AgentDR. 87

6.10 Prompts in AgentDR for VDCG evaluation. 91
6.11 Overall performance comparison of AgentDR with baselines. 92
6.12 VDCG comparison of AgentDR with other baselines. 94
6.13 Ensemble methods with LLM reasoning modules from AgentDR. 95
6.14 Performance of AgentDR with and without general reranking. 97

List of Figures

2.1 Construction of task hypergraphs in UPRTH. 15
2.2 The framework of UPRTH with the proposed TA layer. 17
2.3 Performance of UPRTH with and without unified pretraining. 24
2.4 Performance of UPRTH on inductive users. 27
2.5 Performance curves of UPRTH and other baselines. 28

3.1 A toy example of how IHP benefits recommendation. 31
3.2 The illustration of IHP with the proposed Prompt and PHC layers. 35
3.3 Performance of IHP w.r.t. different instruction constructions. 45
3.4 Performance of IHP with different target node finetuning strategies. 46
3.5 IHP finetuned with pretrained data added with $\lambda_t = 1$. 46
3.6 Performance of IHP w.r.t. prompt layer finetuning strategies. 46
3.7 Performance of IHP w.r.t. adaptation intensity coefficient λ_t. 46

4.1 A toy example of the input and the output in GLTA. 51
4.2 The framework of GLTA. 53
4.3 Ablation study on alignment methods in GLTA. 57
4.4 Performance of GLTA with different optimization methods. 59

5.1 The framework of ERLSC and the proposed LGHC layer. 64
5.2 Performance of ERLSC compared to its variants. 69

6.1 Two challenges in directly deploying LLMs for recommendation. 72
6.2 The framework of AgentDR. 77
6.3 VDCG of AgentDR with and without LLM-based modules. 94
6.4 Ablation study on comparison and reranking modules in AgentDR. 96

List of Abbreviations

AB:	Also-bought item co-occurrence signal
AgentDR:	Dynamic recommendation with item-item relations via LLM-based agents, the framework presented in Chapter 6
AU:	Alignment and uniformity, a recommendation training objective
AV:	Also-viewed item co-occurrence signal
BPR:	Bayesian personalized ranking
BT:	Bought-together item co-occurrence signal
CF:	Collaborative filtering
CW:	Compared-with item co-occurrence signal
DCG:	Discounted cumulative gain
ERLSC:	Enhancing recommendation with LLM-inferred substitute and complement signals, the framework presented in Chapter 5
GI:	Group identification
GLLM:	Graph-language logits matching
GLTA:	Graph-language token alignment, the framework presented in Chapter 4
GNN:	Graph neural network
ID:	Identifier
IHP:	Instruction-based hypergraph pretraining, the framework presented in Chapter 3
LGHC:	LLM-guided hypergraph convolution
LLM:	Large language model
MF:	Matrix factorization
MLP:	Multilayer perceptron
NDCG:	Normalized discounted cumulative gain
PHC:	Prompt hypergraph convolution
PLM:	Pretrained language model
RAG:	Retrieval-augmented generation
RecSys:	Recommender system

TA:	Transitional attention
UPRTH:	Unified pretraining for recommendation via task hypergraphs, the framework presented in Chapter 2
VDCG:	Vicinity discounted cumulative gain

Summary

Recommender systems (RecSys) are widely deployed in online platforms to help users discover relevant items from large collections. Despite their success, they still face the sparsity of observed user-item interactions and the difficulty of incorporating semantic knowledge that interaction records do not express. This dissertation studies how the knowledge held in language models should be coupled with collaborative graph structures, and argues that the appropriate granularity of this coupling depends on the recommendation task. Tighter coupling at the representation level admits richer semantic transfer but raises training cost, while offline relation inference and bounded agentic reranking offer lighter alternatives that remain grounded in the item catalog.

Five frameworks are developed along a progression from structural modeling to semantic guidance and then to explicit reasoning. The first treats pretext tasks and auxiliary relations as hyperedges, so heterogeneous and higher-order signals are unified in one pretraining stage. The second describes each task in plain text, encodes the descriptions with a pretrained language model, and uses the resulting semantics to guide hypergraph pretraining across domains. The third aligns user and item representations with the token embeddings of a large language model (LLM), placing collaborative patterns and pretrained linguistic knowledge in a common space. The fourth queries an LLM offline for substitute and complementary relations between items and uses the inferred relations to weight hyperedges. The fifth moves the language model to decision time, where an agent selects among recommendation tools and reasons over item relations to rerank a bounded candidate set without further training.

Experiments on public benchmarks from Amazon, Steam, Goodreads and Instacart show that each form of coupling improves recommendation quality over the baselines of its own setting, and that no single coupling strategy dominates across tasks. The experiments involve only computational analysis of existing datasets and do not involve human subjects, animals, or recombinant DNA. Therefore, approvals from the Institutional Review Board, the Animal Care Committee, or the Institutional Biosafety Committee are not required.

CHAPTER 1

Introduction

1. Motivation and Dissertation Overview

1.1. Motivation and Problem Setting. In the digital era, recommender systems (RecSys) play a crucial role in helping users discover relevant items across online platforms. By modeling historical user-item interactions, RecSys learn user preferences and provide personalized recommendations. To effectively capture collaborative signals, graph learning has become a dominant paradigm in RecSys, where users and items are organized into a bipartite structure for information propagation [70]. However, user-item interactions are inherently sparse due to limited user exploration and the extremely large candidate item space. Under such sparse settings, relying solely on pairwise interaction modeling remains insufficient for robust personalization.

Two observations motivate the work in this dissertation. First, the interactions that a platform records are only one view of the relations that connect users and items, since co-purchase behavior, item groupings, item attributes, and auxiliary supervision signals all carry information that a bipartite graph cannot express. Second, much of the knowledge needed to interpret those relations is textual rather than behavioral, because whether two items serve the same purpose, or serve each other, is a property of what the items are and not of how often they happen to appear together. Pretrained language models (PLMs) and large language models (LLMs) hold knowledge of this kind, so the question that organizes this dissertation is how such knowledge can be brought into a recommendation model that must still rank a full catalog efficiently and must still return identifiers that exist in that catalog.

1.2. Structural Signals, Semantic Knowledge, and Reasoning. The information brought into recommendation in this dissertation is of three distinct kinds, and keeping the

three apart is important for interpreting what each chapter contributes. Structural signals describe which entities co-occur. They are read off observed behavior and side information without any language model, and higher-order structure of this kind is naturally expressed by hypergraphs, where a single hyperedge connects an arbitrary number of nodes. Semantic knowledge describes what entities mean. It is obtained by encoding text with a PLM or an LLM into representations that a recommendation model can consume, and world knowledge is the special case that concerns properties no single dataset states, such as whether one product can replace another. Reasoning differs from both, since it refers to a language model producing an intermediate decision at inference time, for example choosing which retrieval tool suits a particular user or deciding why a candidate should be promoted.

These three kinds of information are used at different points in the following chapters. Chapter 2 uses structural signals alone. Chapter 3 adds semantic encoding of task descriptions by a PLM, and Chapter 4 tightens that encoding into a shared representation space with an LLM. Chapter 5 draws on world knowledge, but it does so offline and stores the result as structure. Only Chapter 6 asks a language model to reason at decision time. Treating semantic representation, world knowledge, and reasoning as one capability would obscure this progression, so the terms are used separately throughout.

1.3. Thesis Statement and Research Questions. The central claim of this dissertation is that language model knowledge should be coupled with collaborative graph structures at a granularity that suits the recommendation task, rather than at the tightest granularity that is technically available. Coupling at the representation level, where graph embeddings and language tokens share a space, permits the richest semantic transfer, but it requires gradients to pass through a large model and its cost grows with the model that supplies the knowledge. Coupling through offline relation inference, where a language model is queried once and its output is stored as a fixed structural weight, and coupling through bounded agentic reranking, where a language model reorders a short candidate list returned by conventional retrievers, are lighter and keep the recommended items grounded in the catalog. The five frameworks presented here occupy different points on this spectrum, and the picture

that emerges is that the appropriate point depends on the task rather than on the capacity of the language model.

Three research questions organize the work.

RQ1. How can higher-order and heterogeneous relations be modeled so that they alleviate the sparsity of user-item interactions rather than introduce noise?

RQ2. At what level should structural representations and language model knowledge be aligned, and what does a tighter alignment cost in training?

RQ3. How can the knowledge and the reasoning ability of LLMs be used in recommendation while preserving grounding in the item catalog and scalable full-catalog ranking?

Chapter 2 and Chapter 3 mainly address RQ1, Chapter 3 and Chapter 4 address RQ2, and Chapter 5 and Chapter 6 address RQ3. Chapter 7 returns to the three questions and reads the five sets of results together.

1.4. Overview of the Five Frameworks. Table 1.1 and Table 1.2 summarize how the five works relate to one another. The modeled relations move from pretext tasks within a single domain to functional relations between items, while the language model shifts from an external encoder to a component trained jointly with the graph model, and finally to a decision maker at inference time. The coupling between the two sides is therefore not monotonic, as Chapter 4 represents the tightest integration and the following two chapters deliberately loosen it to keep the cost of using increasingly large models manageable.

1.5. Contributions and Organization. The main contributions of this dissertation are summarized as follows.

- A unified pretraining framework that generalizes heterogeneous pretext tasks as hyperedge prediction and learns the relevance of each task to recommendation through a transitional attention layer.
- An instruction-based hypergraph pretraining framework that encodes plain-text task descriptions with a PLM and injects them into hyperedge-level propagation for cross-domain recommendation.

Chapter	Relations modeled	Semantic source
Chapter 2 (UPRTH)	Pretext task hyperedges in one domain	Not used
Chapter 3 (IHP)	Task hyperedges across domains	Plain-text task instructions
Chapter 4 (GLTA)	User-item interaction graph	Textual item descriptions
Chapter 5 (ERLSC)	Item group hyperedges from co-occurrence	World knowledge of item function
Chapter 6 (AgentDR)	Substitute and complementary item relations	World knowledge of item function

TABLE 1.1. Relations modeled and semantic sources used by the five works presented in this dissertation.

Chapter	Role of the language model	Coupling with the graph model	Stage of use
Chapter 2 (UPRTH)	Not used	Structural signals only	Pretraining
Chapter 3 (IHP)	Frozen sentence encoder	Prompt representations injected into hypergraph convolution	Pretraining
Chapter 4 (GLTA)	Frozen LLM supplying a shared token space	Gradients pass through the frozen LLM	Training
Chapter 5 (ERLSC)	Offline relation annotator	Hyperedge weights, no gradients through the LLM	Offline preprocessing
Chapter 6 (AgentDR)	Inference-time decision maker	Pretrained graph model called as one of the ranking tools	Inference

TABLE 1.2. Role of the language model, its coupling with the graph model, and the stage at which it is used.

- A token alignment framework that places graph representations and LLM token embeddings in a shared space and predicts items by logits matching instead of free-form generation.
- A relation-aware hypergraph framework that obtains substitute and complementary judgments from an LLM offline and converts them into adaptive hyperedge weights.
- An LLM agent framework that delegates full-catalog ranking to conventional recommenders and reasons over substitute and complementary relations to rerank a bounded

candidate set, together with an evaluation metric that measures semantic alignment jointly with ranking correctness.

- A cross-chapter analysis of how the granularity of graph-language coupling trades semantic transfer against computational cost, presented in Chapter 7.

The remainder of this dissertation is organized as follows. The rest of this chapter introduces the five works in turn. Chapter 2 presents UPRTH, Chapter 3 presents IHP, Chapter 4 presents GLTA, Chapter 5 presents ERLSC, and Chapter 6 presents AgentDR. Chapter 7 synthesizes the findings, discusses limitations, and outlines future directions.

2. Hypergraph Learning for Relation Modeling in Recommendation

(Part of this section was previously published in [91].)

Although graph-based RecSys have shown strong performance, they mainly model pairwise user-item interactions, which limits their ability to capture higher-order relations and heterogeneous signals. In real-world recommendation scenarios, rich relational structures naturally exist, including multi-entity dependencies and auxiliary supervision signals. Hypergraph learning provides a principled framework to model such complex relations, as a hyperedge can connect multiple nodes simultaneously.

In this work, we propose a multitask framework named Unified Pretraining for Recommendation via Task Hypergraphs (UPRTH). To provide a unified learning paradigm for diverse relational signals, we generalize different pretext tasks as hyperedge prediction problems within a task hypergraph. In this way, heterogeneous relations and auxiliary attributes can be integrated into recommendation through a shared hypergraph structure. Furthermore, a transitional attention layer is designed to adaptively learn the relevance between each auxiliary task and the main recommendation objective. Experimental results on three benchmark datasets verify the effectiveness of hypergraph learning for modeling diverse relations in recommendation.

3. Hypergraph Pretraining Guided by PLM-Encoded Textual Prompts

(Part of this section was previously published in [86].)

Building upon the flexibility of hypergraph construction introduced in UPRTH, we extend this paradigm to cross-domain recommendation by incorporating semantic guidance into hypergraph pretraining. In cross-domain settings where domains share the same users but completely different items, discrepancy in data distribution hinders effective knowledge transfer via structural signals alone. To address this limitation, we introduce plain-text prompts that explicitly describe task semantics and domain connections, and encode them using a PLM to guide hypergraph pretraining. To be concrete, we propose Instruction-based Hypergraph Pretraining (IHP), a framework that injects PLM-encoded textual instructions into hypergraph learning to align source-domain knowledge with target-domain recommendation objectives. A prompt hypergraph convolution layer is designed to inject the encoded prompt representations into hyperedge-level information propagation, enabling instruction-aware high-order relation modeling. By integrating semantic guidance from text prompts with structural hypergraph learning, IHP enhances cross-domain knowledge transfer and improves recommendation performance across multiple datasets.

4. Training Large Recommendation Models via Graph-Language Token Alignment

(Part of this section was previously published in [90].)

While IHP introduces plain-text prompts to provide task-level semantic guidance for hypergraph learning, the amount of injected knowledge remains limited to a small number of instructions, each compressed into a single fixed-dimensional vector. To integrate richer prior knowledge from language models, we explore a finer alignment between graph-based recommendation models and pretrained language representations. Traditional collaborative filtering models primarily rely on user-item interactions and lack direct access to the extensive semantic knowledge encoded in LLMs. At the same time, directly applying LLMs for recommendation often suffers from ambiguous item generation and poor scalability. To address

these challenges, we propose training large recommendation models via Graph-Language Token Alignment (GLTA). Instead of generating items through free-form text, GLTA aligns user and item representations from the interaction graph with pretrained LLM token embeddings, allowing the graph model to inherit semantic knowledge from frozen LLMs in a structured manner. Furthermore, we introduce Graph-Language Logits Matching (GLLM) to optimize token alignment for end-to-end item prediction, which avoids the ambiguity of text generation while preserving scalability. Experimental results on benchmark datasets demonstrate that token-level alignment effectively enhances recommendation performance.

5. LLM-Inferred Substitute and Complement Signals for Recommendation

Token alignment couples the two sides tightly, but it requires backpropagation through the entire language model, and as the size of LLMs continues to grow this becomes increasingly expensive. To obtain a more lightweight integration of LLM knowledge into recommendation, we instead query an LLM offline to evaluate functional relations between items, and enhance recommendation through the refined relational signals that result.

Substitution and complementarity are essential item-item relations for understanding user intent in RecSys. However, reliable annotations of such relations are rarely available in public datasets, and behavior-based signals such as co-view and co-purchase often conflate true semantic relations with incidental co-occurrences. To address this challenge, we propose Enhancing Recommendation with LLM-Inferred Substitute and Complement Signals (ERLSC), a framework that leverages LLMs to assess functional relations within item groups and guide hyperedge weighting. We adopt a hypergraph formulation to represent item groups induced by co-occurrence signals, and use LLMs to evaluate pairwise substitute and complementary relations within each group. Based on these semantic assessments, we design an LLM-guided hypergraph convolution layer that initializes and adaptively updates hyperedge weights, enabling the model to emphasize semantically meaningful item groups while

suppressing noisy co-occurrences. Extensive experiments on three public datasets demonstrate that ERLSC consistently outperforms strong baselines, and ablation studies verify the effectiveness of each component.

6. Dynamic Recommendation with Item-Item Relations via LLM-based Agents

(Part of this section was previously published in [84].)

While ERLSC leverages LLMs to refine structural signals through LLM-guided hyperedge weighting within a unified graph model, it still follows a single-model learning paradigm. In this setting, the final recommendation performance ultimately relies on a single trained model, even when it is enhanced by LLM-derived relational signals. Motivated by the reasoning capabilities of modern LLMs, we instead explore an LLM-based agent framework that can coordinate multiple recommendation tools and perform explicit relational reasoning at inference time, without being restricted to the output of a single recommendation model.

Recent agent-based recommendation methods use memory and prompting to mimic user behavior, but they often hallucinate non-existent items and struggle with full-catalog ranking. To address these challenges, we propose AgentDR, an LLM-agent framework that combines commonsense reasoning with scalable recommendation tools. AgentDR treats the LLM as a high-level decision maker, where full-catalog ranking is delegated to traditional models for efficiency and ID grounding, while the LLM integrates multiple recommendation outputs based on personalized tool suitability. Building on the effectiveness of leveraging LLM world knowledge to evaluate item-item relations in ERLSC, we further enable the agent to explicitly reason over substitute and complementary relations inferred from the user's historical interactions, guiding the selection and refinement of candidate items. This modular design preserves scalability and stability while adding relational reasoning at the decision level. Experiments on three public grocery datasets show that AgentDR improves full-ranking performance, achieving on average a twofold gain over its underlying tools, and we further introduce an LLM-based evaluation metric that jointly measures semantic alignment and ranking correctness.

CHAPTER 2

Hypergraph Learning for Relation Modeling in Recommendation

(This chapter was previously published as "Unified Pretraining for Recommendation via Task Hypergraphs" [91] in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pp. 891 - 900. 2024. [doi:10.1145/3616855.3635811])

1. Introduction

As outlined in Chapter 1, user-item interactions alone are too sparse to support robust personalization, so this chapter follows the first direction of this dissertation and models auxiliary relations and attributes as hyperedges, allowing heterogeneous signals beyond pairwise interactions to be integrated into a single pretraining framework. Multitask pretraining has gained considerable attention in the field of RecSys. On the one hand, since users and items are usually tied with unique IDs in each dataset for recommendation, the prior knowledge obtained by pretraining on only user-item interactions in a dataset can hardly be transferred to other downstream datasets where IDs are different. On the other hand, if pretraining and finetuning are performed on user-item interactions in the same dataset, the model has a high risk of memorizing the specific patterns and distributions of the pretraining data and being affected by the overfitting problem. On the contrary, when a model is pretrained on multiple related tasks simultaneously, it is exposed to diverse contextual cues for personalized recommendation. As a result, the model is encouraged to discover relevant commonalities and underlying structures across various tasks, leading to improved generalization capabilities and robust representations.

Previous works applying multitask pretraining for recommendation rely on language models to capture contextual information from text. P5 [16] converts all data, such as user-item interactions, item metadata, and user reviews, to natural language sequences and then pretrains on a Transformer [64] model that produces target responses according to textual

inputs. Analogously, M6-Rec [11] converts all tasks in RecSys into language understanding and language generation, and leverages a generative pretrained language model M6 [37] to predict user preferences as plain texts. These pretraining frameworks based on language models can unify various subtasks and exploit abundant semantics inside the training corpora for recommendation.

However, obvious linguistic biases are observed in the pretraining frameworks based on language models. In other words, such frameworks prefer predicting fluent and grammatically correct text to recommending items based on user preferences [99]. In addition, unifying subtasks by converting all data to plain text inevitably introduces extra noise into original structure information, which is crucial in capturing user preference through collaborative filtering [70]. Besides these drawbacks, the lack of raw plain text data in datasets further prevents the wide application of language-based multitask pretraining frameworks, although language models support models with sufficient flexibility to unify diverse pretext tasks during pretraining.

Even so, designing a unified learning pattern that effectively handles the diverse requirements and nuances of each task is still important for a pretraining framework. Unifying the learning across various tasks allows the framework to develop shared representations capturing common patterns and features across the tasks. Tasks with similar underlying structures or require reasoning abilities can benefit from shared representations capturing common knowledge. Moreover, unification across tasks helps address the issue of data sparsity for individual tasks. When certain tasks have limited labeled data, the shared knowledge learned from other tasks can be transferred to improve performance.

To both preserve the original structure information in pretext tasks and realize the unification across tasks, we propose a novel framework called Unified Pretraining for Recommendation via Task Hypergraphs (UPRTH). Above all, the hypergraph is an irreplaceable data structure to build a unified architecture for multitask pretraining. Although edges in a graph are widely used to represent pairwise user-item interactions in RecSys, they can hardly express more complex high-order relations in other tasks, such as multiple items in

the same category for category classification tasks, or multiple users in the same group for group identification tasks. On the contrary, the hyperedge simultaneously connecting multiple nodes is more flexible to represent various relations in different pretext tasks. In order to ensure the unification across pretext tasks in pretraining, we generalize all pretext tasks to the prediction of different hyperedges in task hypergraphs. For example, for an item category classification task as a pretext task, we regard each item as a node and each category as one hyperedge, and then pretrain the model to predict if an item in a specific category is connected to the corresponding hyperedge.

Next, to learn prior knowledge from the original structure information during the pretraining on task hypergraphs, only the embeddings of user and item nodes are learned to integrate all the contextual information from pretext tasks into user and item representations. In this way, user preferences and item properties reflected by different pretext tasks are completely preserved in these embeddings for the following finetuning.

Within a unified pretraining framework, a formidable challenge lies in balancing the degrees of relevance between auxiliary tasks and the target recommendation task during pretraining. Certain tasks may be more closely related to the recommendation problem, while others might be less informative or even potentially detrimental. As a result, it becomes essential to determine the appropriate level of knowledge transfer from each task to ensure that the most relevant and useful information is incorporated for personalized recommendations. Additionally, while the framework should learn generalized representations from multitask pretraining to enhance generalization, it must also capture task-specific nuances to provide accurate and personalized recommendations. Striking the right balance between these two aspects is crucial to avoid overgeneralization on the downstream task.

To adaptively learn the degrees of relevance between each auxiliary task and the target recommendation task during pretraining, we propose a novel transitional attention (TA) layer. This attention mechanism ensures the discriminative incorporation of the information from various pretext tasks for the target recommendation task. Additionally, this TA layer

directly transmits task-specific knowledge of users and items in auxiliary tasks to the recommendation task during multitask pretraining, which encourages the framework to capture task-specific nuances to avoid overgeneralization. The main contributions of UPRTH are summarized as follows:

- We design UPRTH, a model-agnostic unified multitask pretraining framework that generalizes heterogeneous pretext tasks as task hypergraphs for recommendation.
- We devise a novel TA layer to harness the knowledge from each pretext task, which discriminatively learns the degrees of relevance between each auxiliary task and the target recommendation task during pretraining.
- We conduct extensive experiments on three real-world datasets. The consistent improvement of UPRTH on all datasets indicates its effectiveness as a pretraining framework for recommendation.

2. Related Work

2.1. Hypergraph Learning for Recommendation. The application of hypergraph learning has proven successful in various graph-based tasks [13, 15]. Consequently, researchers have also explored the application of hypergraphs in RecSys, recognizing their ability to capture complex high-order dependencies through hyperedge-node connections [95, 79, 27, 85]. Thanks to the flexibility of hyperedge construction, some previous works leverage hypergraph learning to integrate contextual information in addition to user-item interaction history for personalized recommendation. MRH [4] constructs hyperedges among users, music, and music tags to create a hypergraph for music recommendation. MHCN [94] introduces a multi-channel hypergraph convolutional network that leverages high-order user-user relations to enhance social recommendation. Unlike prior methods, UPRTH uses hypergraphs to unify heterogeneous pretext tasks within a shared pretraining framework.

2.2. Graph Neural Networks with Pretraining. Graph Neural Networks (GNNs) have garnered considerable attention due to the prevalence of graph-structured data in various domains. In order to enhance learning effectiveness on graphs, researchers have delved

into the exploration of pretraining GNNs using unlabeled graph data. GCC [51] applies contrastive learning to capture the universal topological properties across multiple networks. GPPT [61] leverages a graph prompting function to enhance the reliability and efficiency of pretrained GNNs. GPT-GNN [24] introduces an attributed graph generation task in pretraining to capture the semantic properties of the graph. Most existing pretrained GNN frameworks are designed and optimized for classification tasks as downstream tasks [29, 42, 51]. These classification frameworks heavily depending on node attributes during the fine-tuning process are not well-suited for recommendation when attributes are unavailable in the downstream recommendation task.

2.3. RecSys with Pretraining. RecSys play a crucial role in helping users discover relevant items from vast collections [70, 18, 89]. Recent studies have explored the integration of pretraining techniques to enhance the performance of RecSys [81, 7]. One notable approach in this context is self-supervised pretraining with data augmentation [77, 104], which offers a promising avenue for leveraging large amounts of unlabeled data as the source of prior knowledge.

Given the increasing abundance of contextual data in RecSys, researchers have begun to explore other pretraining strategies that incorporate additional sources of information. PNMTA [49] proposes a meta-learning-based method that makes use of the existing non-cold-start users for pretraining to improve cold-start recommendation. P5 [16] and M6-Rec [11] convert all data in RecSys to natural language sequences and deploy language models for pretraining. Compared to these methods, our proposed framework is model-agnostic and only requires graph-structured data during pretraining, which are widely used in RecSys [70], instead of depending on relatively rare visual or textual information.

3. Preliminaries

3.1. Recommendation Task. Given two disjoint node sets, including a user set $\mathcal{U}$ and an item set $\mathcal{I}$, and the interactive edges, i.e., user-item edges $E_{\mathcal{U},\mathcal{I}}$, an interaction graph is defined as $G = (V, E)$ where $V = \mathcal{U} \cup \mathcal{I}$. A personalized recommendation task t_{rec} for a user

u is to predict a ranking list of items $\{i_1, i_2, \ldots, i_m\}$, with which this user has no interactions in the graph G.

3.2. Auxiliary Graph-based Pretext Task. Within an auxiliary pretext task set T, for a specific task $t \in T$, given an additional node property set A or an additional edge set E_t associated with the task-related node set $V_t \subseteq V$, an auxiliary graph-based pretext task is to predict the value of property $a_t \in A_t$ for an unlabeled node v_t, or the unobserved edge e_t in the auxiliary graph $G_t = (V_t, E_t)$.

3.3. Task Hypergraph. Given a pretext task t which is t_{rec} or an auxiliary pretext task $t \in T$, the task hypergraph is denoted as $\mathcal{H}_t = (V_t, \mathcal{E}_t)$, where V_t are nodes related to the specific task in the interaction graph G and $\mathcal{E}_t$ are task-specific hyperedges. An incidence matrix $\mathbf{H}_t^{\mathbf{v}} \in \{0,1\}^{|V_t| \times |\mathcal{E}_t|}$ is used to represent connections among the related nodes. In other words, we leverage hyperedges to represent associations among task-related nodes in each pretext task.

3.4. Hypergraph Encoder. The hypergraph encoder in our framework consists of hypergraph convolution layers, each of which is formulated as:

$$\begin{aligned} \mathbf{E}_t'^{\mathbf{v}} &= (\mathbf{D}_t^{\mathbf{v}})^{-1}\mathbf{H}_t^{\mathbf{v}} \cdot \mathbf{E}_t^{\epsilon}, \\ &= (\mathbf{D}_t^{\mathbf{v}})^{-1}\mathbf{H}_t^{\mathbf{v}} \cdot (\mathbf{B}_t^{\mathbf{v}})^{-1}(\mathbf{H}_t^{\mathbf{v}})^{\top}\mathbf{E}_t^{\mathbf{v}}, \end{aligned} \tag{1}$$

where $\mathbf{E}_t^{\mathbf{v}}$ and $\mathbf{E}_t'^{\mathbf{v}}$ are input and output embeddings of task-related node set V_t for a certain pretext task t in a hypergraph convolution layer, $\mathbf{H}_t^{\mathbf{v}}$ is the incidence matrix of the task hypergraph, $\mathbf{E}_t^{\epsilon}$ denotes the hyperedge embeddings, $\mathbf{D}_t^{\mathbf{v}}$ is the degree matrix of nodes and $\mathbf{B}_t^{\mathbf{v}}$ is the degree matrix of hyperedges for normalization.

4. Proposed Framework: UPRTH

In this section, we present the proposed UPRTH framework for unified pretraining. The illustration of UPRTH is demonstrated in Figure 2.2. We start by introducing all the embeddings to be pretrained in this framework. Thereafter, we define three types of pretext tasks and explain how to construct the task hypergraph corresponding to each type of task.

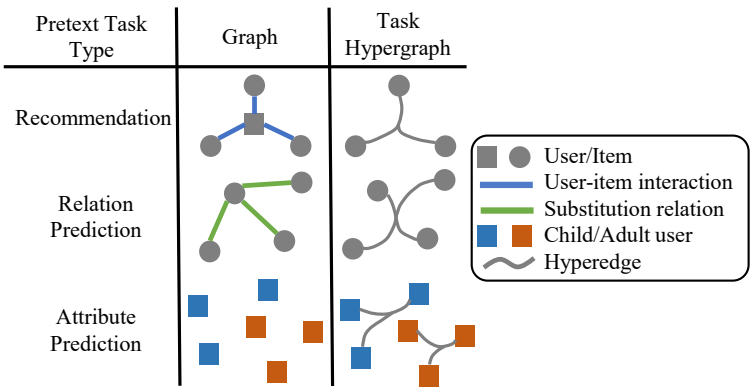


FIGURE 2.1. Construction of task hypergraphs in UPRTH.

Specifically, we adopt a TA layer to learn the relevance between each auxiliary task and the recommendation task, and then discriminatively integrate the prior knowledge from pretext tasks to the pretrained embeddings. Our framework can be easily extended to other graph-based tasks besides recommendation by replacing the target recommendation task. We leave this extension to the future work.

4.1. Embedding Layer. We maintain an embedding layer $\mathbf{E} \in \mathbb{R}^{(|\mathcal{U}|+|\mathcal{I}|)\times d}$, where d is the feature dimension and rows represent all trainable embeddings for users and items during pretraining. We denote the initial user embedding as $\mathbf{E}^{\mathbf{u}}_{t_{rec}}$, and the initial item embedding as $\mathbf{E}^{\mathbf{i}}_{t_{rec}}$. By using the pretrained embeddings $\mathbf{E}^{\mathbf{u}}_{t_{rec}}$ and $\mathbf{E}^{\mathbf{i}}_{t_{rec}}$, prior knowledge learned from the pretraining phase can be leveraged when building a downstream recommendation model.

4.2. Task Hypergraph Construction. For the purpose of a unified architecture to handle various pretext tasks, we first generalize each pretext task to the format of a corresponding task hypergraph. A toy example is shown in Figure 2.1. Compared to a graph where edges only connect two nodes, the advantage of a hypergraph where hyperedges can simultaneously connect multiple nodes lies in its ability to represent complex relationships. In multitask pretraining, the association among task-related nodes in each pretext task is diverse. Thus, we design a task hypergraph for each pretext task, in order to express these associations in different pretext tasks in a unified format of hyperedges for further hypergraph learning.

Without loss of generality, we categorize all pretext tasks into three types, namely *Recommendation*, *Relation Prediction*, and *Attribute Prediction*. Then we propose the construction of the hypergraph corresponding to each type as follows:

- **Recommendation**. Recommendation is to predict an unobserved relation between users and items, including purchasing, clicking, viewing or other user-item interactions. In this case, we let each type of node be the hyperedges to connect the other type of nodes. For example, if multiple users purchase an item, a hyperedge is constructed to connect them simultaneously. Two incident matrices $\mathbf{H}^{\mathbf{u}}_{t_{rec}} \in \{0,1\}^{|\mathcal{U}|\times|\mathcal{I}|}$ and $\mathbf{H}^{\mathbf{i}}_{t_{rec}} \in \{0,1\}^{|\mathcal{I}|\times|\mathcal{U}|}$ are used to represent the hyperedge connection in such tasks.
- **Relation Prediction**. The tasks belonging to this type are to predict an unobserved relation among nodes with the same type, such as whether a substitution relation exists between item nodes. In this type of task, we let the homogeneous relation itself be the hyperedge. For example, items that are substitutions of a target item are connected by a hyperedge. The incidence matrix for this hypergraph is $\mathbf{H}^{\mathbf{i}}_t \in \{0,1\}^{|\mathcal{I}|\times|\mathcal{E}^{\mathbf{i}}_t|}$ where $|\mathcal{E}^{\mathbf{i}}_t|$ represents the number of substitution relations.
- **Attribute Prediction**. In this type of task, we let each type of attribute be the hyperedge to connect the nodes with such a type of attribute. For example, in a category classification task, items belonging to the same category are connected by one hyperedge. For the attributes whose values are continuous, such as ratings of items, we quantize those values to certain discrete magnitudes. The incidence matrix is denoted by $\mathbf{H}^{\mathbf{u}}_t \in \{0,1\}^{|\mathcal{U}|\times|A_t|}$ or $\mathbf{H}^{\mathbf{i}}_t \in \{0,1\}^{|\mathcal{I}|\times|A_t|}$, where $|A_t|$ is the number of possible values of the attribute to predict in the task t.

Both relation prediction and attribute prediction contain more fine-grained pretext tasks, such as friend recommendation and rating prediction. For all three types of pretext tasks, hyperedges are leveraged to connect homogeneous nodes in our construction of task hypergraphs. In this way, users or items with specific commonalities corresponding to this task are connected by one hyperedge for a certain task. Henceforth, we can effectively capture

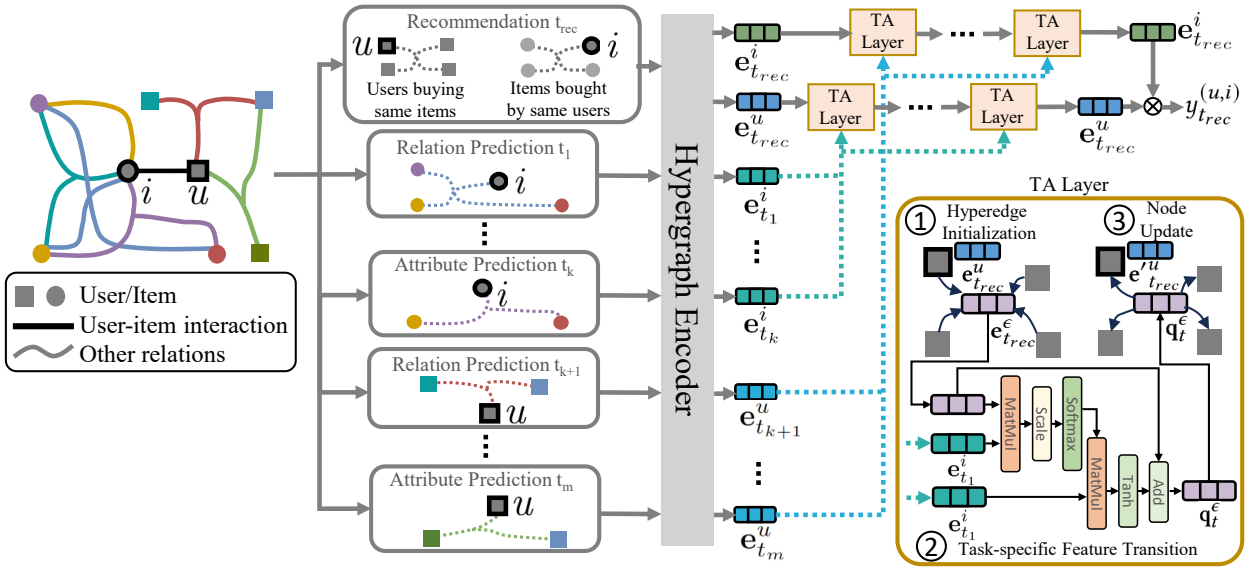


FIGURE 2.2. The framework of UPRTH with the proposed TA layer.

collaborative signals among users and items from different pretext tasks through hypergraph learning.

4.3. Transitional Attention Layer. To leverage multiple pretext tasks to improve the performance of the target recommendation task, it is important to adaptively learn the degrees of relevance between each auxiliary pretext task and the target recommendation task during pretraining. In this way, the model can identify which tasks provide the insights that are crucial for the recommendation task. Hence, we propose a Transitional Attention (TA) Layer to ensure that the model allocates appropriate attention to each task based on its relevance, thereby enabling effective knowledge transfer during pretraining. The illustration is in Figure 2.2(b). Next, we present the calculation details of a TA layer that discriminatively transfers knowledge from item-related auxiliary pretext tasks to the recommendation task by pretraining user embeddings. For simplicity, we use T to represent the set of item-related auxiliary tasks in this section. We denote the output user embedding as $\mathbf{e}'^{u}_{t_{rec}}$ to distinguish it from the input user embedding $\mathbf{e}^{u}_{t_{rec}}$ only for this section.

4.3.1. *Hyperedge Initialization.* Following the construction of the task hypergraph for the recommendation task, in Figure 2.2(b)①, we use item hyperedges to connect users and

initialize those hyperedges by aggregating users' preferences as below:

$$\mathbf{e}^{\epsilon}_{t_{rec}} = \frac{1}{|\mathcal{N}_\epsilon|} \sum_{u^* \in \mathcal{N}_\epsilon} \mathbf{e}^{u^*}_{t_{rec}}, \tag{2}$$

where $\mathbf{e}^{\epsilon}_{t_{rec}}$ denotes temporary embedding of the hyperedge ϵ, $\mathcal{N}_\epsilon$ is the set of users connected by this hyperedge, and $\mathbf{e}^{u^*}_{t_{rec}}$ is the embedding of those users as input of the l-th TA layer. $\mathbf{e}^{\epsilon}_{t_{rec}}$ represents the preferences of users who are connected by this hyperedge.

4.3.2. *Task-Specific Feature Transition.* In this step, we first calculate the attention between the users' preferences and the task-specific item features from each auxiliary pretext task. Then we generate task-aware hyperedge embeddings accordingly. Inspired by self-attention mechanism [64], the attention is calculated based on the hyperedge embedding and the task-specific item embedding for item i from each auxiliary pretext task:

$$\begin{aligned} \mathbf{a}^{\epsilon} &= \tanh(\sum_{t \in T} \alpha^i_t \cdot \mathbf{e}^i_t), \\ \alpha^i_t &= \frac{\exp(\mathbf{e}^{\epsilon}_{t_{rec}} \cdot \mathbf{e}^i_t / \sqrt{d})}{\sum_{t^* \in T} \exp(\mathbf{e}^{\epsilon}_{t_{rec}} \cdot \mathbf{e}^i_{t^*} / \sqrt{d})}, \end{aligned} \tag{3}$$

where $\mathbf{a}^{\epsilon}$ is the task-aware hyperedge embedding. Embeddings $\mathbf{e}^i_t$ and $\mathbf{e}^i_{t^*}$ are embeddings of item node i in auxiliary pretext tasks t and t^*, respectively. Attention α^i_t represents the attention of the connected users, who are interested in this item, on its certain feature reflected by each task. The hyperbolic tangent $\tanh(\cdot)$ is used as the activation function. Next, we fuse $\mathbf{a}^{\epsilon}$ with $\mathbf{e}^{\epsilon}_{t_{rec}}$ by addition:

$$\mathbf{q}^{\epsilon} = \mathbf{e}^{\epsilon}_{t_{rec}} + \gamma \mathbf{a}^{\epsilon}, \tag{4}$$

where $\mathbf{q}^{\epsilon}$ is the fused hyperedge embedding, and γ is a scalar hyper-parameter to control the transition intensity. This step is demonstrated in ② of Figure 2.2(b).

4.3.3. *Node Update.* We update the user embedding by aggregating the fused embeddings from all its connected hyperedges. For a target user u, the aggregation step is formulated as

follows:

$$\mathbf{e}'^{u}_{t_{rec}} = \frac{1}{|\mathcal{N}_u|} \sum_{\epsilon \in \mathcal{N}_u} \mathbf{q}^{\epsilon}, \tag{5}$$

where $\mathbf{e}'^{u}_{t_{rec}}$ denotes the output user embedding, $\mathcal{N}_u$ is the set of hyperedges connected to this user, and $\mathbf{q}^{\epsilon}$ is the fused embedding of its connected hyperedges from Eq. (4). This step is demonstrated in Figure 2.2(b)③.

4.3.4. *TA Layer in Matrix Form.* To offer a holistic view, we formulate the matrix form of the transitional attention layer (equivalent to Eqs. (2)-(5)) as:

$$\begin{aligned} \mathbf{E}'^{\mathbf{u}}_{t_{rec}} &= \mathrm{TA}(\mathbf{E}^{\mathbf{u}}_{t_{rec}}, \mathbf{H}^{\mathbf{u}}_{t_{rec}}, \mathbf{E}^{\mathbf{i}}_{t}) = (\mathbf{D}^{\mathbf{u}}_{t_{rec}})^{-1}\mathbf{H}^{\mathbf{u}}_{t_{rec}} \cdot \mathbf{Q}^{\epsilon} \\ \mathbf{Q}^{\epsilon} &= (\mathbf{E}^{\epsilon}_{t_{rec}} + \gamma \cdot \tanh(\mathrm{softmax}(\frac{\mathbf{E}^{\epsilon}_{t_{rec}}(\mathbf{E}^{\mathbf{i}}_{t})^{\top}}{\sqrt{d}})\mathbf{E}^{\mathbf{i}}_{t})), \\ \mathbf{E}^{\epsilon}_{t_{rec}} &= (\mathbf{B}^{\mathbf{u}}_{t_{rec}})^{-1}(\mathbf{H}^{\mathbf{u}}_{t_{rec}})^{\top}\mathbf{E}^{\mathbf{u}}_{t_{rec}}, \end{aligned} \tag{6}$$

where $\mathbf{E}^{\mathbf{u}}_{t_{rec}}$ is the input embedding from l-th layer, $\mathbf{H}^{\mathbf{u}}_{t_{rec}}$ is the incidence matrix, $\mathbf{E}^{\mathbf{i}}_{t}$ denotes task-specific item features from auxiliary pretext tasks, and $\mathbf{E}'^{\mathbf{u}}_{t_{rec}}$ is the output. $\mathbf{D}^{\mathbf{u}}_{t_{rec}}$ is the degree matrix of nodes and $\mathbf{B}^{\mathbf{u}}_{t_{rec}}$ is degree matrix of hyperedges. First, we aggregate the preferences of users to the items they are interested in. Next, we calculate the attention to reflect the relevance between the user preferences and the item features related to each task. At last, the task-aware user preferences are aggregated back to each user to support recommendation.

4.4. Prediction and Optimization. We conduct prediction and optimization for different tasks depending on their types. For bipartite relation prediction, i.e., the recommendation task, we calculate the ranking score $y^{(u,i)}_{t_{rec}}$ of the user-item pair (u, i) by the inner product:

$$y^{(u,i)}_{t_{rec}} = \mathbf{e}^{u}_{t_{rec}} \cdot \mathbf{e}^{i}_{t_{rec}}, \tag{7}$$

where $\mathbf{e}^u_{t_{rec}}$ and $\mathbf{e}^i_{t_{rec}}$ are user and item embeddings output by the last TA layer. We apply the alignment loss [66] to optimize the recommendation:

$$\mathcal{L}_{rec} = \sum_{(u,i)\in\mathcal{D}_r} \|\mathbf{e}^u_{t_{rec}} - \mathbf{e}^i_{t_{rec}}\|^2, \tag{8}$$

where $\mathcal{D}_r = \{(u,i)|i \in G^+_u\}$ is the user-item interaction data for pretraining.

For homogenous relation and node attribute prediction, since hyperedges represent relations and attributes, we use the inner product between the node embedding and the corresponding hyperedge embedding as the prediction score:

$$y^{(v,\epsilon)}_t = \mathbf{e}_{(v,t)} \cdot \mathbf{e}^\epsilon_t, \tag{9}$$

where $\mathbf{e}_{(v,t)}$ and $\mathbf{e}^\epsilon_t$ are the task-related node embedding and the hyperedge embedding for a specific auxiliary pretext task t, as calculated in Eq. (1). Then we adopt the pairwise Bayesian Personalized Ranking (BPR) loss [54] to optimize the prediction:

$$\mathcal{L}_t = \sum_{(v,\epsilon,\epsilon')\in\mathcal{D}_a} -\log\sigma(y^{(v,\epsilon)}_t - y^{(v,\epsilon')}_t), \tag{10}$$

where $\mathcal{D}_a = \{(v,\epsilon,\epsilon')|\epsilon \in G^+_a, \epsilon' \in G_a \backslash G^+_a\}$ is the pretraining data for each auxiliary pretext task.

Finally, we jointly optimize the recommendation pretext task and the auxiliary pretext tasks as follows:

$$\mathcal{L} = \beta \cdot \mathcal{L}_{rec} + (1-\beta) \cdot \sum_{t\in T} \mathcal{L}_t + \lambda_\Theta \|\Theta\|^2_2, \tag{11}$$

where Θ is all trainable parameters in the framework, which is regularized by λ_Θ, and β is a coefficient to balance the loss of recommendation and auxiliary tasks. Adam [33] is chosen as the optimizer.

The trainable parameters of UPRTH are only the initial user and item embeddings, i.e., $\Theta = \{\mathbf{E}^\mathbf{u}_{t_{rec}} \cup \mathbf{E}^\mathbf{i}_{t_{rec}}\}$. In other words, the complexity of encoding task hypergraphs is $\mathcal{O}(|V|^2)$ as the standard matrix factorization [18]. Without additional trainable parameters, the

Dataset	XMarket-CN	XMarket-MX	Steam
# users	18,806	221,890	50,292
# items	5,937	35,235	1,809
# user-item edges	23,065	305,569	65,379
# auxiliary tasks	3	4	4
# node attributes	11,874	70,470	215,285
# homogeneous edges	318	8,212	6,398

TABLE 2.1. Statistics of datasets for UPRTH.

proposed TA layer with the softmax dot-product attention takes $\mathcal{O}(|V|^2)$. Thus, the model complexity is $\mathcal{O}(|V|^2)$, which is as efficient as the matrix factorization.

5. Experiment

5.1. Experiment Settings.

5.1.1. *Datasets.* We conduct experiments on three real-world datasets, namely XMarket-CN, XMarket-MX, and Steam. XMarket[1] is a publicly available large-scale dataset obtained from Amazon. We use the data from China and Mexico for our experiments. For these two datasets, XMarket-CN and XMarket-MX, item categories and average rating of items are predicted as node attributes in auxiliary pretext tasks. Items bought together are regarded as items in complementary relations, which are predicted as homogeneous edges. In addition, substitute relations between items compared together are predicted as homogeneous edges for XMarket-MX. Steam [48] includes users' transaction records on the Steam online game store. Categories, publishers of the games, and groups of users are used for attribute prediction in auxiliary pretext tasks. Friendships among users are predicted as homogeneous edges. The statistics of three datasets are shown in Table 2.1. For Steam and XMarket-CN, we randomly select 80% of user-item interactions for training in both pretraining and finetuning and the remaining 20% for testing in finetuning. For XMarket-MX, the split ratio is 70% for training and 30% for testing.

5.1.2. *Baselines.* As UPRTH is a pretraining framework, we compare the performance of graph-based recommendation models with and without using user and item embeddings pretrained by UPRTH, including LightGCN [18], GCN [34], DirectAU [66], UltraGCN [44],

[1]`https://xmrec.github.io/`

HGNN [13], HCCF [79] and DHCF [27]. We also compare UPRTH with the following pretraining baselines:

- **GCC** [51]. This self-supervised pretraining framework leverages contrastive learning with random walks to capture structure information in graphs.
- **SGL-PreRec**. In SGL-Pre [77], edge drop as data augmentation is used for contrastive learning in pretraining. To further differentiate it from GCC, we jointly pretrain the model via both the contrastive learning task loss and the recommendation task.
- **AttriMask** [23]. This method applies GNN to obtained node embeddings and then adds a linear model to predict masked attributes during pretraining.

In finetuning of pretraining baselines and UPRTH, we deploy one hypergraph convolution layer to encode user and item embeddings on the hypergraph of the recommendation task, and use BPR loss for optimization. Our source code and the three datasets are released online.[2]

Dataset	XMarket-CN				XMarket-MX				Steam			
Metric	R@10	R@20	N@10	N@20	R@10	R@20	N@10	N@20	R@10	R@20	N@10	N@20
LightGCN	0.0036	0.0068	0.0015	0.0024	0.0123	0.0181	0.0073	0.0089	0.0715	0.1075	0.0358	0.0448
DirectAU	0.0030	0.0040	0.0014	0.0017	0.0098	0.0160	0.0055	0.0072	0.0574	0.0860	0.0290	0.0362
UltraGCN	0.0031	0.0053	0.0017	0.0023	0.0017	0.0026	0.0009	0.0011	0.0416	0.0787	0.0206	0.0299
HGNN	0.0061	0.0074	0.0029	0.0033	0.0095	0.0154	0.0046	0.0062	0.0780	0.1104	0.0418	0.0502
HCCF	0.0023	0.0038	0.0008	0.0012	0.0077	0.0122	0.0043	0.0055	0.0976	0.1453	0.0489	0.0609
DHCF	0.0033	0.0047	0.0022	0.0026	0.0139	0.0230	0.0073	0.0097	0.1214	0.1959	0.0566	0.0753
GCC	0.0112	0.0177	0.0068	0.0086	0.0229	0.0318	0.0146	0.0169	0.1147	0.1833	0.0565	0.0738
SGL-PreRec	0.0036	0.0079	0.0014	0.0025	0.0102	0.0153	0.0058	0.0071	0.1120	0.1796	0.0537	0.0708
AttriMask	0.0387	0.0693	0.0179	0.0261	0.0500	0.0748	0.0261	0.0327	0.1398	0.2247	0.0714	0.0929
UPRTH	**0.0801**	**0.1093**	**0.0428**	**0.0506**	**0.0822**	**0.0966**	**0.0574**	**0.0612**	**0.1721**	**0.2704**	**0.0841**	**0.1088**
Improv.	106.98%	57.72%	139.11%	93.87%	64.40%	29.14%	119.92%	87.16%	23.10%	20.34%	17.79%	17.12%

TABLE 2.2. Overall performance comparison of UPRTH with baselines.

5.1.3. *Evaluation Metrics.* We evaluate the pretraining framework by ranking all non-interacted items for each test user during finetuning. Recall@$\{10, 20\}$ and NDCG@$\{10, 20\}$ are adopted as evaluation metrics.

5.2. Overall Performance. We show overall comparison results in Table 2.2. The best and second-best results are in boldface and underlined respectively. We summarize the following key observations:

[2]`https://github.com/mdyfrank/UPRTH`

- The proposed UPRTH framework achieves the best results and outperforms all the baseline pretraining frameworks in the three datasets. We hypothesize that these consistent gains result from the abundant and balanced contextual information learned from unified pretext tasks.
- Pretraining frameworks achieve better performance than recommendation models without pretraining in most cases. However, pretraining based on only user-item interactions sometimes degrades the performance. Besides sparse data providing limited prior knowledge during pretraining, we believe that overfitting caused by optimizing the same recommendation objective in both pretraining and finetuning also leads to performance degradation.
- Among the three baseline pretraining frameworks, AttriMask, which additionally considers attributes in pretraining, is better than GCC and SGL-PreRec in all datasets. This justifies the necessity of capturing contextual cues besides user-item interactions in pretraining. Compared with it, UPRTH realizes unification across various pretext tasks so it obtains the capacity to transfer knowledge from a wider range of tasks in pretraining, which is a more comprehensive framework.

Dataset	XMarket-CN		XMarket-MX		Steam	
Metric	R@10	N@10	R@10	N@10	R@10	N@10
LightGCN	0.0036	0.0015	0.0123	0.0073	0.0715	0.0358
+UPRTH	0.0679	0.0393	0.0680	0.0407	0.1522	0.0740
DirectAU	0.0030	0.0014	0.0098	0.0055	0.0574	0.0290
+UPRTH	0.0612	0.0352	0.0714	0.0490	0.1473	0.0727
UltraGCN	0.0031	0.0017	0.0017	0.0009	0.0416	0.0206
+UPRTH	0.0607	0.0412	0.0644	0.0383	0.1049	0.0460
HGNN	0.0061	0.0029	0.0095	0.0046	0.0780	0.0418
+UPRTH	0.0510	0.0264	0.0411	0.0195	0.0951	0.0475
HCCF	0.0023	0.0008	0.0077	0.0043	0.0976	0.0489
+UPRTH	0.0341	0.0223	0.0376	0.0234	0.1143	0.0556
DHCF	0.0033	0.0022	0.0139	0.0073	0.1214	0.0566
+UPRTH	0.0562	0.0322	0.0382	0.0220	0.1408	0.0625

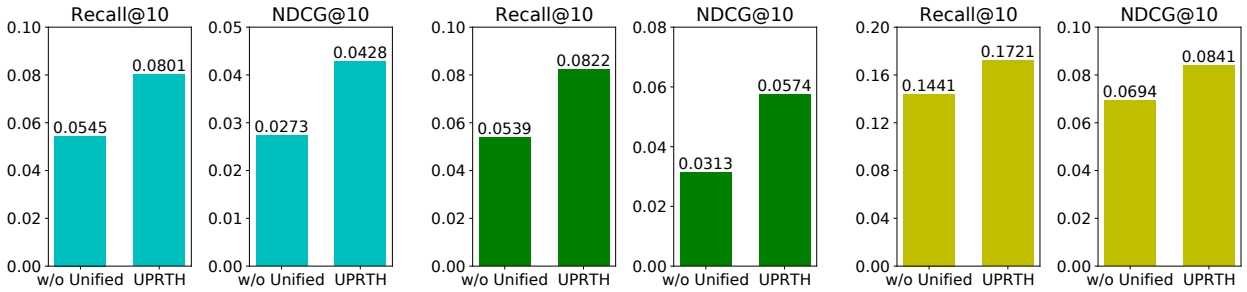
TABLE 2.3. Performance of recommendation models with and without embeddings from UPRTH.

Next, we show the performance of baseline recommendation models with and without leveraging pretrained embeddings from UPRTH in Table 2.3. The results indicate that all recommendation models benefit from embeddings pretrained by UPRTH, especially in the

XMarket-CN and XMarket-MX datasets with sparser user-item interactions. It indicates that our framework can adapt well to various downstream recommendation models, showing robustness and flexibility in its pretrained embeddings. The benefit on sparse datasets reflects that unified multitask pretraining efficiently addresses the issue of data sparsity in the recommendation task.

5.3. Ablation Study.

5.3.1. *Unification on Pretext Tasks.* We compare the performance of UPRTH and its variant without unified pretext tasks on the three datasets in Figure 2.3. In the variant without unified pretext tasks, relations are still predicted as hyperedges in relation prediction tasks, but attributes are predicted via applying one linear layer followed by a softmax activation function on node embeddings output by hypergraph encoders. We can observe that the performance is consistently improved by a unified pretraining framework on all three datasets. We attribute this improvement to the complete transfer of prior knowledge from our unified multitask pretraining. Given that no linear transformation is included in hypergraph encoders, and the prediction of hyperedges is directly calculated based on the node embeddings, prior knowledge from a pretext task is completely preserved in node embeddings and transferred to the downstream recommendation task. Thus, we believe UPRTH is a concise and efficient design to unify pretext tasks.



(A) XMarket-CN (B) XMarket-MX (C) Steam

FIGURE 2.3. Performance of UPRTH with and without unified pretraining.

5.3.2. *Transitional Attention Layer Settings.* To verify the effectiveness of TA, we demonstrate the performance of UPRTH and three other variants in Table 2.4: (a) UPRTH w/o

TA. In this setting, embeddings output by hypergraph encoders are directly used to calculate ranking scores of user-item pairs; (b) UPRTH-sum. Instead of calculating the attention of the hyperedge embedding on node embeddings from different auxiliary tasks, the sum of the hyperedge embedding and those node embeddings are used as the fused hyperedge embedding for further node update; (c) UPRTH-concat. Same as UPRTH-sum, but we concatenate node embeddings from different auxiliary tasks and transform it to an embedding whose dimension is the same as the hyperedge embedding by a linear layer, and then add it to the hyperedge embedding as the fused hyperedge embedding.

We observe that UPRTH performs best on all three datasets. This justifies the design of UPRTH with TA layers, which discriminatively integrate information from auxiliary tasks to recommendation. The poor performance of UPRTH-concat verifies the negative impact brought by the additional linear transformation in pretraining. UPRTH-sum performs better than UPRTH w/o TA in three datasets, which indicates the necessity of considering contextual information from other tasks in the pretext recommendation task. However, UPRTH-sum ignores the different relevance between each auxiliary task and recommendation, so it is worse than UPRTH.

Dataset	XMarket-CN		XMarket-MX		Steam	
Metric	R@10	N@10	R@10	N@10	R@10	N@10
UPRTH w/o TA	0.0744	0.0411	0.0798	0.0548	0.1645	0.0795
UPRTH-sum	0.0770	0.0413	0.0818	0.0547	0.1690	0.0825
UPRTH-concat	0.0800	0.0424	0.0796	0.0470	0.1235	0.0599
UPRTH	**0.0801**	**0.0428**	**0.0822**	**0.0574**	**0.1721**	**0.0841**

TABLE 2.4. Ablation study on the proposed TA Layer in UPRTH.

5.4. Pretraining and Finetuning Loss Analysis. As aforementioned, UPRTH applies alignment loss in pretraining and BPR loss in finetuning to optimize the recommendation task. We further investigate different combinations of loss for pretraining and finetuning stages. In addition, we try AU loss [66] which optimizes both alignment and uniformity, and BPR_{pos} loss which only optimizes for positive instead of both positive and negative pairs in BPR loss. The performances are shown in Table 2.5. We observe that using two different loss functions for pretraining and finetuning improves performance in most cases, since the same

loss functions increase the risk of overfitting. Moreover, the results show that employing BPR loss in finetuning is better than in pretraining. We believe that exposing different pretext tasks to different objective functions results in more robust pretrained representations for downstream tasks.

Dataset	XMarket-CN		XMarket-MX		Steam	
($\mathcal{L}_{\text{Pretrain}}$, $\mathcal{L}_{\text{Finetune}}$)	R@10	N@10	R@10	N@10	R@10	N@10
(BPR, BPR)	0.0527	0.0284	0.0707	<u>0.0439</u>	0.1053	0.0527
(BPR_{pos}, BPR_{pos})	0.0231	0.0114	0.0632	0.0354	0.1106	0.0564
(AU, AU)	0.0460	0.0238	0.0159	0.0084	0.1152	0.0568
(Align., Align.)	0.0373	0.0237	0.0110	0.0059	0.0180	0.0069
(BPR, BPR_{pos})	0.0519	0.0283	0.0633	0.0384	0.1205	0.0601
(BPR, AU)	0.0574	0.0284	0.0643	0.0407	0.1158	0.0575
(BPR, Align.)	0.0575	0.0281	0.0317	0.0125	0.1062	0.0543
(BPR_{pos}, BPR)	<u>0.0623</u>	0.0301	0.0704	0.0419	<u>0.1338</u>	<u>0.0669</u>
(AU, BPR)	0.0623	<u>0.0346</u>	<u>0.0712</u>	0.0418	0.1075	0.0545
(Align., BPR)	**0.0801**	**0.0427**	**0.0822**	**0.0579**	**0.1721**	**0.0795**

TABLE 2.5. Performance of UPRTH w.r.t. objective functions in pretraining and finetuning.

5.5. Cold-Start Recommendation. As an inductive framework, UPRTH has the capability of handling the cold-start problem by leveraging contextual information from auxiliary tasks. We conduct an analysis regarding the ability of UPRTH to tackle users who have no historical user-item interactions in both pretraining and finetuning. We randomly select a certain ratio of user nodes as cold-start users and remove all user-item edges connected to them. The embeddings of cold-start users are obtained by feeding them with the removed edges to the downstream encoder during only inference. For comparison, we report testing recall of UPRTH, AttriMask and UPRTH without auxiliary pretext tasks. The results are shown in Figure 2.4.

We observe that UPRTH and AttriMask perform better than UPRTH without auxiliary tasks, which verifies the importance of contextual information from multitask pretraining in cold-start recommendation. UPRTH outperforms AttriMask in most cases since more diverse contextual cues are captured for recommending items to cold-start users by unified multitask pretraining. Additionally, we observe that sometimes the performance on cold-start users is even better than the average performance on all users shown in Table 2.2,

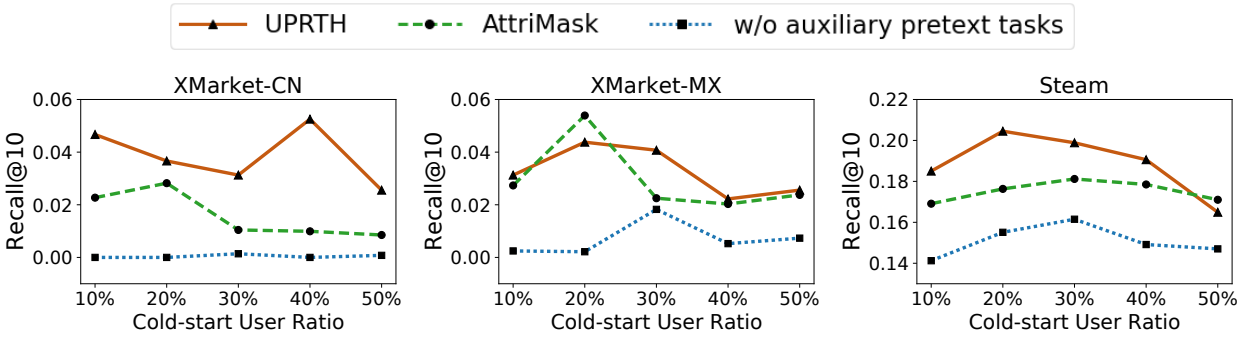


FIGURE 2.4. Performance of UPRTH on inductive users.

such as cold-start user ratio $r = 20\%$ in the Steam dataset. We hypothesize that when cold-start users are pretrained with only social relations, such as user-user and user-group hyperedges, UPRTH can well leverage user similarity for recommendation without the noise in the user-item interaction information.

5.6. Performance Curves Analysis. To verify that the pretrained embeddings with prior knowledge transferred from various pretext tasks can benefit downstream recommendation, we conduct analyses of testing performance curves with and without embeddings pretrained by UPRTH, as shown in Figure 2.5. For the downstream model, we choose the one-layer hypergraph convolution encoder used in UPRTH, denoted as HyperConv, with BPR loss for recommendation. Performances of SGL-PreRec, GCC and AttriMask are shown for comparison. The result indicates that UPRTH and AttriMask, which leverage contextual information besides user-item interactions from pretraining, outperform the other baselines. Compared with AttriMask, UPRTH discriminatively learns from various pretext tasks with a unified multitask framework, leading to more comprehensive and informative pretrained embeddings for downstream recommendation.

6. Chapter Summary

In this chapter, we present Unified Pretraining for Recommendation via Task Hypergraphs (UPRTH), a hypergraph-based pretraining framework for recommendation. UPRTH

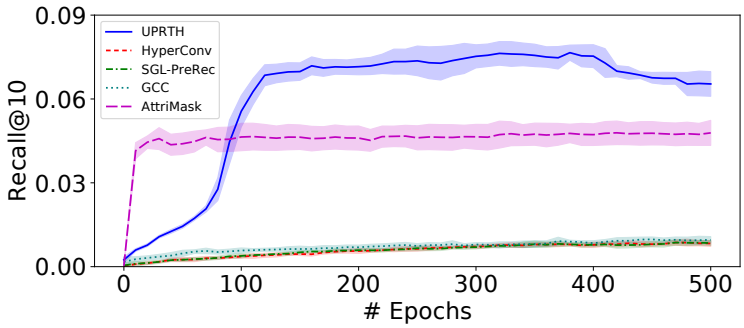


FIGURE 2.5. Performance curves of UPRTH and other baselines.

models diverse pretext tasks as hyperedge prediction problems within a unified task hypergraph, enabling the integration of heterogeneous relational signals into a shared structural representation. Through hypergraph learning, the framework captures higher-order relations and transfers prior knowledge to the downstream recommendation task. In addition, a Transitional Attention (TA) layer is designed to adaptively learn the relevance between auxiliary tasks and the target recommendation objective during pretraining. Extensive experiments on three benchmark datasets demonstrate the effectiveness and robustness of UPRTH.

Moreover, this chapter establishes hypergraph learning as a flexible foundation for modeling complex relations in recommendation. The flexibility of hyperedge construction and information propagation further opens the door to incorporating semantic knowledge beyond structural signals. In the next chapter, we extend this framework by introducing plain-text prompts that explicitly describe task semantics and domain connections, and encode them with a pretrained language model to guide hypergraph pretraining for cross-domain recommendation.

CHAPTER 3

Hypergraph Pretraining Guided by PLM-Encoded Textual Prompts

(This chapter was previously published as "Instruction-based Hypergraph Pretraining" [86] in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 501 - 511. 2024. [doi:10.1145/3626772.3657715])

1. Introduction

Chapter 2 shows that hypergraphs offer a flexible structure for unifying heterogeneous relations in recommendation, where the relevance of each pretext task is inferred purely from structural signals, and this chapter extends that idea to cross-domain settings by introducing plain-text instructions that describe task semantics explicitly and injecting them into hypergraph pretraining. The burgeoning field of graph-based deep learning has witnessed remarkable advancements, with applications spanning diverse domains such as social networks, bioinformatics, and RecSys. GNNs have emerged as powerful tools for capturing intricate relationships and dependencies within graph-structured datasets. To transfer knowledge from large and diverse datasets to a downstream task with limited data, pretraining has been widely explored to augment the adaptability of GNNs [24, 51]. However, pretraining also presents several critical issues. Self-supervised learning for pretraining graph models [51, 23] addresses the lack of labeled data in the pretext stage, but the training objective gap between the constructed pretext and dedicated downstream tasks hinders the efficient transfer of pretrained knowledge [61]. Moreover, when labeled data is available during the pretraining stage, the data distribution discrepancy between pretraining and downstream datasets degrades the performance [69]. In addition, pretraining can suffer from catastrophic forgetting [103], resulting in poor generalization ability, especially within small-scale downstream datasets.

Prompt learning, which has been prevalently engaged in Pretrained Language Models (PLMs) [38, 3], has shown remarkable success in dealing with the aforementioned pretraining issues. Nonetheless, it is far from straightforward to apply prompts in graphs. In contrast with PLMs, the format of instructions as plain text does not naturally align with the graph-structure data. On the one hand, from a static perspective, graph prompts are supposed to be applied for a certain part of the graph to guide the model depending on specific graph queries. Some specific tasks, such as group identification, can correspond to a relationship among multiple entities. However, traditional graphs are limited in their ability to represent complex relationships, as each edge can only connect two nodes. On the other hand, from a dynamic perspective, prompts need to participate in the context-aware information propagation of the graph model to help capture contextual dependencies among relationships and entities. Nevertheless, dyadic edges in traditional graphs can only propagate information through simple pairwise relationships, which is neither flexible nor efficient. Therefore, we believe that the hypergraph, where each hyperedge is able to connect multiple nodes [13, 102] simultaneously, is a better structure for prompt-based pretraining in graph-based tasks.

However, it is challenging to design the prompt for hypergraph pretraining. Most pioneering works in prompt-based graph pretraining design prompt functions with learnable prompt vectors, which are initialized randomly [62, 40] or based on pretrained node representations [61]. Such prompts are not related to semantic instructions for specific tasks, and the effectiveness heavily depends on the quality and diversity of the finetuning data in downstream tasks. The learnable prompt vectors can bring uncertainty in outcomes and the risk of poor generalization, particularly when finetuning data is biased or scarce. In addition, most previous works freeze all parameters except learnable prompts during downstream tasks [62, 40, 61]. However, this paradigm is suboptimal if abundant unseen nodes appear in the downstream task. The frozen nodes pretrained by pretext tasks cannot offer a direct reference to represent unseen nodes in the downstream task, and the uncertainty from learnable prompts can be exacerbated with those unseen data. These limitations hinder the implementation of these pretraining frameworks in graph-based applications, given that new

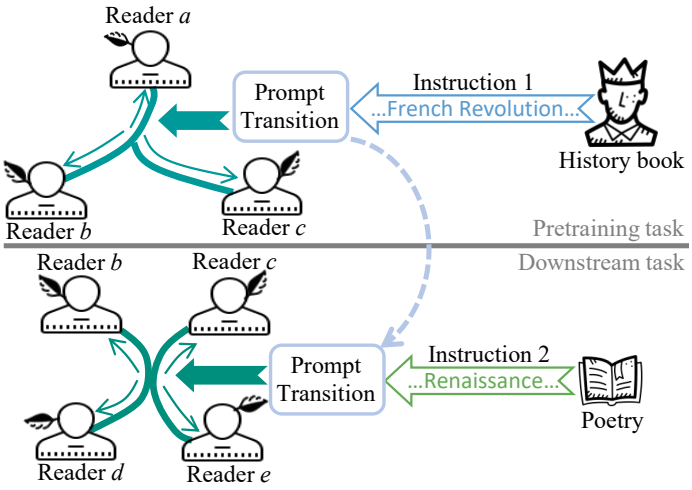


FIGURE 3.1. A toy example of how IHP benefits recommendation.

nodes frequently appear in dynamically evolving graph structures such as social networks, biological networks, and RecSys.

In light of the above limitations in the existing prompt learning methods for graph-based tasks, it is necessary to design prompts that can provide explicit guidance to the model, helping it focus on specific aspects of the task. To this end, instruction-based prompts represent a more advantageous solution [59, 3, 63]. Such prompts guide the model through explicit instructions, which are able to accurately describe specific task requirements related to graph data. By training the model with a variety of instructions, it can adapt to different graph-related queries, improving its generalization across a range of scenarios. In the situation with biased or scarce data, compared to learnable prompts which may overfit the limited examples, instructions allow for the incorporation of domain-specific knowledge that might not be present in the data itself. Instruction-based prompts are also beneficial for addressing unseen nodes in downstream tasks. Rather than relying solely on previously seen examples, text-based instructions provide side information on the new nodes appearing in the graph. Meanwhile, this adaptation to the unseen graph structure requires no extensive computational resources for re-training prompts.

In this chapter, we propose a novel framework called Instruction-based Hypergraph Pretraining (IHP). In this framework, high-order relations are represented as hyperedges, and the dependencies among nodes are captured under the guidance of instructions. We show an

example of how to integrate instructions into hypergraph pretraining in Figure 3.1. If three history enthusiasts read a history book about the French Revolution, we can use one hyperedge to connect them and prompt the hyperedge with instruction through pretraining. In the following downstream task of promoting poetry from the Renaissance period to readers, the model can accordingly locate the readers similar to these three history enthusiasts as the target audiences according to the instruction. To learn node representations under the guidance of instructions, we design a Prompt Hypergraph Convolution (PHC) layer. This PHC layer endows the framework with the ability to be prompted by text-based instructions through hyperedges. Within the hypergraph learning framework, each task-related instruction transformed into prompts is precisely directed to the respective nodes interconnected by the hyperedge. Participating in hypergraph learning, instructions enable the framework to capture high-order relations with task information in a context-aware manner. Furthermore, we design an instruction-based finetuning paradigm. In the downstream task, we freeze the prompt transformation process and update the representations of seen and unseen nodes according to the instructions, ensuring that relationships among nodes are captured in the context of the overall graph. Node representations are updated with different adaptation intensities to achieve a balance between retraining prior knowledge and adapting efficiently for downstream tasks. The main contributions of this work are summarized as follows.

- We design a novel instruction-based pretraining framework based on hypergraphs. To the best of our knowledge, this is the first pretraining framework to leverage instructions to capture high-order relations for graph tasks like cross-domain recommendation.
- We propose a PHC layer that injects text-based instructions into hyperedges, enabling semantic task information to guide higher-order relational learning. This PHC layer allows instruction information to participate in information propagation during hypergraph convolution, enhancing the flexibility of the hypergraph learning and the generalization of the pretrained model.
- We conduct extensive experiments on three real-world datasets. The consistent improvements observed in the performance of IHP verify its effectiveness.

2. Related Work

2.1. Prompt Learning on Graphs. Pretraining has been widely adopted to transfer knowledge from large graphs to downstream tasks with limited data [24, 51, 22]. Since a gap usually exists between the pretext objective and the downstream objective, prompt learning has been introduced to graphs following its success in language models [38, 3]. GPPT [61] reformulates node classification as link prediction and inserts learnable prompt vectors into pretrained representations, GraphPrompt [40] unifies pretraining and downstream tasks through a shared subgraph similarity template, and All-in-One [62] aligns multiple task types with a unified prompt graph. These methods rely on randomly initialized or representation-based prompt vectors that carry no explicit task semantics, and most of them freeze all parameters except the prompts during finetuning, which restricts their ability to represent unseen nodes.

2.2. Instructions as Semantic Guidance. Instruction-based prompts describe task requirements in plain text and therefore convey explicit guidance that learnable vectors cannot express [59, 63]. Sentence encoders such as Sentence-BERT [52] and MiniLM [71] map such text into dense embeddings that downstream models can consume directly. In recommendation, textual side information has also been used to relieve sparsity when preferences are transferred across domains [74]. Different from these works, we encode instructions that describe both node attributes and task semantics, and inject them into hyperedges so that they participate in information propagation instead of serving as static input features.

3. Preliminaries

3.1. Problem Formulation. Let the graph for the pretraining task be $G = (V, E)$ where V and E are sets of nodes and edges, respectively. Similarly, let the graph for the downstream task be $G' = (V', E')$, and the two instruction sets be I_p and I_d for the pretraining task and the downstream task. Our target is to pretrain the representations of the overlapping nodes, which are defined as target nodes $\mathcal{V}_\mathrm{t} = V \cap V'$. Given the target node set $\mathcal{V}_\mathrm{t}$, we define the nodes other than target nodes in the pretraining task as pretraining

context nodes $\mathcal{V}_{\mathrm{c}} = V \setminus \mathcal{V}_{\mathrm{t}}$ in the pretraining task, and the unseen nodes in the downstream task as downstream context nodes $\mathcal{V}'_{\mathrm{c}} = V' \setminus \mathcal{V}_{\mathrm{t}}$.

Although the proposed framework is applicable to general graph learning tasks, we focus on the recommendation scenario in this dissertation. Under a cross-domain recommendation setting, the target nodes correspond to the overlapping users shared across domains, while the context nodes represent domain-specific items. Specifically, $\mathcal{V}_c$ denotes source-domain items used during pretraining, and $\mathcal{V}'_c$ denotes target-domain items introduced during finetuning. We further denote the textual descriptions of source-domain items and target-domain items as $\mathcal{A}$ and $\mathcal{A}'$, respectively, and use $\mathcal{T}$ to represent domain-related task descriptions. Our objective is to pretrain user representations by modeling their interactions with source-domain items, and subsequently adapt these representations through finetuning on target-domain items to achieve accurate recommendation performance in the downstream task.

4. Proposed Framework: IHP

In this section, we present the proposed IHP framework with the illustration demonstrated in Figure 3.2. We start by introducing all the embeddings to be pretrained in this framework. Thereafter, we define two types of hypergraphs and explain how to construct corresponding hyperedges in these hypergraphs. Specifically, we adopt a PHC layer to prompt instructions into hyperedges during hypergraph learning in the pretraining and finetuning stages. We apply an instruction-based finetuning paradigm to update both seen and unseen nodes in the downstream task.

4.1. Embedding Layer. We maintain an embedding layer $\mathbf{E} \in \mathbb{R}^{(|\mathcal{V}_{\mathrm{t}}|+|\mathcal{V}_{\mathrm{c}}|)\times d}$ during pretraining, where d is the feature dimension and rows represent all trainable node embeddings. The initial target node embeddings are denoted as $\mathbf{E}_{\mathrm{t}}$, and the initial context node embeddings are denoted as $\mathbf{E}_{\mathrm{c}}$. Since target nodes exist in pretraining and downstream tasks, prior knowledge learned from the pretraining tasks can be preserved in $\mathbf{E}_{\mathrm{t}}$ and then leveraged in the downstream tasks.

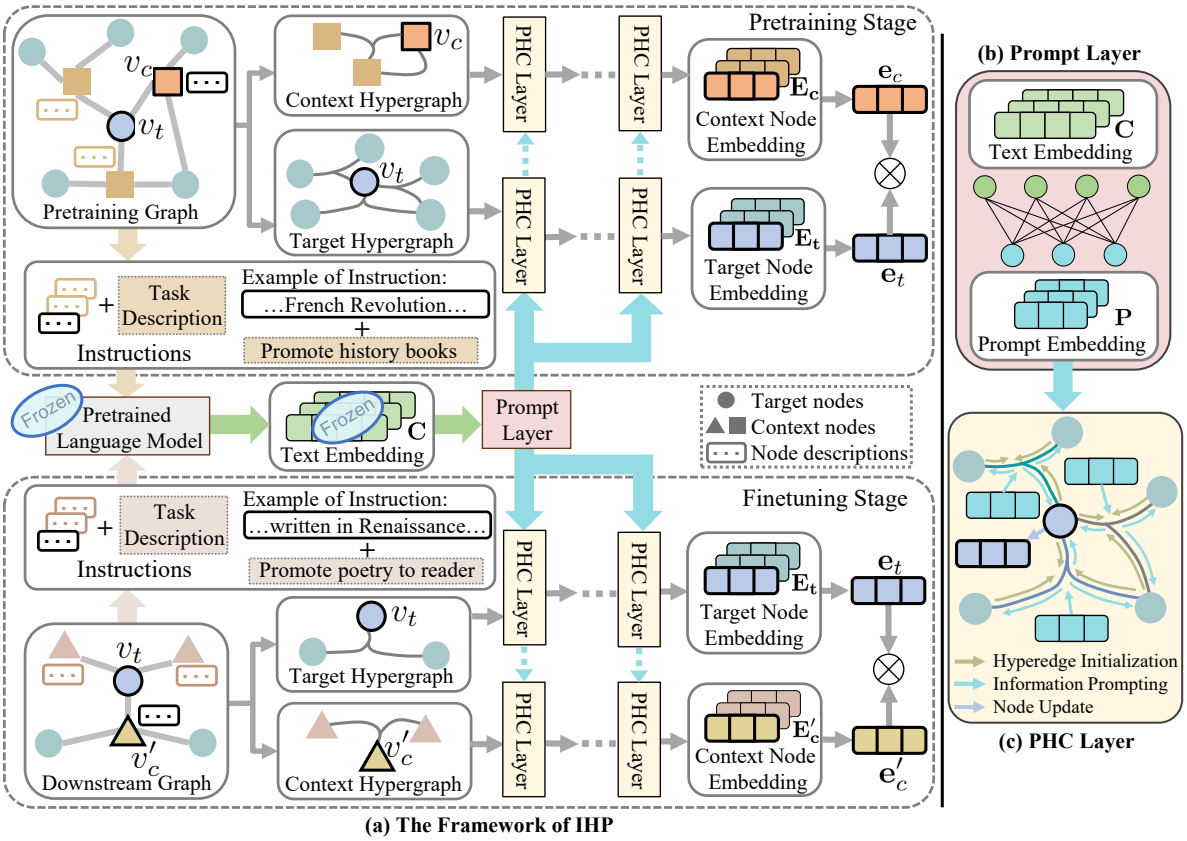


FIGURE 3.2. The illustration of IHP with the proposed Prompt and PHC layers.

4.2. Hypergraph Construction. We construct the target hypergraph and the context hypergraph based on the original graph. Compared to a graph where edges only connect two nodes, the advantage of a hypergraph is that hyperedges can simultaneously connect multiple nodes as the objectives of an instruction. According to the type of nodes in the graph, we construct two hypergraphs based on four basic types of hyperedges as below:

- **Target Hypergraph**. The target hypergraph $\mathcal{H}_{\mathrm{t}}$ consists of target nodes $\mathcal{V}_{\mathrm{t}}$ and hyperedges $\mathcal{E}_{\mathrm{t}}$. Each target-target hyperedge is used to connect a target node with its one-hop target node neighbors. For each group of target nodes connected to one context node, a target-context hyperedge is used to connect them in the hypergraph. Its incidence matrix is denoted as $\mathbf{H}_{\mathrm{t}}$.
- **Context Hypergraph**. Similar to the target hypergraph, the context hypergraph $\mathcal{H}_{\mathrm{c}}$ consists of context nodes $\mathcal{V}_{\mathrm{c}}$ and hyperedges $\mathcal{E}_{\mathrm{c}}$. Each context-context hyperedge is used

to connect a context node with its one-hop context node neighbors. A context-target hyperedge is used to connect each group of context nodes to one target node simultaneously in the hypergraph. Its incidence matrix is denoted as $\mathbf{H}_{\mathrm{c}}$.

In other words, we use the two hypergraphs to ensure the distinction between the target nodes and the context nodes. In this way, the framework can learn from the homogeneity of target nodes and context nodes in a discriminative manner during the following hypergraph pretraining. On the one hand, the information from pretraining is preserved in the target node embeddings and directly transferred to the downstream task. On the other hand, although context nodes are not in the downstream task, they provide the necessary information for the model to capture broader contextual patterns that are not limited to the nodes seen during the downstream task. The relation between target nodes can depend on the similarity among context nodes in pretraining, which is reflected by the context hypergraph.

Similarly, a target hypergraph $\mathcal{H}'_{\mathrm{t}} = (\mathcal{V}_{\mathrm{t}}, \mathcal{E}'_{\mathrm{t}})$ and a context hypergraph $\mathcal{H}'_{\mathrm{c}} = (\mathcal{V}'_{\mathrm{c}}, \mathcal{E}'_{\mathrm{c}})$ are defined for the downstream task with their corresponding incidence matrices $\mathbf{H}'_{\mathrm{t}}$ and $\mathbf{H}'_{\mathrm{c}}$. For pretraining and downstream tasks, the isolation between the two hypergraphs separates the information propagation paths. In this way, the information of context nodes will not be directly aggregated to the target nodes through hyperedges in hypergraph learning, and vice versa. This design prevents the over-smoothing issue [1, 32] among target and context nodes, preventing the representations of target nodes from converging to the same values, especially within dense graph data.

4.3. Prompt Hypergraph Convolution. In hypergraph learning, hypergraph convolution is prevalently used for information propagation across nodes in hypergraphs, as formulated in Section 3.4. Nonetheless, current hypergraph convolution methods focus on the direct aggregation of structural information from neighbors connected by hyperedge. To integrate the information from instructions into the hypergraph convolution process, we devise a novel Prompt Hypergraph Convolution layer. First, we construct the hyperedge information based on connected nodes. This step is denoted as *hyperedge initialization* of the PHC layer. Then, an *information prompting* step is proposed to endow the fusion of

hyperedge information from resources other than the hypergraph structure, such as instructions. Finally, fused information is aggregated to update the node embeddings in the *node update* step. To be more concrete, we present the calculation details of a PHC layer in the target hypergraph. The illustration is in Figure 3.2(c).

4.3.1. *Hyperedge Initialization.* In the target hypergraph $\mathcal{H}_{\mathrm{t}}$, target nodes are connected by target-target and target-context hyperedges. Therefore, we initialize the representation of a hyperedge by aggregating embeddings of all nodes connected by this hyperedge. The initialized representation of a hyperedge $\epsilon \in \mathcal{E}_{\mathrm{t}}$ is formulated as follows:

$$\mathbf{h}_{\epsilon}^{(l)} = \frac{1}{|\mathcal{N}_{\epsilon}|} \sum_{v \in \mathcal{N}_{\epsilon}} \mathbf{e}_{v}^{(l)}, \tag{12}$$

where $\mathbf{h}_{\epsilon}^{(l)}$ represents initialized embedding of the hyperedge ϵ, $\mathcal{N}_{\epsilon}$ is the set of target nodes connected by this hyperedge, and $\mathbf{e}_{v}^{(l)}$ is the embedding of those target nodes as input of the l-th PHC layer. We use mean-pooling as the information aggregation method to aggregate structural information from nodes to hyperedges.

4.3.2. *Information Prompting.* To enable the framework to capture high-order relations under the guidance of instructions during hypergraph convolution, we fuse the initialized representation of a hyperedge with prompted information. With the prompt embedding of a hyperedge ϵ denoted as $\mathbf{p}_{\epsilon}$, we fuse $\mathbf{h}_{\epsilon}^{(l)}$ with $\mathbf{p}_{\epsilon}$ by addition:

$$\mathbf{q}_{\epsilon}^{(l)} = \mathbf{h}_{\epsilon}^{(l)} + \gamma \mathbf{p}_{\epsilon}^{(l)}, \tag{13}$$

where $\mathbf{q}_{\epsilon}^{(l)}$ is the fused hyperedge embedding, and γ is a scalar hyperparameter to control the prompting intensity. The target hypergraph and the context hypergraph each carry their own prompting intensity, which are tuned independently.

With instructions accessible, the prompt embedding can be obtained based on text information from instructions, as shown in Figure 3.2(b) and 3.2(c). If the instruction information is not available, the flexibility of this PHC layer allows other information to be prompted to the hyperedge. For example, a context-target hyperedge in the context hypergraph is converted from a target node. Hence, we can adopt prompt embeddings from the target node

embeddings for a context hypergraph without instructions, i.e., $\mathbf{P}^{(l)} = \mathbf{E}_{\mathrm{t}}^{(l+1)}$, as shown in Figure 3.2(a). The target node embeddings $\mathbf{E}_{\mathrm{t}}^{(l+1)}$ are output by the l-th PHC layer applied on the target hypergraph, after node update in Eq. (14).

4.3.3. *Node Update.* We update the target node embedding by aggregating the fused embeddings from all its connected hyperedges. For a target node v_t, the aggregation step is formulated as follows:

$$\mathbf{e}_t^{(l+1)} = \frac{1}{|\mathcal{N}_t|} \sum_{\epsilon \in \mathcal{N}_t} \mathbf{q}_\epsilon^{(l)}, \tag{14}$$

where $\mathbf{e}_t^{(l+1)}$ denotes the output target node embedding, $\mathcal{N}_t$ is the set of hyperedges connected to this target node, and $\mathbf{q}_\epsilon^{(l)}$ is the fused embedding of its connected hyperedges from Eq. (13).

4.3.4. *PHC in Matrix Form.* To offer a holistic view of convolution, we formulate the matrix form of prompt hypergraph convolution (equivalent to Eq. (12)-(14)) as:

$$\begin{aligned} \mathbf{E}_{\mathrm{t}}^{(l+1)} &= \mathrm{PHC}(\mathbf{E}_{\mathrm{t}}^{(l)}, \mathbf{H}_{\mathrm{t}}, \gamma\mathbf{P}^{(l)}) \\ &= \mathbf{D}^{-1}\mathbf{H}_{\mathrm{t}} \cdot (\mathbf{B}^{-1}\mathbf{H}_{\mathrm{t}}^{\top}\mathbf{E}_{\mathrm{t}}^{(l)} + \gamma\mathbf{P}^{(l)}), \end{aligned} \tag{15}$$

where $\mathbf{E}_{\mathrm{t}}^{(l)}$ is the input embedding from l-th layer, $\mathbf{H}_{\mathrm{t}}$ is the incidence matrix, $\mathbf{P}^{(l)}$ denotes prompt embeddings for the hyperedges in the hypergraph, and $\mathbf{E}_{\mathrm{t}}^{(l+1)}$ is the output. $\mathbf{D}$ is the degree matrix of nodes and $\mathbf{B}$ is degree matrix of hyperedges for normalization. PHC layer is a more general version of hypergraph convolution, which degrades to existing hypergraph convolution [94] with prompting intensity $\gamma = 0$.

4.4. Instruction-based Prompt Embedding. To provide explicit guidance for hypergraph learning, instruction-based prompt embeddings are constructed based on available task-related information and fed into the PHC layer for further information propagation. Each instruction is expected to assist in capturing the relationship between the nodes connected by one hyperedge. Hence, the instructions and the formulated hyperedges are designed in one-to-one correspondence. Instructions can be formulated from task descriptions, node descriptions, or a concatenation of both.

We show an example of leveraging the concatenation of task and node descriptions as instructions in Figure 3.2. Given that target-context hyperedges are context nodes in the original graph, we concatenate the node descriptions $\mathcal{A}_c$ and the descriptions $\mathcal{T}_c$ of tasks related to these nodes as instructions $\mathcal{I}_c$, where $|\mathcal{I}_c| = |\mathcal{E}_{t,c}|$. In this chapter, descriptions of context nodes are obtained from the dataset, and the task descriptions are manually created for different tasks. For instance, if target nodes are readers and context nodes are history books in pretraining, the task description can be formulated as 'promote history books to readers.'

After the construction of instructions, a pretrained sentence transformer [71] is deployed to encode the instructions as text embeddings $\mathbf{C} \in \mathbb{R}^{|\mathcal{I}_c| \times d'}$, which are frozen in both pretraining and downstream tasks. A linear layer is applied to adaptively learn the transformation from text embeddings to prompt embeddings:

$$\mathbf{P} = \mathbf{C} \cdot \mathbf{W}_{\mathrm{p}} + \mathbf{b}_{\mathrm{p}}, \tag{16}$$

where $\mathbf{W}_{\mathrm{p}} \in \mathbb{R}^{d' \times d}$ and $\mathbf{b}_{\mathrm{p}} \in \mathbb{R}^{d}$ are the learnable weight matrix and bias for prompt transformation. And $\mathbf{P}$ is the output prompt embedding, which prompts the target-context hyperedges with instruction information in every PHC layer for the target hypergraph.

4.5. Pretraining and Optimization.

4.5.1. *Prediction and optimization.* We use the inner product of two nodes as the link prediction score. For example, in Figure 3.2(a), the link prediction score $y_{t,c}$ between a target node and a context node in pretraining is formulated as:

$$y_{t,c} = \mathbf{e}_t \cdot \mathbf{e}_c, \tag{17}$$

where $\mathbf{e}_t$ and $\mathbf{e}_c$ are the final embeddings of the target node and the context node after PHC layers. We remove the superscripts for simplicity. Then, we adopt the pairwise BPR loss [54]

to optimize the prediction:

$$\mathcal{L}_{\text{bpr}} = \sum_{(u,v,v^-)\in\mathcal{D}} -\log\sigma(y_{u,v} - y_{u,v^-}) + \lambda_\Theta\|\Theta\|_2^2, \tag{18}$$

where $\mathcal{D} = \{(u, v, v^-)|v \in G_u^+, v^- \in V\backslash G_u^+\}$ is the training data, and positive set G_u^+ contains all nodes connected to node u. All parameters Θ in the pretraining are regularized by λ_Θ. Adam [33] is used as the optimizer.

4.5.2. *Instruction-based finetuning.* The output Θ of the pretraining stage is the optimal target node embeddings $\mathbf{E}_{\text{t}}$ and all parameters $\Theta_{\text{p}} = \{\mathbf{W}_{\text{p}}, \mathbf{b}_{\text{p}}\}$ in the prompt layer. This pretrained parameter Θ is used to initialize the target nodes and prompt layer in the downstream task. In the downstream task, we freeze Θ_{p} in the prompt transformation and further finetune the embeddings of pretrained target nodes and unseen context nodes with downstream instructions. The reasons are twofold. Firstly, freezing this transformation ensures that the model responds to instructions consistently in both pretext and downstream tasks. Since the task-related information is inherently encapsulated in instructions, it is unnecessary to finetune the transformation for learning the task-related information. Second, updating the unseen context nodes with the pretrained target nodes ensures that their features and relationships are captured in the context of the overall graph. This allows for dynamic adaptation of node representations to the specific context of the downstream task, guaranteeing that all nodes are represented according to the downstream instructions.

To address the catastrophic forgetting issue in finetuning, we deploy an adaptation intensity coefficient λ_{t} for the learning rate η_{t} of target node embeddings. We denote the learning rate of context node embeddings denoted as η_{c} and set $\eta_{\text{t}} = \lambda_{\text{t}} \cdot \eta_{\text{c}}$. A lower learning rate for target nodes prevents the framework from forgetting the prior knowledge preserved in the target node embeddings. In this way, the pretrained model achieves a balance between retraining prior knowledge and effectively adapting to downstream tasks.

4.5.3. *Efficiency.* Our framework is efficient in both space and time complexity. Node embeddings $\mathbf{E}$, prompt transformation matrix $\mathbf{W}_{\text{p}}$ and prompt transformation bias $\mathbf{b}_{\text{p}}$ are the only learnable parameters in IHP. Given the size of pretrained text embeddings as $\mathcal{O}(|V|)$ for

Dataset	Goodreads-P	Goodreads-H	Amazon
# nodes	69,511	220,704	362,900
# edges	370,326	1,673,926	726,531
# target nodes	10,000	10,000	22,899
# pretrained nodes	52,698	163,752	342,738
# pretrained edges	271,344	1,407,108	665,695
# node descriptions	59,511	210,704	340,001

TABLE 3.1. Statistics of datasets for IHP.

Dataset	Goodreads-P				Goodreads-H				Amazon			
Metric	R@10	R@20	N@10	N@20	R@10	R@20	N@10	N@20	R@10	R@20	N@10	N@20
LightGCN	0.2427	0.2958	0.1684	0.1846	0.0956	0.1111	0.0818	0.0869	0.0661	0.0828	0.0497	0.0545
SGL	0.2365	0.2935	0.1637	0.1808	0.0960	0.1108	0.0810	0.0856	0.0683	0.0812	0.0494	0.0530
DirectAU	0.2106	0.3152	0.1721	0.2011	0.0933	0.1084	0.0807	0.0861	0.0689	0.0808	0.0599	0.0630
HCCF	0.2427	0.3175	0.1884	0.2092	0.0721	0.1062	0.0603	0.0735	0.0682	0.0809	0.0605	0.0639
DHCF	0.2440	0.2995	0.1702	0.1846	0.0735	0.0919	0.0569	0.0636	0.0299	0.0399	0.0177	0.0207
GraphFormers	0.2147	0.2770	0.1388	0.1583	0.0889	0.1187	0.0757	0.0854	0.0401	0.0496	0.0285	0.0313
AttriMask	-	-	-	-	0.0640	0.0960	0.0489	0.0600	0.0741	0.0871	0.0633	0.0670
GCC	0.2585	0.3185	0.1933	0.2117	0.0623	0.0947	0.0465	0.0584	0.0749	0.0905	0.0611	0.0656
GraphMAE	0.2574	0.3220	0.1984	0.2178	0.0678	0.1020	0.0557	0.0678	0.0768	0.0920	0.0585	0.0627
IHP	**0.2782**	**0.3380**	**0.2189**	**0.2351**	**0.1032**	**0.1319**	**0.0915**	**0.1005**	**0.0814**	**0.0980**	**0.0692**	**0.0738**
Improv.	7.63%	4.99%	10.31%	7.93%	7.49%	11.07%	11.77%	15.71%	5.97%	6.59%	9.29%	10.17%

TABLE 3.2. Overall performance comparison of IHP with baselines.

pretraining and $\mathcal{O}(|V'|)$ for finetuning, the total space complexity is $\mathcal{O}(|V|d+|V|d'+d'd)$ for pretraining and $\mathcal{O}(|V'|d+|V'|d'+d'd)$ for finetuning, where $\forall x \in \{d, d'\}, \forall y \in \{|V|, |V'|\}, x \ll y$. The time complexity depends on the interaction between hyperedges and nodes and the number of PHC layers L. For the four types of hyperedges constructed in IHP, the number of interactions is $\mathcal{O}(|E|)$ for pretraining and $\mathcal{O}(|E'|)$ for finetuning. Therefore, the time complexity is only $\mathcal{O}(L|E|d + d'd)$ for pretraining and $\mathcal{O}(L|E'|d)$ for finetuning, where $\forall x \in \{d, d'\}, \forall y \in \{|E|, |E'|\}, L \ll x \ll y$. In contrast, a typical graph model GCN [34] needs $\mathcal{O}(L|V|d^2 + L|E|d + |V|d)$ time with $|V|$ nodes and $|E|$ edges.

5. Experiment

5.1. Experiment Settings.

5.1.1. *Datasets.* We conduct experiments on three real-world datasets: Goodreads-P, Goodreads-H, and Amazon. Goodreads is a publicly available large-scale dataset including information about online readers and books on their shelves [65]. We regard online readers as the target nodes for the two Goodreads datasets. For Goodreads-P, poetry books are

used as context nodes for pretraining, and comics are used as downstream context nodes. For Goodreads-H, history books, and biographies are downstream context nodes, while pretraining context nodes contain books in the categories of children, young adults, mystery, thriller, and crime. Amazon includes users and items purchased by them [46]. Users are regarded as target nodes, and instruments are used as context nodes in the downstream task. For pretraining in Amazon, context nodes are items in the categories of arts, crafts, sewing, grocery, gourmet food, office products, electronics, sports, outdoors, toys, and games. The main statistics of the three datasets are summarized in Table 3.1. We split 70% of edges for training and the remaining 30% for testing.

5.1.2. *Baselines.* We regard users as target nodes and items as context nodes in all the datasets. Thus recommendation methods, including LightGCN [18], SGL [77], HCCF [79] and DHCF [27], are applied as strong baselines to prediction interactions between users and items, besides baseline models designed for general graph tasks, such as DirectAU [66] and GraphFormers [83]. As a pretraining framework, we also compare IHP with the following pretraining baselines:

- **AttriMask** [23]. This method applies GNN to obtained node embeddings and then adds a linear model to predict masked attributes during pretraining. We use node categories as the masked attributes, so it is not suitable for Goodreads-P, where all pretraining context nodes are poetry books.
- **GCC** [51]. This self-supervised pretraining framework leverages contrastive learning with random walks to capture structure information in graphs.
- **GraphMAE** [22]. This method applies a masked graph autoencoder for generative self-supervised graph pretraining.

For the pretraining baselines, we use the same finetuning stage as IHP, which is shown in Figure 3.2(a), with no instructions and instruction prompting intensity $\gamma = 0$.

5.1.3. *Evaluation metric.* We evaluate the pretraining framework by ranking all non-interacted downstream items for each test user during finetuning. Recall@$\{10, 20\}$ and NDCG@$\{10, 20\}$ are adopted as evaluation metrics.

TABLE 3.3. The total training time of IHP.

Dataset	Goodreads-P	Goodreads-H	Amazon
Pretraining time (s)	7.80	5.60	23.79
Finetuning time (s)	10.61	28.78	24.11

5.1.4. *Implementations.* For all the datasets, the pretraining learning rate is set to 0.1. The regularization parameter λ_Θ is set to 1e-7. The node embedding size is set to 64. The learning rate to finetune context nodes is set to 0.1 for the two Goodreads datasets and 0.01 for Amazon. For the Goodreads-P/Goodreads-H/Amazon dataset, we set the number of PHC layers as 4/1/3, the prompting intensity in the target hypergraph as 1e-3/1e-4/1e-1, the prompting intensity in the context hypergraph as 1/1/1e-4, and the adaptation intensity as 1e-2/1e-3/10. We implement early stopping based on loss for pretraining and Recall@10 for finetuning. All the experiments are conducted on an 8GB RTX 2080. As Table 3.3 shows, the framework can be pretrained and finetuned within one minute for all the datasets, benefiting from the compact size of learnable parameters and the high efficiency. Our implementation is available at `https://github.com/mdyfrank/IHP`.

5.2. Overall Performance. We show the overall performance comparison in Table 3.2. The best results are in boldface. We observe that the proposed IHP framework achieves the best results and outperforms all the baselines in the three datasets. We hypothesize these large and stable gains result from prior knowledge learned from pretraining under the explicit guidance of instructions. In addition, baseline pretraining frameworks achieve better performance than models without pretraining in Goodreads-P and Amazon. However, they are surpassed by other baselines in Goodreads-H. Since the pretrained edges in Goodreads-H are densest, and pretrained node types in it are fewer than in Amazon, the risk of overfitting for baseline pretraining frameworks is high in such a dataset. In contrast, instructions provide explicit information about certain tasks, which explains IHP's better generalization and adaptation for the downstream task.

5.3. Ablation Study.

Dataset	Goodreads-P		Goodreads-H		Amazon	
Metric	R@10	N@10	R@10	N@10	R@10	N@10
w/o PLM	0.2717	0.1974	0.0944	0.0824	0.0545	0.0406
Instructor [59]	0.2763	0.2225	0.1019	0.0905	0.0787	0.0662
GTR [47]	0.2764	0.2226	0.1020	0.0900	0.0793	0.0665
S-BERT [52]	0.2765	0.2218	0.1024	0.0903	**0.0831**	**0.0698**
E5 [68]	0.2764	**0.2227**	0.1023	0.0909	0.0808	0.0675
MiniLM [71]	**0.2782**	0.2189	**0.1032**	**0.0915**	0.0814	0.0692

TABLE 3.4. Performance of IHP with different text encoders.

5.3.1. *PLMs as text encoders.* To quantify the contribution of PLMs as the text encoder in our framework, we compare the link prediction performance of IHP with different PLMs and without PLM in Table 3.4. For the variant of IHP without PLM, we randomly initialize the text embedding as the input of the prompt layer. It is apparent that employing PLM to encode instructions leads to a clear improvement in all the datasets. The results also imply that the benefits of different PLMs depend on the data distribution of the certain dataset. In IHP, we use MiniLM as the sentence encoder because of its stable performance in all the datasets.

5.3.2. *Task information in Instructions.* In addition to node descriptions, the instructions in IHP also contain task descriptions (e.g., "promote electronics to the reader"). In Figure 3.3, we demonstrate the performance of IHP with different ways of leveraging task information as a part of the instruction. In addition to concatenating node descriptions and task information for text encoding, we investigate three other variants of IHP: (N) Without text-based task information, only node descriptions are encoded as text embeddings and then concatenated to a learnable task vector for each task before prompt transformation; (S1) Node descriptions and task information are encoded by the PLM separately and then concatenated for prompt transformation; (S2) Same as S1, but the encoded task embeddings skip both prompt transformation and hypergraph convolution. Instead, they are concatenated to corresponding node embeddings after PHC layers for the final prediction.

We observe that IHP performs best on all three datasets. This justifies the design of IHP, which simultaneously encodes node and task information inside instructions. The poor performance of N verifies the limitation of learnable vectors compared to text-based task descriptions in instruction construction. S1 allows task information to participate in



FIGURE 3.3. Performance of IHP w.r.t. different instruction constructions.

information propagation through PHC layers, which results in better performance than S2 in Goodreads datasets. When tasks become diverse in Amazon, S2 allocating task information to corresponding nodes can more accurately capture dependencies among nodes. However, separately encoding node descriptions and task information ignores the semantic connection between them, so both S1 and S2 are worse than IHP.

5.3.3. *Instruction-based finetuning settings.* The prior knowledge is preserved in target node embeddings $\mathbf{E}_{\mathrm{t}}$ and parameters of prompt layer Θ_{p} in the pretraining stage and transferred to the downstream tasks. Hence, we conducted an ablation study to investigate the influence of different finetuning settings on downstream performance. For instruction-based finetuning in IHP, we freeze the prompt layer and update the target node embeddings with context node embeddings during downstream tasks. We compare IHP with the other four variants of finetuning settings: (i) IHP-TR with $\mathbf{E}_{\mathrm{t}}$ randomly initialized in finetuning; (ii) IHP-TF with pretrained $\mathbf{E}_{\mathrm{t}}$ frozen in finetuning; (iii) IHP-PR with Θ_{p} randomly initialized in finetuning; (iv) IHP-PU with pretrained Θ_{p} updated by the objective in finetuning.

The comparison between the performance of IHP and its variants in Figures 3.4 and 3.6 confirms the advantage of our instruction-based finetuning paradigm. Freezing target node embeddings reduces the flexibility and limits the adaptability to downstream data for the pretraining framework. This issue becomes more pronounced with the growing discrepancy between pretrained and downstream data. As shown in Figure 3.4, IHP-TF with frozen target node embeddings can hardly adapt to the downstream instrument data from the pretrained item data from diverse categories in Amazon. Additionally, updating the prompt layer

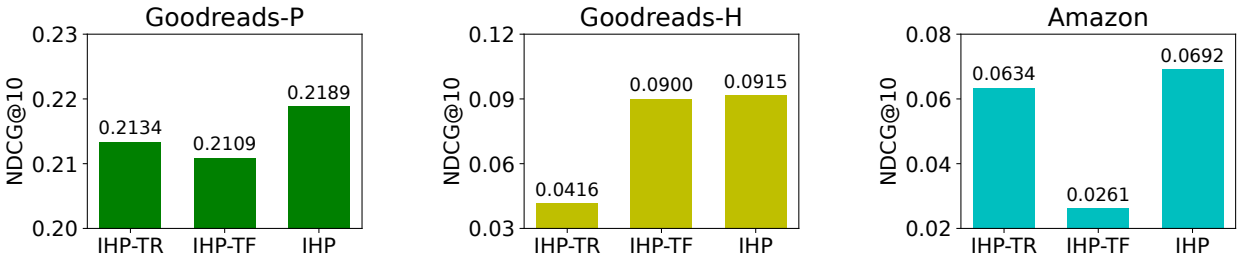


FIGURE 3.4. Performance of IHP with different target node finetuning strategies.

scarcely improves the downstream performance, compared with using randomly initialized parameters in Figure 3.6. On the contrary, IHP with a frozen prompt layer maintains the consistency of the model's response to instructions during pretraining and finetuning and thus achieves a better performance.

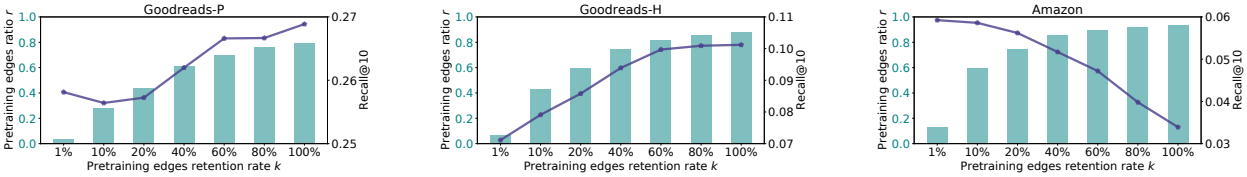


FIGURE 3.5. IHP finetuned with pretrained data added with $\lambda_t = 1$.

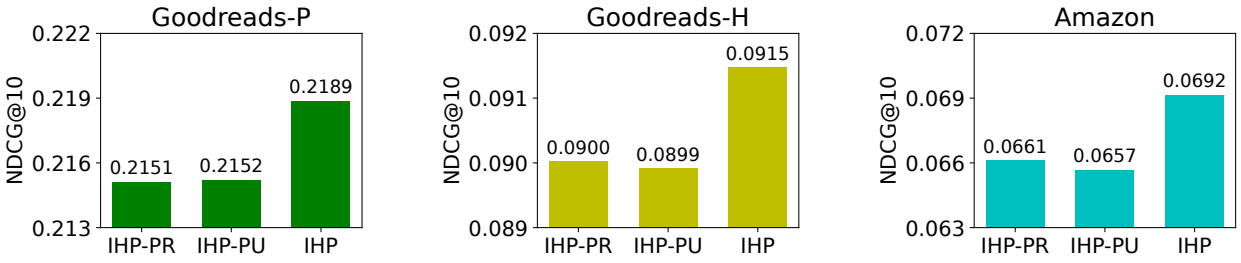


FIGURE 3.6. Performance of IHP w.r.t. prompt layer finetuning strategies.

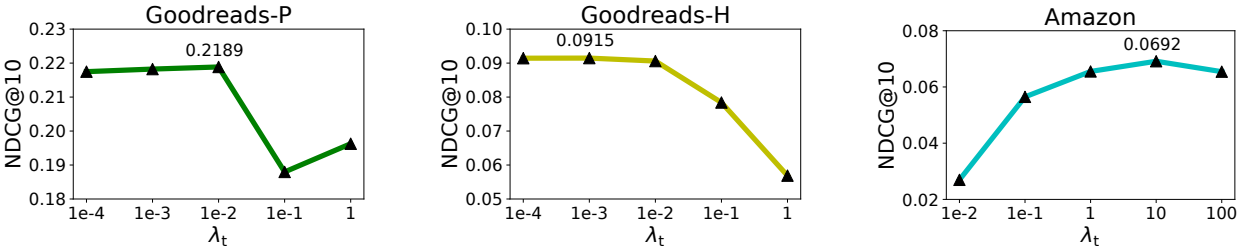


FIGURE 3.7. Performance of IHP w.r.t. adaptation intensity coefficient λ_t.

Dataset	Goodreads-P		Goodreads-H		Amazon	
Metric	R@10	N@10	R@10	N@10	R@10	N@10
GraphSage	0.2415	0.1988	0.0305	0.0210	0.0330	0.0166
AttriMask	-	-	0.0605	0.0489	0.0887	0.0505
GCC	0.2601	0.1960	0.0589	0.0483	0.0929	0.0520
GraphMAE	0.2620	0.1981	0.0606	0.0503	0.0869	0.0494
IHP	**0.2622**	**0.2002**	**0.0998**	**0.0783**	**0.0939**	**0.0672**

TABLE 3.5. Performance of IHP on inductive user nodes.

5.4. Inductive Learning Analysis. To validate the effectiveness of IHP in dynamically evolving graph structures, we set inductive users which account for 25%, 40%, and 50% of the original users into Amazon, Goodreads-P, and Goodreads-H datasets. The embeddings of inductive user nodes are never trained in either pretraining or finetuning stages. Instead, the inductive nodes are connected to other downstream training nodes by hyperedges defined in Section 4.2 and fed into IHP only for inference. Then, we evaluate the link prediction performance on those inductive nodes. In addition to the pretraining baselines [23, 51, 22], we also compare IHP with GraphSage [17], which is a general inductive graph learning method.

The results are demonstrated in Table 3.5. One can observe that IHP exhibits the best performance on inductive users in all the datasets, especially in larger Goodreads-H and Amazon datasets. With prior knowledge from abundant pretrained data in these two datasets, pretraining baselines always perform better than scratch-training GraphSage. Nonetheless, they are worse than IHP, which better captures dependencies between inductive and trained nodes by directly prompting text-based instruction to hyperedges.

5.5. Adaptation Intensity Analysis. To maintain the balance between retaining prior knowledge and adapting for various downstream tasks, we deploy the adaptation intensity coefficient λ_t to differentiate the learning rates of target and context node embeddings during finetuning. We demonstrate the impact of this coefficient on three datasets in Figure 3.7. It becomes obvious that the performance is always suboptimal when the learning rates of target and context node embeddings remain the same, i.e., $\lambda_t = 1$. For the two Goodreads datasets, a small λ_t prevents the framework from forgetting the prior knowledge

during finetuning. For Amazon, a large λ_t is required to endow the model with the higher adaptivity to the downstream data.

We believe the difference of λ_t in Goodreads and Amazon results from the discrepancy between the pretraining and downstream data in each dataset. For verification, we evaluate the model performance with the same learning rates of target and context nodes after merging the pretraining and downstream data. To be specific, we randomly select a subset E_s of edges from the set E of all the pretraining edges, add them to the downstream data, and then finetune the model with $\lambda_t = 1$ on the downstream tasks. In other words, the edges become $E_s \cup E'$ for the downstream tasks. The proportion of the selected edges in all the pretrained edges is defined as pretraining edges retention rate $k = |E_s|/|E|$. The ratio of these selected edges to all the downstream edges is denoted as pretraining edges ratio $r = |E_s|/(|E_s|+|E'|)$. Pretraining context nodes connected by E_s are also added to the downstream task. In this way, the model is finetuned directly with the original pretraining data.

The results are demonstrated in Figure 3.5. The curve represents the performance, and the bars denote the ratios of the edges added from pretraining in finetuning. In the two Goodreads datasets, we observe that the downstream performance is generally improved with the pretraining data added to the downstream task. In Amazon, on the contrary, the performance is degraded after adding the pretraining data when $\lambda_t = 1$. The reason is that all the pretraining and downstream context nodes are books in Goodreads, so the discrepancy between the pretraining and downstream data in Goodreads is minor compared to the discrepancy in Amazon, where pretraining and downstream context nodes are items in more diverse categories. That explains why a small λ_t is necessary to prevent the framework from forgetting the prior knowledge in Goodreads, while a large λ_t can help the pretrained model adapt faster to the considerably different downstream data in Amazon.

6. Chapter Summary

In this chapter, we propose a novel graph pretraining framework IHP, which applies text-based instructions to overcome the discrepancy between pretraining and downstream

recommendation tasks. In IHP, we construct hyperedges to capture high-order relations among nodes under the guidance of instructions, and a PHC layer is introduced to integrate instructions into context-aware information propagation in hypergraph learning. We conduct extensive experiments and detailed analyses on three real-world datasets to verify the effectiveness of IHP in cross-domain recommendation.

PLMs in this framework are primarily used to encode item and domain descriptions from instructions into semantic embeddings. Through these representations, abstract user preferences learned in the source domain can be semantically aligned with related items in the target domain. In this way, the prior knowledge embedded in PLMs serves as a semantic bridge between pretraining and downstream tasks, enabling cross-domain knowledge transfer beyond pure structural signals. However, this approach still relies on fixed-dimensional sentence-level embeddings produced by PLMs as encoders, which may limit the granularity and flexibility of semantic alignment. With the rapid advancement of LLMs and their increasingly powerful generative and representation capabilities, it becomes possible to move beyond static embedding extraction and achieve deeper integration between language models and RecSys. The next chapter moves beyond treating PLMs as external semantic encoders and instead introduces user and item tokens directly into the LLM vocabulary space. By aligning ID-based behavioral patterns with the language space at the token level, we aim to establish a tighter integration between pretrained linguistic knowledge and collaborative interaction signals in RecSys.

CHAPTER 4

Training Large Recommendation Models via Graph-Language Token Alignment

(This chapter was previously published as "Training Large Recommendation Models via Graph-Language Tokens Alignment" [90] in *Companion Proceedings of the ACM on Web Conference 2025*, pp. 1470 – 1474. 2025. [doi:10.1145/3701716.3715583])

1. Introduction

The instructions in Chapter 3 are encoded into fixed-dimensional sentence embeddings, which limits the granularity of semantic alignment between language and interaction signals, so this chapter pursues a tighter integration by aligning user and item representations with the token embeddings of an LLM, representing collaborative patterns and pretrained linguistic knowledge in a common space. Graph-based RecSys often struggle to integrate rich semantic information from textual data, limiting their ability to capture nuanced user preferences and item characteristics. Leveraging large language models (LLMs) can bridge this gap by enhancing understanding of textual data and improving recommendation accuracy [53]. A straightforward idea is to train an LLM on recommendation data to serve as a recommender [16]. While previous studies have demonstrated that LLMs can function as recommenders [6], they overlook that established graph-based recommendation models effectively leverage user-item interactions for collaborative filtering. Moreover, when the LLM generates language tokens as item predictions, the output free-form text introduces ambiguity when matching the actual items, compared to traditional recommendation models that output a clear list of item IDs. To mitigate this, additional post-processing is needed to parse text back to specific items, which hinders the actual performance when candidate items are similar. Instead of deploying LLMs directly as RecSys, some recent works integrate LLM embeddings as supplementary features to enhance existing recommendation models [53,

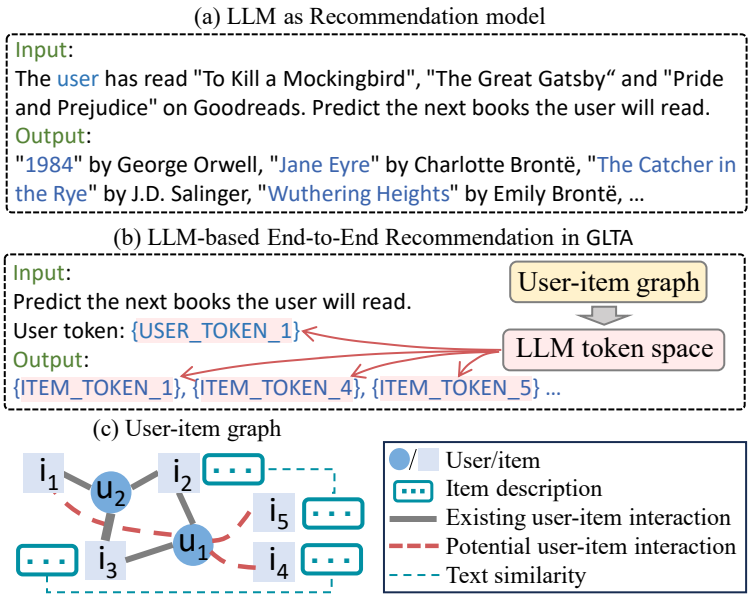


FIGURE 4.1. A toy example of the input and the output in GLTA.

75]. However, the recommendation process in their approaches is primarily driven by the graph-based model, leaving the reasoning capabilities of LLMs underutilized.

To effectively harness the powerful reasoning capability of LLMs for recommendation, we propose a novel framework, training large recommendation models via **G**raph-**L**anguage **T**oken **A**lignment, which aligns LLMs with graphs using a carefully designed token alignment paradigm. To be concrete, we first align items with their text descriptions to obtain item tokens, and then align users with pretrained item and text tokens for recommendation. To implement end-to-end recommendation, we design a Graph-Language Logits Matching (GLLM) layer to optimize the token alignment by matching predicted item logits with ground-truth items. This GLLM layer eliminates the hallucination issue that non-existing items are generated when directly using LLM outputs as item prediction results. As shown in Figure 4.1, different from directly adopting LLMs as recommendation models, our GLTA accurately generates existing item tokens instead of plain text outputs. The key contributions of this chapter are:

- We propose a novel recommendation framework, GLTA, integrating LLMs and recommendation in an end-to-end manner, where each output of the LLM corresponds precisely to

an item in RecSys, eliminating the hallucination issue and the ambiguity from free-form text as output.

- We adaptively align nodes pretrained on the graph with pretrained tokens of the LLM by token projectors, and introduce a novel GLLM layer for optimizing end-to-end recommendation based on these aligned node tokens. Only projectors and GLLM layers are finetuned for this efficient end-to-end prediction.
- To verify the preeminence of GLTA, we conduct extensive experiments on three publicly available benchmark datasets. The effectiveness of each component in GLTA is verified through ablation studies.

2. Related Work

2.1. LLMs as Recommenders. One line of work reformulates recommendation as a language modeling problem and trains an LLM to generate items directly. P5 [16] converts recommendation data into natural language sequences and unifies several tasks under a single text-to-text objective, and GraphGPT [63] further aligns graph structures with an LLM through instruction tuning. Aligning language models with user preferences has also been shown to improve their behavior as recommenders [6]. Because the output is free-form text, these methods require additional post-processing to map generated strings back to items, which becomes unreliable when candidate items share similar descriptions.

2.2. LLMs as Feature Encoders. A second line of work keeps a conventional recommendation model in place and uses the LLM only to produce semantic features. RLMRec [53] augments collaborative filtering with profile representations generated by an LLM, and LLMRec [75] uses LLM outputs to augment the interaction graph itself. In both cases the recommendation decision is still driven by the graph model, so the reasoning ability of the LLM is largely unused. Our framework instead aligns graph representations with the token space of the LLM, which keeps item prediction deterministic while allowing pretrained linguistic knowledge to take part in the prediction.

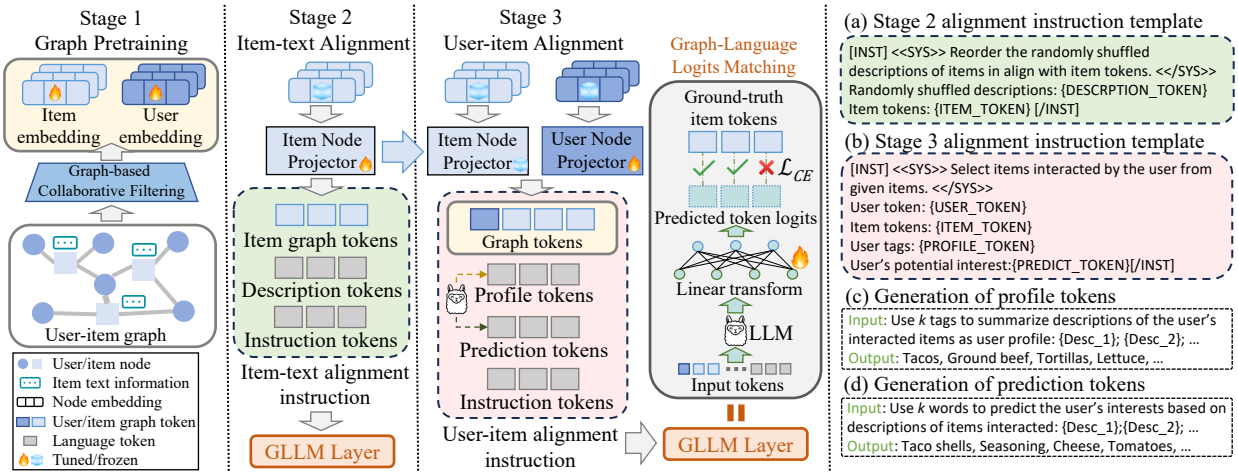


FIGURE 4.2. The framework of GLTA.

3. Preliminaries

3.1. Graph Pretraining. Graph-based CF enhances recommendation by using graph structures to model user-item interactions. Following the notation introduced in Chapter 2, we use LightGCN [18] as a graph-based CF method to capture and encode structure information on the user-item graph $G = (V, E)$. Node set $V = \mathcal{U} \cup \mathcal{I}$ contains all users $\mathcal{U}$ and items $\mathcal{I}$, and E denotes all the user-item interactions. In the first stage, user node embeddings and item node embeddings are pretrained as $\mathbf{E}_u \in \mathbb{R}^{|\mathcal{U}| \times d}$ and $\mathbf{E}_i \in \mathbb{R}^{|\mathcal{I}| \times d}$ and frozen in the following alignment stages, where d denotes the dimension of pretrained node embeddings.

4. Proposed Framework: GLTA

4.1. Item-text Alignment. The proposed GLTA is shown in Figure 4.2. Following graph pretraining, it is essential to align item node embeddings $\mathbf{E}_i$ with item descriptions tokens pretrained from the LLM. These two types of embeddings typically reside in different spaces. The item node embeddings capture collaborative signals from the graph, while the text embeddings capture semantic information from the textual descriptions. Inspired by GraphGPT [63], we apply a simple linear layer as an item node projector that maps these item nodes into the same language token space with the descriptions of these items:

$$\mathbf{V}_i = \mathbf{E}_i \mathbf{W}_i + \mathbf{b}_i, \tag{19}$$

where $\mathbf{V}_i \in \mathbb{R}^{|\mathcal{I}| \times d}$ is the embeddings of item tokens. $\mathbf{W}_i \in \mathbb{R}^{d \times d}$ and $\mathbf{b}_i \in \mathbb{R}^{1 \times d}$ denote the weight and bias of the item node projector. Then, an item-text alignment instruction template, shown in Figure 4.2 (a), queries the LLM to reorder the item language information and match the token with the predicted logits.

4.2. User-item Alignment. Besides the item-text alignment, a user-item alignment is proposed to allow the LLM to process both user and item information in the same context. Similarly, a linear layer is used as a user node projector to map user nodes into the LLM token space.

$$\mathbf{V}_u = \mathbf{E}_u \mathbf{W}_u + \mathbf{b}_u, \tag{20}$$

where $\mathbf{V}_u \in \mathbb{R}^{|\mathcal{U}| \times d}$ is the embeddings of user tokens. $\mathbf{W}_u \in \mathbb{R}^{d \times d}$ and $\mathbf{b}_u \in \mathbb{R}^{1 \times d}$ denote the weight and bias of the user node projector. This projector establishes the correspondence between the user nodes, the language tokens and the item tokens pretrained in previous item-text alignment.

Besides these user tokens and pretrained item tokens, we introduce profile tokens and prediction tokens into the user-item alignment instruction template. These profile and prediction tokens are generated by the LLM based on item descriptions. The instruction templates for generating profile and prediction tokens are shown in Figure 4.2(c) and Figure 4.2(d), respectively. In this way, user node embeddings are mapped to the same token space with semantic characteristics reflecting their historical interactions and potential preferences. Then, the user-item alignment instruction template is fed into the LLM to generate the predicted item tokens for each user. During training, the item token prediction is optimized by GLLM for end-to-end recommendation.

4.3. Graph-Language Logits Matching. In a traditional setup when using LLMs as RecSys, additional steps are required to parse the text and map it back to specific items [63, 16]. To eliminate the ambiguity that arises when interpreting free-form text outputs of the LLM, a GLLM layer is designed for the end-to-end recommendation in GLTA. After inputting the instruction template to the LLM, a linear layer is applied to transform the

Dataset	Goodreads	Amazon	MovieLens
#Users	10,131	1,032	6,040
#Items	10,725	17,609	3,706
#U-I interactions	478,334	30,510	1,000,209
Density	0.440%	0.168%	4.468%

TABLE 4.1. Statistics of the datasets for GLTA

last-layer hidden states of the LLM into item token logits $\mathbf{Z}_i \in \mathbb{R}^{L\times|\mathcal{I}|}$, where L denotes the maximum sequence length in the LLM. Then, a cross-entropy loss is applied to match the predicted logits to the ground-truth items interacted by the user:

$$\mathcal{L}_{CE} = -\frac{1}{L}\sum_{t=1}^{L}\log\left(\frac{\exp\left(\mathbf{Z}_i[t, y_{i,t}^+]\right)}{\sum_{j=1}^{|\mathcal{I}|}\exp\left(\mathbf{Z}_i[t,j]\right)}\right), \tag{21}$$

where $y_{i,t}^+$ is the ground-truth item ID at position t in the sequence, and $\mathbf{Z}_i[t, y_{i,t}^+]$ represents the logit corresponding to the ground-truth item at position t. This cross-entropy loss provides direct supervision by comparing the model's predicted probability distribution against the actual ground-truth item IDs. In real-world datasets, the number of items interacted by the user is not necessarily equal to L, and the order information of the interacted item $y_{i,t}^+$ can be unavailable. In that case, we only optimize item token logits in the first k positions, according to k ground-truth items randomly shuffled from all the items interacted by the user. In this work, we use a quantized version of LLaMA-2-7B[1] as the LLM for training efficiency and adopt Adam [33] as the optimizer.

5. Experiment

5.1. Experiment Settings.

5.1.1. *Datasets.* We conduct experiments on three publicly available datasets: Goodreads [65], Amazon [46] and MovieLens[2]. We use history books and groceries as items with their corresponding descriptions in Goodreads and Amazon datasets, respectively. For MovieLens, we regard movies as items and movie genres as item descriptions

[1] `https://huggingface.co/TheBloke/Llama-2-7B-GPTQ`
[2] `https://grouplens.org/datasets/movielens/1m/`

Dataset	Goodreads				Amazon				MovieLens			
Metric	P@5	P@10	N@5	N@10	P@5	P@10	N@5	N@10	P@5	P@10	N@5	N@10
LightGCN	0.1999	0.1618	0.2346	0.2352	0.0204	0.0139	0.0301	0.0305	0.0470	0.0426	0.0463	0.0466
HCCF	0.1998	0.1632	0.2371	0.2439	0.0209	0.0143	0.0275	0.0304	0.0467	0.0419	0.0482	0.0486
LightGCN+	0.2004	0.1616	0.2347	0.2358	0.0211	0.0150	0.0268	0.0290	0.0527	0.0451	0.0467	0.0469
HCCF+	0.2034	0.1641	0.2379	0.2483	0.0217	0.0156	0.0310	0.0315	0.0516	0.0436	0.0486	0.0488
RLMRec-Con	0.2062	0.1676	0.2441	0.2542	0.0227	0.0142	0.0348	0.0358	0.0499	0.0460	0.0488	0.0492
RLMRec-Gen	0.2062	0.1653	0.2407	0.2424	0.0223	0.0145	0.0336	0.0339	0.0517	0.0451	0.0475	0.0477
GLTA	**0.2465**	**0.1882**	**0.2722**	**0.2905**	**0.0359**	**0.0236**	**0.0499**	**0.0500**	**0.0968**	**0.0821**	**0.1162**	**0.1403**
Improv.	19.54%	12.29%	11.51%	14.28%	58.14%	51.28%	43.39%	39.66%	83.68%	78.48%	138.11%	185.16%

TABLE 4.2. Overall performance comparison of GLTA with baselines.

since no movie descriptions are provided in this dataset. The details of the datasets are shown in Table 4.1.

5.1.2. *Baselines.* To demonstrate the effectiveness of GLTA, we compare it with three groups of representative baselines. 1.) GNN-based recommendation with only interaction information (LightGCN [18], HCCF [79]) that applies graph or hypergraph neural network on the user-item interaction graph for information propagation. 2.) GNN-based recommendation with text information (LightGCN+, HCCF+) that uses InfoNCE loss to align the encoded node embeddings with description embeddings from a sentence transformer [71]. 3.) LLM-based recommendation [53] that leverages contrastive (RLMRec-Con) or generative (RLMRec-Gen) alignment.

5.1.3. *Evaluation Metrics.* We evaluate the recommendation in an end-to-end manner by ranking the test users with all non-interacted items. Precision (P@5, P@10) and NDCG (N@5, N@10) are adopted as evaluation metrics.

5.2. Overall Performance. Performance comparison between GLTA and other baselines are shown in Table 4.2. We have the following observations. First, GLTA exhibits superior performance on all datasets, especially in MovieLens where the dense user-item interactions lead to over-smoothing in GNN-based baselines [32]. Second, compared to graph-based recommendation with only interaction information, introducing text information into graph-based methods improves the recommendation performance in most cases, which verifies the importance of incorporating rich semantic content to enhance user-item matching and better capture the contextual nuances of items. Third, the LLM-based recommendation method, RLMRec-Con, has better overall performance than other baselines. This

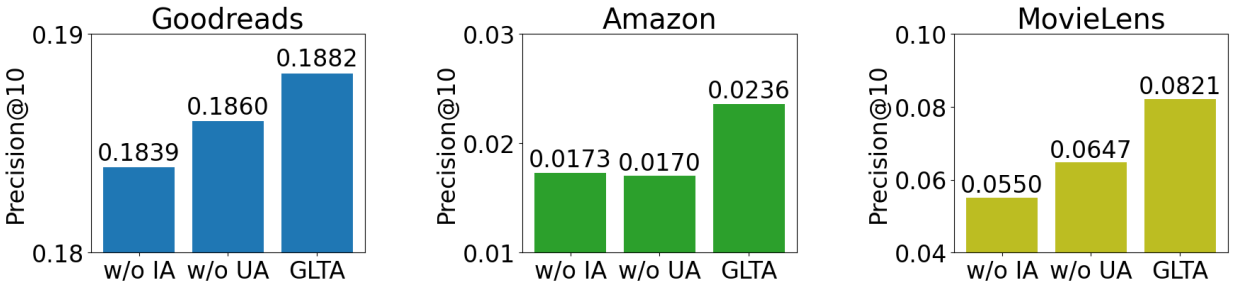


FIGURE 4.3. Ablation study on alignment methods in GLTA.

justifies that aligning the knowledge of LLMs with collaborative relation learning through contrastive learning is able to enhance recommendation performance. However, the recommendation process is still done by the backbone LightGCN model in RLMRec, leaving the reasoning abilities of LLMs untouched. On the contrary, GLTA projects users and items into language space as tokens first, and then completely leverages the LLM for end-to-end recommendation, which is a more holistic approach that fully utilizes the reasoning capabilities and contextual understanding of LLMs.

5.3. Item-text and User-item Alignment. In GLTA, item-text alignment and user-item alignment are designed to align CF with the reasoning ability of the LLM. To verify the effectiveness of these two alignment methods, we compare the performance of GLTA to its variant without item-text alignment (*w/o IA*) and without user-item alignment (*w/o UA*) on the three datasets in Figure 4.3. For the variant *w/o IA*, we directly remove the item-text alignment stage and update item and user node projectors simultaneously in the user-item alignment stage. For the variant *w/o UA*, we use user IDs instead of aligned user tokens in the user-item alignment instruction template before feeding it into the GLLM layer. We find that employing item-text alignment or user-item alignment leads to stable enhancement in all datasets, which implies that both alignment methods are crucial for effectively leveraging the LLM's reasoning capabilities in the recommendation process.

5.4. Profile and Prediction Tokens. To quantify the contribution of profile and prediction tokens used in user-item alignment, we conduct an ablation study to investigate the performance of GLTA without these tokens generated by the LLM. The results are shown

Dataset	Goodreads		Amazon		MovieLens	
Metric	P@5	N@5	P@5	N@5	P@5	N@5
w/o both	0.2462	0.2705	0.0316	0.0408	0.0736	0.0658
w/o PF	0.2466	0.2668	0.0318	0.0414	0.0763	0.0713
w/o PD	**0.2467**	0.2688	0.0341	0.0449	0.0860	0.0761
GLTA	0.2465	**0.2722**	**0.0359**	**0.0499**	**0.0968**	**0.1162**

TABLE 4.3. Ablation study on profile and prediction tokens in GLTA.

in Table 4.3, where *w/o PF* removes the profile tokens, *w/o PD* removes the prediction tokens, and *w/o both* removes both. Both profile and prediction tokens generally improve performance if included in the instruction template. Notably, the advantages of using these LLM-generated tokens become more pronounced when the distinctions between items in the dataset are clearer. For instance, the performance improvement is more evident in MovieLens, where items span a wide range of movie genres, but less pronounced in Goodreads, where items consist only of history books. We speculate that the broader diversity of items in datasets like MovieLens allows the LLM-generated tokens to better capture user features from nuanced differences between items, thereby enhancing recommendation accuracy. In contrast, as the distinctions between items are less varied and therefore less reliant on the LLM's enhanced representation capabilities, the additional benefits provided by these tokens are also limited.

5.5. Item Prediction in GLLM. We further explore two other item prediction patterns in GLLM layers. Autoregressive inference(AR): During inference, we strictly follow the text generation of LLMs in which each item token is generated based on the previous item tokens the model has generated. First-logit optimization (FL): To predict the top k favorite items for each user, we use the largest k elements in the first logit from $\mathbf{Z}_i \in \mathbb{R}^{L\times|\mathcal{I}|}$, instead of the first k logits in GLTA. The results are demonstrated in Figure 4.4. The poor performance of autoregressive inference indicates that, unlike natural language processing tasks where contextual continuity is vital, the sequential dependency between items is not as strong in recommendation. The inconsistency between training and inference patterns exacerbates this issue, particularly because the model is trained using item descriptions and collaborative signals rather than direct sequential user-item interaction data.

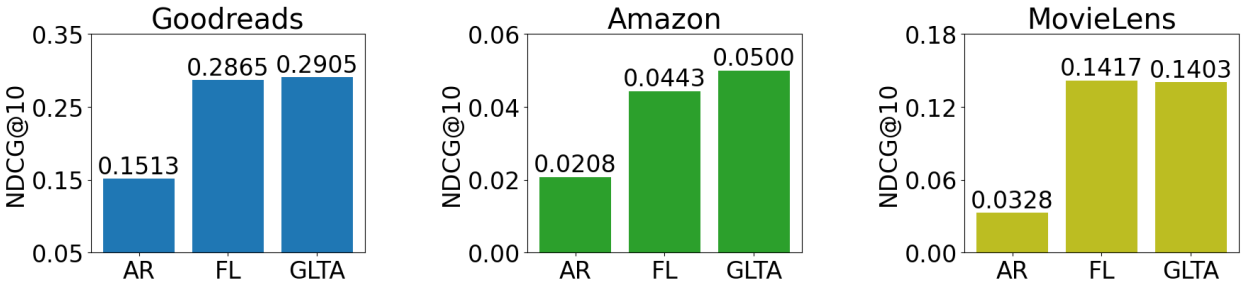


FIGURE 4.4. Performance of GLTA with different optimization methods.

6. Chapter Summary

In this chapter, we propose a novel recommendation framework GLTA, which applies token alignment to integrate graph-based CF with the reasoning capabilities of LLMs. In GLTA, we begin by employing a graph encoder to capture user and item node features from the collaborative relationships within the user-item graph. Next, item and user node embeddings are adaptively aligned with other language tokens using node projectors. Finally, the user and item node projectors are optimized for end-to-end recommendation through the GLLM layer.

One potential limitation of GLTA is that, although the LLM parameters remain frozen, backpropagation is still required through the entire LLM to optimize the node projectors and the GLLM layer. As modern LLMs continue to scale in size, this training paradigm can become memory-intensive, limiting its practicality for resource-constrained scenarios. In the next chapter, we therefore investigate a more lightweight yet effective paradigm that incorporates LLM knowledge and generative capabilities into recommendation without requiring gradient propagation through the full LLM, enabling a more efficient and scalable integration strategy.

CHAPTER 5

LLM-Inferred Substitute and Complement Signals for Recommendation

(This chapter is based on the manuscript "Enhancing Recommendation with LLM-Inferred Substitute and Complement Signals," which is being prepared for submission.)

1. Introduction

Aligning graph representations with LLM tokens in Chapter 4 requires backpropagation through the entire LLM, which becomes costly as model sizes continue to grow. This chapter explores a lighter alternative, where an LLM is queried offline to assess functional relations between items and the resulting assessments are used to weight hyperedges that follow the formulation of Chapter 2, with each hyperedge here representing an item group induced by co-occurrence signals instead of a pretext task. Substitution and complementarity are fundamental item-item relations for understanding user intent in RecSys [98, 76]. Substitutes represent alternatives that satisfy the same user need, while complements capture items that are jointly consumed within a coherent usage context. For example, an on-ear headphone and an over-ear headphone serve similar listening purposes and can be regarded as substitutes, whereas adapters are typically complementary accessories that support headphone usage in cellphones. Recommending such substitute or complementary items to an on-ear headphone buyer can improve user satisfaction by offering meaningful alternatives. However, ground-truth annotations of substitute and complementary relations are rarely available in public datasets [46, 2], and obtaining reliable annotations through manual labeling is costly and time-consuming. Consequently, prior work [10, 39, 80] typically relies on co-view or co-purchase signals as weak supervision. However, directly treating these signals as substitutes or complements implicitly assumes that behavioral co-occurrence aligns with real-world item relations, an assumption that frequently fails in practice [28, 92, 60]. Many

co-occurring item pairs are neither substitutes nor complements, but arise from incidental browsing or exploratory interactions. On e-commerce platforms, users often interact with multiple unrelated items within a short time window, resulting in co-occurrence records that are incidental rather than semantically meaningful.

Ideally, a solution should possess human-like world knowledge to distinguish true substitute and complementary relations from incidental co-occurrences, while avoiding the high cost and inefficiency of large-scale manual annotation. Recent advances in LLMs offer a promising way to address these challenges, as they encode rich world knowledge about usage contexts acquired from large-scale text corpora. This knowledge enables LLMs to move beyond surface-level co-occurrence patterns and reason about the functional roles items play in real-world scenarios. Benefiting from this semantic reasoning capability, LLMs have been applied for labeling substitute and complementary relations in previous works [50, 20].

To effectively harness the world knowledge of LLMs on substitutes and complements for recommendation, we propose ERLSC, which leverages LLMs to distinguish true substitute and complementary relations from noisy co-occurring item relations for enhanced recommendation. We adopt a hypergraph formulation because co-view and co-purchase signals naturally induce item groups rather than isolated item pairs, and hyperedges provide a principled way to model such higher-order co-occurrence relations. By assigning weights to hyperedges via an LLM, ERLSC can selectively emphasize item groups that exhibit coherent substitute or complement semantics while down-weighting irrelevant co-occurrences. To incorporate LLM-derived semantic signals into this structure, we design an LLM-Guided Hypergraph Convolution (LGHC) layer to initialize and update hyperedge weights in an item-item hypergraph. Specifically, we first construct hyperedges by connecting frequently co-occurring items in user interaction logs. We then query LLMs to assess how many item pairs within each hyperedge reflect true substitute or complementary relations. The resulting relation composition is used to initialize hyperedge weights, which are further refined during training through weight modulation. The key contributions are summarized as follows.

- We propose a novel recommendation framework ERLSC, which enhances recommendation via LLM-inferred substitute and complement signals.
- We introduce an LGHC layer that seamlessly integrates LLM-based relation assessment into hypergraph modeling by initializing and adaptively updating hyperedge weights.
- We conduct extensive experiments on three public datasets to demonstrate the effectiveness of ERLSC, and perform ablation studies to verify the contribution of each component.

2. Related Work

2.1. Substitute and Complementary Relations. Modeling substitutes and complements has long been recognized as a way to capture user intent beyond plain co-occurrence [98, 76]. Since annotated relations are seldom available, most prior work derives weak supervision from co-view and co-purchase records [10, 39, 80]. Later studies show that such behavioral signals frequently disagree with real-world item relations, because many co-occurring pairs arise from incidental browsing rather than a shared function [28, 92, 60]. Recent work therefore turns to LLMs for relation labeling [50, 20], although the resulting labels are typically consumed as hard supervision rather than as a signal that the recommendation model can weigh.

2.2. Hypergraph Learning for Recommendation. Hypergraphs represent relations among multiple entities within a single hyperedge and have been used to capture high-order dependencies in recommendation [79]. Chapter 2 adopts this formulation to unify heterogeneous pretext tasks, where each hyperedge corresponds to a task-specific group of nodes. Existing methods generally treat hyperedges as equally reliable once they are constructed, which is problematic when the hyperedges come from noisy behavioral co-occurrence. We keep the hypergraph formulation but make hyperedge weights depend on the functional relations that an LLM infers among the items inside each group.

3. Preliminaries

3.1. Problem Formulation. We follow the notation introduced in Chapter 2. Let $\mathcal{I}$ and $\mathcal{U}$ denote the complete item pool and the set of users in the dataset, respectively. Given the user-item interactions as edges $E_{\mathcal{U},\mathcal{I}}$, a user-item graph is defined as $G = (V, E)$ where $V = \mathcal{U} \cup \mathcal{I}$. Each item $i \in \mathcal{I}$ is accompanied by its corresponding textual description $\text{desc}(i)$. In addition, items that appear in the same co-view or co-purchase group are connected by a hyperedge. An incidence matrix $\mathbf{H} \in \{0,1\}^{|\mathcal{I}|\times|\mathcal{E}|}$ is used to represent the connections between items $\mathcal{I}$ and hyperedges $\mathcal{E}$ in the item-item hypergraph. Given a user $u \in \mathcal{U}$, the recommendation task is to predict a ranked list of items $\{i_1, i_2, \ldots, i_m\}$ that the user has not interacted with in the graph G.

3.2. Collaborative Filtering. To learn representations of users and items from interactions, we apply collaborative filtering (CF) on the user-item graph G. CF captures collaborative signals by modeling historical user-item interactions, which are essential for inferring user preferences and item relevance in recommendation. We maintain an embedding layer $\mathbf{E} \in \mathbb{R}^{(|\mathcal{U}|+|\mathcal{I}|)\times d}$, where d denotes the embedding dimension. Through CF, we obtain user embeddings $\mathbf{E}'_{\mathbf{u}}$ and item embeddings $\mathbf{E}'_{\mathbf{i}}$, which encode collaborative patterns from the interaction graph. The learned user embeddings are subsequently used to perform personalized ranking over candidate items in the recommendation task. In this work, we adopt a one-layer LightGCN [18] as the underlying graph-based CF model due to its simplicity and effectiveness.

4. Proposed Framework: ERLSC

4.1. LLM-based Relation Assessment. The overall architecture of ERLSC is illustrated in Figure 5.1. Beyond the collaborative signals captured by CF, propagating information over item-item hypergraphs is essential for modeling the semantic and functional relationships implicit in item co-occurrence. In particular, co-view and co-purchase relations often reflect shared functionality or joint usage patterns that cannot be fully captured by user-item interactions alone. To assess the reliability of each hyperedge in representing such

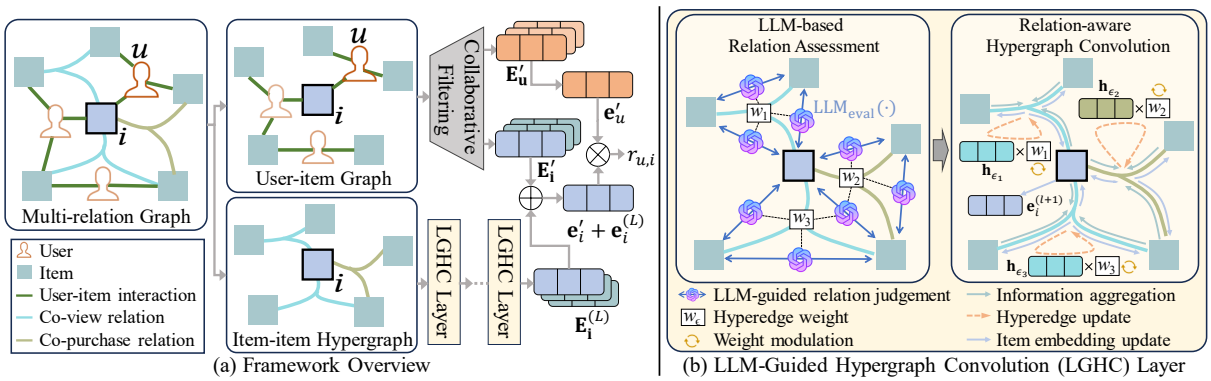


FIGURE 5.1. The framework of ERLSC and the proposed LGHC layer.

relations, as the first component of the proposed LGHC layer, we leverage LLMs to evaluate its semantic coherence based on LLMs' world knowledge.

Since substitute and complementary relations are more reliably identified at the pairwise level than at the group level, we first query the LLM to assess pairwise relations within each co-occurrence group. Specifically, given a hyperedge ϵ that connects a group of items $\mathcal{N}_\epsilon$, we enumerate all unordered item-item pairs $\mathcal{P}(\mathcal{N}_\epsilon) = \{(i_j, i_k) \mid i_j, i_k \in \mathcal{N}_\epsilon,\ j < k\}$, and compute the number of item pairs that exhibit substitute or complementary relations as follows:

$$y_\epsilon^t = \sum_{(i_j, i_k) \in \mathcal{P}(\mathcal{N}_\epsilon)} \mathrm{LLM}_{\mathrm{eval}}(t, \mathrm{desc}(i_j), \mathrm{desc}(i_k)) \tag{22}$$

where $t \in \{\mathrm{sub}, \mathrm{com}\}$ denotes the relation type being queried, corresponding to substitute or complementary relations, respectively. The function $\mathrm{LLM}_{\mathrm{eval}}(\cdot)$ returns a binary outcome indicating whether the item pair (i_j, i_k) exhibits the specified relation. The two labels are mutually exclusive, because items that serve as alternatives to each other are not the items that are jointly consumed in a single usage context, so at most one of them is positive for any given pair and $\mathrm{LLM}_{\mathrm{eval}}(\mathrm{sub}, \mathrm{desc}(i_j), \mathrm{desc}(i_k)) + \mathrm{LLM}_{\mathrm{eval}}(\mathrm{com}, \mathrm{desc}(i_j), \mathrm{desc}(i_k)) \leq 1$. Accordingly, y_ϵ^t represents the total number of item pairs with a substitute or complementary relation within the hyperedge, and summing this inequality over all pairs gives $y_\epsilon^{\mathrm{sub}} + y_\epsilon^{\mathrm{com}} \leq |\mathcal{P}(\mathcal{N}_\epsilon)|$.

We then initialize the hyperedge weight based on the proportion of relation-consistent pairs within the group. Specifically, the weight of hyperedge ϵ is defined as $w_\epsilon = \frac{y_\epsilon^{\mathrm{sub}} + y_\epsilon^{\mathrm{com}}}{|\mathcal{P}(\mathcal{N}_\epsilon)|}$.

Since no pair contributes to both counts, the numerator never exceeds the denominator, so $w_\epsilon \in [0, 1]$ and is a valid proportion of relation-consistent item pairs within the group. A larger value of w_ϵ indicates that a higher fraction of item pairs exhibit either substitute or complementary relations, suggesting that the hyperedge corresponds to a semantically coherent and functionally meaningful co-occurrence pattern. In contrast, a smaller value implies that most item pairs lack clear semantic relations, indicating that the hyperedge is likely noisy or less informative.

4.2. Relation-aware Hypergraph Convolution. Starting from the hypergraph convolution formulated in Section 3.4, we incorporate LLM world knowledge as prior semantic guidance and adaptively refine hyperedge importance during training. The resulting relation-aware hypergraph convolution module consists of three sequential components, namely information aggregation, hyperedge update, and item embedding update.

4.2.1. *Information Aggregation.* We first construct a representation for each hyperedge by aggregating the embeddings of all items incident to it:

$$\mathbf{h}_\epsilon^{(l)} = \frac{1}{|\mathcal{N}_\epsilon|} \sum_{i \in \mathcal{N}_\epsilon} \mathbf{e}_i^{(l)}, \tag{23}$$

where $\mathbf{h}_\epsilon^{(l)}$ denotes the embedding of hyperedge ϵ and $\mathbf{e}_i^{(l)}$ is the embedding of each connected item. We adopt mean pooling as the aggregation method to summarize information from item nodes.

4.2.2. *Hyperedge Update.* To enable the model to exploit the LLM-initialized hyperedge weights while allowing adaptive refinement during training, we perform weight modulation based on hyperedge-level features. Each hyperedge ϵ is associated with a feature vector $\mathbf{x}_\epsilon$ that captures both structural and semantic characteristics, including: the hyperedge degree, the average and maximum degrees of its connected item nodes, y_ϵ^{sub}, y_ϵ^{com}, $|y_\epsilon^{\text{sub}} - y_\epsilon^{\text{com}}|$, and $|\mathcal{P}(\mathcal{N}_\epsilon)|$. These features are fed into a two-layer multilayer perceptron (MLP) to generate a modulation factor for the hyperedge weight. The reweighted hyperedge embedding is then

computed as

$$\mathbf{q}_\epsilon^{(l)} = f_{\text{MLP}}(\mathbf{x}_\epsilon) \cdot w_\epsilon \cdot \mathbf{h}_\epsilon^{(l)}, \tag{24}$$

where w_ϵ is the LLM-initialized hyperedge weight, $f_{\text{MLP}}(\cdot)$ denotes the weight-modulation MLP, and $\mathbf{q}_\epsilon^{(l)}$ represents the final reweighted embedding of hyperedge ϵ at layer l. This design allows the model to jointly leverage prior semantic reliability from the LLM and task-specific structural cues learned during training.

4.2.3. *Item Embedding Update.* Finally, we update each item embedding by aggregating the reweighted embeddings of all hyperedges incident to the item. For a target item i, the update rule is given by

$$\mathbf{e}_i^{(l+1)} = \frac{1}{|\mathcal{N}_i|} \sum_{\epsilon \in \mathcal{N}_i} \mathbf{q}_\epsilon^{(l)}, \tag{25}$$

where $\mathbf{e}_i^{(l+1)}$ denotes the updated embedding of item i, and $\mathcal{N}_i$ is the set of hyperedges connected to item i. Through this operation, each item aggregates information from its surrounding hyperedges, with contributions weighted by both LLM-derived semantic confidence and learned structural relevance.

4.3. Ranking Prediction. The final item embedding is obtained by adding CF-based item embedding $\mathbf{e}'_i$ to the updated item embedding $\mathbf{e}_i^{(L)}$ from the last LGHC layer. We then compute the ranking score $r_{u,i}$ of the user-item pair (u, i) by the inner product of user and item embeddings:

$$r_{u,i} = \mathbf{e}'_u \cdot (\mathbf{e}'_i + \mathbf{e}_i^{(L)}), \tag{26}$$

Then, we adopt the pairwise BPR loss [54] to optimize the ranking prediction:

$$\mathcal{L} = \sum_{(u,i,i^-) \in \mathcal{D}} -\log \sigma(r_{u,i} - r_{u,i^-}) + \lambda_\Theta \|\Theta\|_2^2, \tag{27}$$

where $\mathcal{D} = \{(u, i, i^-) | i \in \mathcal{I}_u^+, i^- \in \mathcal{I} \backslash \mathcal{I}_u^+\}$ is the training data and item set $\mathcal{I}_u^+$ contains all the items interacted by user u. We denote all trainable parameters as Θ, regularized by λ_Θ. Adam [33] is chosen as the optimizer.

Dataset	Electronics	Office	Home
#Users	12,828	7,528	1,534
#Items	19,297	5,743	4,923
#U-I interactions	93,788	40,330	7,810
#*Bought-together* relations	2,769	1,555	326
#*Compared-with* relations	5,746	706	614
#*Also-bought* relations	5,852	696	715
#*Also-viewed* relations	15,288	3,838	2,819

TABLE 5.1. Statistics of datasets for ERLSC.

5. Experiment

5.1. Experiment Settings.

5.1.1. *Datasets.* We conduct experiments on three sub-datasets from the public XMarket dataset [2], namely Electronics, Office, and Home. These categories are chosen because they naturally exhibit abundant substitute and complementary item relations, due to overlapping product functionalities and frequent accessory co-usage. The statistics of the selected datasets are summarized in Table 5.1. We treat *bought-together* and *also-bought* relations as co-purchase signals, and *compared-with* and *also-viewed* relations as co-view signals.

5.1.2. *Baselines.* To evaluate the effectiveness of ERLSC, we compare it with six baselines, including four representative CF methods, namely LightGCN [18], HCCF [79], SimpleX [43], and DiffRec [72]. Since the datasets also provide sequential order information in user transaction histories, we further compare our method with sequential recommendation baselines, including SASRecCPR [8] and FeaRec [12]. All methods are evaluated under the full-ranking setting, where all non-interacted items are ranked for each test user. We adopt Recall (R@10, R@20) and NDCG (N@10, N@20) as evaluation metrics. The relation assessment function $\mathrm{LLM}_{\mathrm{eval}}(\cdot)$ in Equation (22) is instantiated with the Phi-4 language model.

5.2. Overall Performance. The overall performance comparison between ERLSC and the baseline methods is reported in Table 5.2. We make the following observations. First, ERLSC consistently outperforms all baseline methods across all datasets and evaluation metrics. The improvements are particularly pronounced on the Electronics dataset, where

Dataset	Electronics				Office				Home			
Metric	R@10	R@20	N@10	N@20	R@10	R@20	N@10	N@20	R@10	R@20	N@10	N@20
LightGCN	0.0894	0.0959	0.0786	0.0804	0.2489	0.2692	0.2141	0.2195	0.1118	0.1199	0.0910	0.0930
SimpleX	0.0860	0.0924	0.0779	0.0797	0.2444	0.2583	0.2136	0.2174	0.1098	0.1167	0.0872	0.0888
DiffRec	0.0743	0.0801	0.0680	0.0696	0.2405	0.2524	0.2167	0.2199	0.1074	0.1172	0.0816	0.0838
SASRecCPR	0.0665	0.0961	0.0700	0.0729	0.1925	0.2111	0.1966	0.2010	0.1151	0.1235	0.0683	0.0751
FeaRec	0.0704	0.0897	0.0607	0.0656	0.1829	0.1963	0.1698	0.1739	0.1206	0.1319	0.0887	0.0941
HCCF	0.0827	0.1025	0.0783	0.0845	0.2167	0.2624	0.1986	0.2272	0.1133	0.1214	0.0839	0.0865
ERLSC	**0.1232**	**0.1337**	**0.1034**	**0.1062**	**0.2816**	**0.3045**	**0.2404**	**0.2467**	**0.1325**	**0.1423**	**0.1131**	**0.1157**
Improv.	37.78%	30.45%	31.54%	25.62%	13.11%	13.10%	10.96%	8.60%	9.87%	7.84%	24.20%	22.99%

TABLE 5.2. Overall performance comparison of ERLSC with baselines.

rich co-view and co-purchase signals are available. This result demonstrates the effectiveness of incorporating item-item relations to capture functional item groupings. Second, the hypergraph-based baseline HCCF constructs hyperedges by connecting all items interacted with by the same user. This strategy remains sensitive to sparse user-item interactions, such as the Home dataset, where user behaviors are less dense. In contrast, ERLSC constructs hyperedges from item co-occurrence relations and further leverages LLM-based semantic assessment to weight these hyperedges, allowing the model to emphasize meaningful substitute and complementary relations while suppressing irrelevant co-occurrences. This design enables ERLSC to achieve robust performance even when behavioral signals are noisy or limited.

5.3. Ablation Study. To verify the effectiveness of each component in ERLSC, we conduct ablation studies by comparing it with several variants, as shown in Figure 5.2. The variant *w/o H* considers only user-item interactions without item-item hypergraph modeling. The variant *H* replaces the LGHC layer in ERLSC with a hypergraph convolution layer, where all hyperedges have equal weights $w_\epsilon = 1$. The variant *LH* further incorporates LLM-initialized hyperedge weights, but disables weight modulation during training. From the results, we observe a consistent performance improvement when hypergraph information is introduced. Moreover, by incorporating LLM-initialized hyperedge weights, the *LH* variant further improves performance, demonstrating that LLM-based semantic assessment helps identify hyperedges that better reflect substitute and complementary relations. Finally, the best performance of ERLSC on all datasets highlights the importance of dynamically modulating hyperedge weights during training, which allows the model to adaptively refine

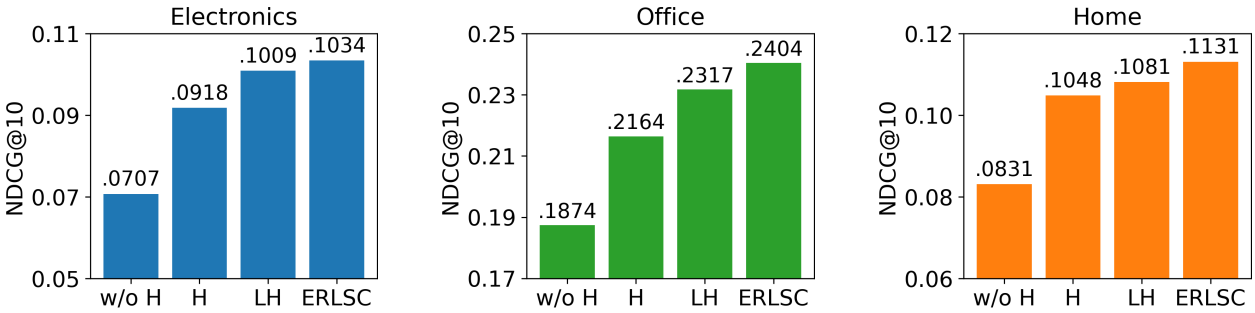


FIGURE 5.2. Performance of ERLSC compared to its variants.

the influence of each hyperedge based on recommendation objectives. Overall, the ablation results confirm that both LLM-guided hyperedge initialization and weight modulation are critical components of ERLSC.

5.4. Performance under Different Signals. We evaluate the performance of ERLSC under only one of the four different types of co-occurrence signals provided in the dataset, including *BT* (*bought-together*), *CW* (*compared-with*), *AB* (*also-bought*), and *AV* (*also-viewed*). For each signal type, we construct item-item hyperedges solely based on the corresponding relation and report the results in Table 5.3. For each signal, we compare the following variants. (i) *BT/CW/AB/AV* only: the LGHC layer is replaced by a naive hypergraph convolution layer. (ii) $\{\text{S,C}\}_{\max}$: hyperedge weights are initialized as $w_\epsilon = \frac{\max\left(y_\epsilon^{\text{sub}}, y_\epsilon^{\text{com}}\right)}{|\mathcal{P}(\mathcal{N}_\epsilon)|}$ which emphasizes whether substitution or complementarity is more prevalent. (iii) S&C: the full ERLSC using both substitute and complement labels while restricting the hypergraph to the selected co-occurrence signal. Compared with the corresponding relation-only variants, the full S&C formulation consistently improves R@10 and N@10 across all reported signals and datasets. In contrast, the $\{\text{S,C}\}_{\max}$ formulation yields mixed results and occasionally underperforms the relation-only variant. Overall, jointly incorporating substitute and complement information generally provides more reliable performance than relying only on the dominant relation.

Dataset	Electronics		Office		Home	
Metric	R@10	N@10	R@10	N@10	R@10	N@10
BT only	0.0921	0.0656	0.2629	0.2170	0.1216	0.0793
+ $\{S,C\}_{max}$	0.0952	0.0719	0.2672	0.2181	0.1199	0.0929
+ S&C	0.1066	0.0826	0.2684	0.2215	0.1226	0.1075
CW only	0.0865	0.0710	0.2525	0.2112	0.1118	0.0860
+ $\{S,C\}_{max}$	0.0893	0.0776	0.2555	0.2162	0.1175	0.0943
+ S&C	0.0979	0.0782	0.2604	0.2189	0.1226	0.1012
AB only	0.0940	0.0765	0.2468	0.1936	0.1182	0.0766
+ $\{S,C\}_{max}$	0.0971	0.0770	0.2450	0.2030	0.1169	0.0889
+ S&C	0.1066	0.0834	0.2527	0.2062	0.1226	0.1063
AV only	0.0998	0.0858	0.2738	0.2273	0.1189	0.0666
+ $\{S,C\}_{max}$	0.1089	0.0845	0.2747	0.2296	0.1216	0.1078
+ S&C	0.1140	0.0878	0.2790	0.2328	0.1274	0.1076

TABLE 5.3. Performance of ERLSC on each single relation type.

6. Chapter Summary

In this chapter, we investigate how to leverage LLMs to assess substitute and complementary relations and to adaptively determine hyperedge weights through the proposed LGHC layer. Experimental results on real-world datasets demonstrate the effectiveness of incorporating LLM-encoded world knowledge on substitution and complementarity into recommendation models. Several limitations remain. Substitution and complementarity are well defined for grocery and retail catalogs, where products have clear functional roles, but they are less natural in domains such as music or news, where items are consumed for reasons that are not functional, so the gains reported here should be read as evidence for catalogs of the former kind rather than for recommendation in general. The relational signals also inherit the properties of the model that produced them, since a judgment that is wrong is stored as a hyperedge weight in the same way as a judgment that is right, which leaves the robustness of ERLSC to noisy annotations an open question. ERLSC further follows a single-model learning paradigm, where the final recommendation performance depends on a single trained model, even when enhanced by LLM-derived relational signals. In the next chapter, we move beyond this paradigm and introduce an LLM-based agent framework that coordinates multiple recommendation tools and performs explicit relational reasoning at inference time.

CHAPTER 6

Dynamic Recommendation with Item-Item Relations via LLM-based Agents

(This chapter was previously published as "AgentDR: Dynamic recommendation with implicit item-item relations via LLM-based agents" [84] in *Proceedings of the ACM on Web Conference 2026*, pp. 6230 - 6241. 2026. [doi:10.1145/3774904.3792304])

1. Introduction

Chapter 5 uses LLM-inferred substitute and complementary relations to refine structural signals inside a single recommendation model, so the final performance still depends on one trained model, whereas this chapter retains the same relational knowledge and moves it to the decision level, where an LLM agent coordinates several recommendation tools and reasons over item relations at inference time without additional training. RecSys provide personalized suggestions to assist users in information discovery and decision-making [89, 85, 41]. However, traditional ID-based RecSys [18, 31, 9] solely based on user-item interactions are constrained by overlooking the rich semantic information contained in textual context. Even when LLMs are employed to encode textual features into embeddings [75, 53, 105], these fixed-length representations inherently struggle to preserve the full semantic richness of textual information for recommendation [35, 78]. There are several attempts to harness the commonsense reasoning and knowledge utilization capabilities of LLMs by directly applying them as RecSys, reformulating recommendation tasks as language modeling problems [16, 21, 11, 19]. However, an inherent gap remains because LLMs are optimized for next-token prediction in natural language rather than learning collaborative and sequential patterns from interactions, limiting their efficiency in user behavior modeling.



FIGURE 6.1. Two challenges in directly deploying LLMs for recommendation.

Given the limitations of using LLMs solely as feature encoders or standalone recommenders, recent studies have investigated leveraging LLMs as interactive agents for recommendation [97, 96, 67]. Rather than relying only on direct prompting with user interaction histories, these agent-based approaches integrate memory modules to preserve long-term context and better harness collaborative signals throughout the recommendation process. While LLM-based agents alleviate the representational constraints of static numerical embeddings and enable adaptive modeling of user behavior over time, they also inherit key limitations from the generative nature of LLMs. As illustrated in Figure 6.1, two primary drawbacks arise in previous agent-based works when directly applying LLMs for recommendation: (i) the tendency to hallucinate non-existent items that are not part of the product catalog, and (ii) the high inference cost and token length constraints, which make them impractical for large-scale item ranking. Moreover, longer input and output sequences further increase the risk of hallucination due to attention bottlenecks and heightened generation uncertainty [25].

As a result, when applied to recommendation scenarios, existing LLM-based agents [97, 82, 96] typically evaluate performance by checking whether the ground-truth item, which is

the next item that the user actually interacted, is correctly ranked against a small set of negative candidates. These negative candidates are items randomly sampled from the catalog that the user did not interact with during the observation window, serving as distractors in the ranking process. However, this setup is fundamentally misaligned with most real-world recommendation scenarios, where systems must select the most relevant items from massive catalogs that often contain thousands of items. Restricting the candidate set to such a small range can artificially inflate performance, overlook the complexity of large-scale ranking, and hinder the system's ability to identify truly relevant items in open-ended domains.

Besides their limitations in full-ranking, previous LLM-based agents primarily focus on modeling user preferences from explicit user-item interactions, which already can be effectively modeled by traditional embedding-based recommendation methods [43, 9, 18]. On the contrary, traditional models often overlook higher-level semantic relations between items that go beyond co-occurrence patterns. To address this gap, the generative capabilities and extensive world knowledge of LLMs make them particularly well-suited for reasoning over implicit relationships, such as complementarity and substitution relations. Substitutes help identify alternative options when a preferred item is unavailable or when the user seeks variety, while complements reveal co-purchase patterns that often indicate planned or contextual consumption behaviors. For example, a user purchasing a digital single-lens reflex camera often requires a tripod as a complement, whereas a user who buys tiramisu is more likely to choose cheesecake as substitute over burgers when the original item is out of stock. Although such relationships can greatly enhance recommendation quality [98, 76, 45], the ground-truth labels of them are typically absent from most datasets [46, 2]. Furthermore, annotating item pairs with accurate relational labels often requires extensive world knowledge and nuanced understanding, making manual annotation both expensive and time-consuming. This gap highlights the potential of LLMs as an alternative source of relational knowledge. By leveraging their extensive pretraining on diverse textual data, LLMs can capture subtle functional relationships between items, enabling automatic discovery of substitutes and complements that would otherwise require labor-intensive human annotation.

Considering the two issues discussed above, we propose a novel LLM-based agent framework that leverages the complementary strengths of LLMs and traditional recommendation models. This framework addresses hallucination by grounding all recommendations in catalog-aware models. The reasoning ability and world knowledge of LLMs are utilized to integrate the ranking results of multiple recommendation tools and to perform fine-grained refinement of the combined ranking results, both in a personalized manner. For each user, an agent is optimized to assess the suitability of each recommendation tool by measuring the gap between the tool's outputs and the user's true preferences. In parallel, the user's intent to purchase substitutes or complements is inferred by the LLM based on the user's historical behavior. During inference, the agent first leverages the stored tool suitability to integrate the outputs from all tools, and then applies the inferred user intent to refine the combined ranking results. Building on this framework, we highlight the main contributions of our work as follows:

- **Novel Framework.** We propose an agent-based framework AgentDR, where LLM-based agents delegate full-ranking recommendation to recommendation tools that excel at modeling complex user behavior patterns. This separation enables the integration of the scalability and efficiency of traditional recommenders with the reasoning capabilities of LLMs.
- **Item-item Reasoning.** We leverage the world knowledge of LLMs for implicit item-item relationship reasoning to enhance recommendation. Each user agent generates substitute and complement candidates based on the user's historical preferences, and accordingly refines recommendation results.
- **Comprehensive Evaluation.** Extensive experiments verify the effectiveness of our approach in full-ranking performance on three public datasets. Besides recall and NDCG, we propose a LLM-based metric to jointly assess the semantic relevance and ordering correctness of the predicted ranking lists.

Method	Full Ranking Evaluation	Text Feature Handling	Generative Reasoning
Recommender based on interactions	✓	✗	✗
LLM as Encoder	✓	✓	✗
LLM as Recommender	✗	✓	✓
LLM-Rec Agents [26, 73, 97, 96, 67, 82]	✗	✓	✓
AgentDR	✓	✓	✓

TABLE 6.1. Comparison of AgentDR and other recommendation methods.

2. Related Work

2.1. LLM-based Agents. LLM-based agents have recently demonstrated strong capabilities in using memory and reflection to learn from past experiences and simulate human-like behaviors [14, 101, 58, 100, 57, 36]. In the context of recommendation, LLM-based agents have been used to investigate social phenomena such as information cocoons and filter bubble effect by simulating user behaviors [67, 96].

2.2. Agents for Recommendation. Given that agent-based frameworks can offer deeper insights into user behavior by leveraging the reasoning capabilities of LLMs, there is a growing interest in applying LLM-based agents to enhance recommendation performance [73, 26, 97, 82]. Nonetheless, since hallucination [93, 56] and token limitation remain two challenges when applying these agents to rank all items in the datasets for each user, most prior works [97, 73, 26, 96, 67, 82] lack of full-ranking ability. The batch-ranking task in these works is fundamentally misaligned with most real-world scenarios, where RecSys are required to rank relevant items from massive catalogs. And these works focus on predicting user behaviors based on user-item interactions, a signal that traditional recommendation models [18, 31, 43] are already well-equipped to capture and exploit. Different from these works, our framework combines the strengths of LLMs and traditional recommenders to achieve personalized full-ranking. Comparison between different approaches is summarized in Table 6.1.

3. Preliminaries

3.1. Problem Formulation. Following the notation introduced in Chapter 2, let $\mathcal{I}$ and $\mathcal{U}$ denote the complete item pool and the set of users in the dataset, respectively. Specifically, each user $u \in \mathcal{U}$ is associated with a behavior sequence $\mathbf{s} = (i_1, i_2, \ldots, i_n)$, where $i_j \in \mathcal{I}$ denotes the j-th item interacted with by user u. Furthermore, each item $i \in \mathcal{I}$ is accompanied by its corresponding textual description. For simplicity, we use desc($\mathbf{s}$) to denote the list of textual descriptions for all items from item sequences $\mathbf{s}$. Given a user's historical interaction sequence $\mathbf{s}_{[:-k]}$ and the corresponding item description list, the goal is to predict the next item $i_{n-k+1} \in \mathcal{I}$ that the user is likely to interact with. This is commonly framed as a ranking task: each user agent is expected to assign higher scores to items that better match the user's preferences, as inferred from both behavioral patterns and textual semantics. The final output is a ranked list of $\mathcal{I}$ tailored to this user u. For clarity, the subscript u will be omitted in subsequent sections, given that each user agent functions independently.

3.2. Recommendation Tools. AgentDR is an agent-based framework compatible with different recommendation methods as tools under the minimal assumption that they produce a full-ranking list of items. These recommendation methods are treated as tools and collectively form the tool set $\mathcal{T}$. For a broad coverage of recommendation modeling strategies, we utilize three types of recommendation tools to perform recommendation tasks within AgentDR: (i) a graph-based model with output ranking score list $\hat{\mathbf{r}}_G$; (ii) a sequential recommendation model with output ranking score list $\hat{\mathbf{r}}_S$; and (iii) a MF-based model with output ranking score list $\hat{\mathbf{r}}_M$. All these output lists are in $\mathbb{R}^{1\times|\mathcal{I}|}$. The three model instances used in this work are elaborated in Section 5.1.2. These models are pretrained on the first $(n-k)$ items, denoted as $\mathbf{s}_{[:-k]}$.

4. Proposed Framework: AgentDR

4.1. Framework Overview. The overall framework of AgentDR is depicted in Figure 6.2. Each user agent is equipped with a profile and two memory modules (Section 4.2).

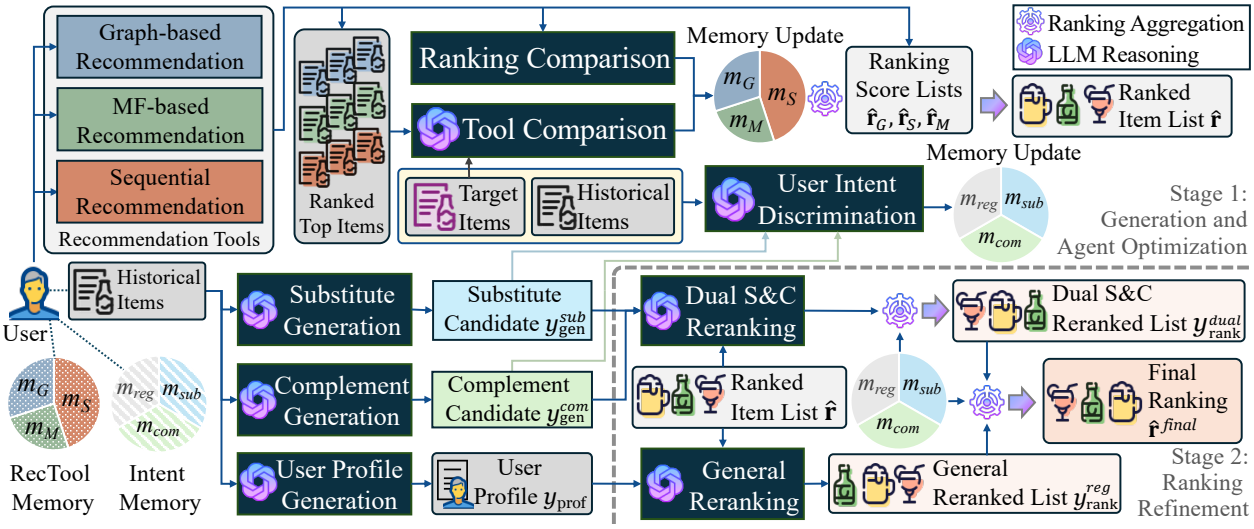


FIGURE 6.2. The framework of AgentDR.

The agent optimization begins with prompting the LLM to generate potential substitutes and complements for each user (Section 4.3). Next, the agent memory is optimized by distinct reflection mechanisms. The RecTool memory is optimized through personalized tool selection (Section 4.4), while the intent memory is optimized via user intent discrimination (Section 4.5). Finally, the top items from the aggregated rankings of multiple tools are refined by dual S&C reranking module (Section 4.6).

4.2. User Profile and Memory.

4.2.1. *User Profile Summarization.* Explicit user profiles are usually unavailable in most public real-world datasets. Thus, we summarize the user profile from users' historical interactions by leveraging the reasoning capabilities and world knowledge of LLMs. To be concrete, we prompt the LLM to summarize the user's preference from descriptions of historical items for each user. And this preference is used as the user profile p of each user agent:

$$y_{\text{prof}} = \text{LLM}_{\text{prof}}(\text{desc}(\mathbf{s})), \tag{28}$$

where $\text{LLM}_{\text{prof}}(\cdot)$ represents the LLM prompted to transform these texts into a coherent user profile. Due to limited space, the details of all prompts used in AgentDR are shown in Section 4.7.

4.2.2. *User Agent Memory.* In contrast to static user profiles, memories are employed to enable user agents to dynamically maintain context in response to agent optimization based on consecutive user behaviors. We equip each user agent with two compact memory modules: *RecTool* memory and *intent* memory. RecTool memory $\mathbf{m}^{rec} \in \mathbb{R}^{1\times|\mathcal{T}|}$ is a list of weights reflecting the suitability of each recommendation tool for each user, where $|\mathcal{T}|$ is the number of recommendation tools. For LightGCN, SASRec and SimpleX deployed in this chapter, we denote their weights in memory as m_G, m_S and m_M, respectively. Similarly, intent memory $\mathbf{m}^{int} \in \mathbb{R}^{1\times3}$ contains three weights m_{sub}, m_{com} and m_{reg} , denoting the user's intent toward substitutes, complements, and other regular items, respectively. In AgentDR, RecTool memory is optimized by tool comparison and ranking comparison modules, detailed in Section 4.4.1 and 4.4.2. Intent memory is optimized by a intent discrimination module in Section 4.5.

4.3. Substitute and Complement Generation. A key challenge in enhancing personalization through substitution and complementarity is that such relationships are usually implicit in most datasets. Annotating these pairwise relationships with LLMs across a large number of items demands substantial computational resource, as it involves $\mathcal{O}(|\mathcal{I}|^2)$ LLM inference calls. To mitigate computational overhead, AgentDR is designed to refine the top-ranked items from recommendation tools based on the substitutes and complements of the user's latest preferences. The first step in this process involves generating potential substitutes and complements based on the user's historical behaviors. Specifically, we prompt the LLM to generate a list of substitutes and a list of complements corresponding to the user's recent interests:

$$y^{sub}_{\text{gen}} = \text{LLM}^{sub}_{\text{gen}}(\text{desc}(\mathbf{s}_{[-(k+c):-k]})) \tag{29}$$

$$y^{com}_{\text{gen}} = \text{LLM}^{com}_{\text{gen}}(\text{desc}(\mathbf{s}_{[-(k+c):-k]})), \tag{30}$$

where $\mathbf{s}_{[-(k+c):-k]}$ denotes the most recent c items visible to the recommendation tools in the sequence. The textual outputs y^{sub}_{gen} and y^{com}_{gen} represent the generated lists of substitutes and complements, each of length c. The functions $\text{LLM}^{sub}_{\text{gen}}(\cdot)$ and $\text{LLM}^{com}_{\text{gen}}(\cdot)$ denote the prompt

functions for generating substitutes and complements, respectively. From a complexity perspective, we reduce the LLM inference calls from $\mathcal{O}(|\mathcal{I}|^2)$ for pairwise annotation to $\mathcal{O}(|\mathcal{U}|)$ for personalized intent generation. This reduction substantially enhances scalability in real-world scenarios, such as online grocery involving millions of items.

4.4. Personalized Tool Selection.

4.4.1. *LLM-based Tool Comparison.* Different users exhibit distinct behavior patterns and therefore benefit from different recommendation strategies. For instance, users who read books like *Harry Potter* often proceed with its sequels, making sequential recommendation models particularly effective. After finishing all the sequels, users may be interested in other fictions read by similar users, where collaborative filtering proves more suitable. To enable personalized tool selection for each user, AgentDR incorporates a tool comparison module that performs a holistic evaluation across multiple recommendation tools. In this module, the user agent is tasked with identifying the best tool based on recommended results of tools and ground-truth user preference:

$$\mathbf{z}_{\text{cpr}} = \text{LLM}_{\text{cpr}}(\{\text{desc}(f_{\text{top}}(\hat{\mathbf{r}}_t, k_{\text{cpr}}))|t \in \mathcal{T}\}, \text{desc}(\mathbf{s}_{[-k:]})), \tag{31}$$

where the output $\mathbf{z}_{\text{cpr}}$ consists of $|\mathcal{T}|$ binary indicators, each indicating whether tool t provides the most relevant recommendation to the user's preference among all tools in $\mathcal{T}$. For instance, $z^G_{\text{cpr}} = 1$ indicates that the LLM has selected the graph-based tool as the best match to the user's preferences. We use $f_{\text{top}}(\hat{\mathbf{r}}_t, k_{\text{cpr}})$ to represent the sequence of top-k_{cpr} items ranked by recommendation tool t. Subsequence $\mathbf{s}_{[-k:]}$ refers to the most recent k user-interacted items, which are withheld from training of recommendation tools and used as ground-truth signals to optimize the agent's decision-making. The function $\text{LLM}_{\text{cpr}}(\cdot)$ is used to prompt the LLM to compare the input sequences and select the tool most aligned with the user's preferences. After the LLM identifies the tool that best aligns with the user's recent preferences, the tool weight $m_t \in \{m_G, m_S, m_M\}$ in RecTool memory is updated accordingly:

$$m_t \leftarrow m_t + \alpha \cdot z^t_{\text{cpr}}, \tag{32}$$

where α is the learning rate to update RecTool memory by this module. This tool comparison remains robust even when multiple tools appear effective. It addresses two key challenges: LLMs tend to agree with any reasonable recommendation, and single-domain datasets often make all outputs appear relevant.

4.4.2. *Ranking Comparison and Aggregation.* Besides the LLM-based tool comparison, we also compare the ranking performance based on the ranks of the most recent k items in each tool:

$$m_t \leftarrow m_t + \beta \sum_{\{r_t^j | i_j \in \mathbf{s}_{[-k:]}\}} \frac{(r_t^j)^{-1}}{\sum_{t' \in \mathcal{T}} (r_{t'}^j)^{-1}}, \tag{33}$$

where r_t^j is the rank of the j-th item from tool t. This quantitative comparison yields a behavior-grounded signal reflecting how well each tool generalizes to unseen user preferences in $\mathbf{s}_{[-k:]}$. To obtain a comprehensive ranked list from all tools, a weighted sum is computed based on the updated RecTool memory:

$$\hat{\mathbf{r}} = \sum_{t \in \mathcal{T}} m_t \hat{\mathbf{r}}_t = m_G \hat{\mathbf{r}}_G + m_S \hat{\mathbf{r}}_S + m_M \hat{\mathbf{r}}_M, \tag{34}$$

where $\hat{\mathbf{r}} \in \mathbb{R}^{1 \times |\mathcal{I}|}$ is the aggregated ranked item list. This simple yet effective rank-level ensemble strategy allows the agent to adaptively combine tools based on user-specific preferences. It also remains flexible and can be replaced by more complex list-wise combination mechanisms based on backpropagation. We implement and compare different ensemble strategies for ranking comparison and aggregation in Section 5.6.

4.5. Intent Discrimination. Understanding whether a user's next interaction is driven by substitutes, complements, or general interest is crucial for tailoring recommendations that match their intent. Traditional ID-based recommenders struggle to distinguish these patterns, as they require nuanced semantic and functional reasoning that goes beyond co-occurrence statistics. With reasoning ability of LLMs, user agents can allocate attention to the most relevant pathway while ensuring coverage of diverse user behaviors, ultimately improving ranking robustness and relevance.

4.5.1. *Substitution and Complementarity.* Based on implicit item-item relationships, we first distinguish the user's potential intent between substitutes and complements. Given the substitute list y_{gen}^{sub} and the complement list y_{gen}^{com} generated in Section 4.3, we prompt the LLM to access the user's interest in substitute or complements by comparing the generated lists with the ground-truth preferences:

$$\mathbf{z}_{\text{dcm}} = \text{LLM}_{\text{dcm}}(y_{\text{gen}}^{sub}, y_{\text{gen}}^{com}, \text{desc}(\mathbf{s}_{[-k:]})), \tag{35}$$

where the output $\mathbf{z}_{\text{dcm}}$ consists of two binary indicators z_{dcm}^{sub} and z_{dcm}^{com}, which sum to 1. Each indicator reflects whether the corresponding list exhibits greater semantic and functional alignment with the user's recent interactions. The function $\text{LLM}_{\text{dcm}}(\cdot)$ prompts the LLM to identify whether the substitute or complement list better matches the user's recent preferences.

4.5.2. *General Interest.* In addition, a separate module is employed to assess the user's general interest in items other than substitutes or complements, based on the user's historical behavior:

$$z_{\text{dcm}}^{reg} = \text{LLM}_{\text{dcm}}^{reg}(\text{desc}(\mathbf{s})), \tag{36}$$

where the output z_{dcm}^{reg} of prompt function $\text{LLM}_{\text{dcm}}^{reg}(\cdot)$ is a binary indicator that equals 1 if the ground-truth user preferences do not exhibit clear substitute or complement patterns otherwise 0. This additional assessment ensures coverage of a broader spectrum of user behaviors, particularly in scenarios where the user's preferences are not driven by substitution or complementarity. Identifying such general interest prevents the agent from overfitting to relational reasoning. This mitigates the risk of forcing item-item relational interpretations where they do not apply.

4.5.3. *General Reranking.* To account for general interests of users, a reranking module is introduced to refine the recommendations based on the semantic similarity between the candidate items and the users' general preferences. For each user, the general preferences have been summarized from historical items as user profile in Section 4.2.1. Then the LLM is prompted to rerank the top items based on the user profile y_{prof}, denoted as function

$\text{LLM}^{reg}_{\text{rank}}(\cdot)$:

$$y^{reg}_{\text{rank}} = \text{LLM}^{reg}_{\text{rank}}(f_{\text{top}}(\hat{\mathbf{r}}, k'), \text{desc}(f_{\text{top}}(\hat{\mathbf{r}}, k')), y_{\text{prof}}), \tag{37}$$

where $f_{\text{top}}(\hat{\mathbf{r}}, k')$ is the top k' items in the aggregated ranked list $\hat{\mathbf{r}}$, and y^{reg}_{rank} is the reranked list containing k' item IDs.

4.5.4. *Intent Memory Update.* Similar to RecTool memory update in Eq. (32), the corresponding intent $m_{int} \in \{m_{sub}, m_{com}, m_{reg}\}$ is updated based on $z^{int}_{\text{dcm}} \in \{z^{sub}_{\text{dcm}}, z^{com}_{\text{dcm}}, z^{reg}_{\text{dcm}}\}$ by a learning rate γ:

$$m_{int} \leftarrow m_{int} + \gamma \cdot z^{int}_{\text{dcm}}. \tag{38}$$

This intent discrimination module equips the agent with a nuanced understanding of user preferences, enabling it to adaptively allocate attention to different reasoning modes and thereby guide downstream reranking in a more personalized manner.

4.6. Dual S&C Reranking. A dual substitute and complement (S&C) reranking module is proposed to comprehensively refine the recommendation results from tools in AgentDR. First, to leverage substitute and complement relation reasoning to enhance recommendation via world knowledge of the LLM, we design two independent reranking modules to refine the top-ranked items based on their semantic similarity with the candidate substitutes and complements for a user:

$$y^{sub}_{\text{rank}} = \text{LLM}_{\text{rank}}(f_{\text{top}}(\hat{\mathbf{r}}, k'), \text{desc}(f_{\text{top}}(\hat{\mathbf{r}}, k')), y^{sub}_{\text{gen}}) \tag{39}$$

$$y^{com}_{\text{rank}} = \text{LLM}_{\text{rank}}(f_{\text{top}}(\hat{\mathbf{r}}, k'), \text{desc}(f_{\text{top}}(\hat{\mathbf{r}}, k')), y^{com}_{\text{gen}}), \tag{40}$$

where output y^{sub}_{rank} and y^{com}_{rank} are the reranked lists containing the same k' items from $f_{\text{top}}(\hat{\mathbf{r}}, k')$. The LLM is prompted to rerank the top items from the aggregated ranked list based on their similarity to candidate substitutes and complements, denoted as $\text{LLM}_{\text{rank}}(\cdot)$.

Although the expected output of the reranked list is a sequence of exactly k' item IDs selected from the ID-only list $f_{\text{top}}(\hat{\mathbf{r}}, k')$, the LLM may hallucinate by generating invalid outputs such as non-existent item IDs or irrelevant text. To address this issue, we apply a rule-based hallucination filtering mechanism to ensure output validity. Specifically, only valid

item IDs in the item corpus are retained in the reranked list. If the resulting list contains fewer than k' items, the remaining valid IDs from $f_{\text{top}}(\hat{\mathbf{r}}, k')$ are appended to the end of the valid list in original order. This filtering mechanism is also applied in the aforementioned general reranking. This strategy completely eliminates the impact of hallucination on the ranking task, while incurring minimal computational overhead and requiring minimal reliance on the LLM's instruction-following capability.

Subsequently, to account for individual variation in user intent, we personalize the combination of the two reranked ID lists using a ranking fusion function $f_{\text{fuse}}(\cdot)$. This function takes the two ID lists and the corresponding intent weights as input:

$$\hat{\mathbf{r}}^{dual} = f_{\text{fuse}}((m_{sub}, y^{sub}_{\text{rank}}), (m_{com}, y^{com}_{\text{rank}})), \tag{41}$$

where the ranking score $\hat{r}^{dual}_i$ of each item i in this output list $\hat{\mathbf{r}}^{dual}$ is computed as a weighted sum of the item's positions in the two input lists:

$$\hat{r}^{dual}_i = m_{sub} \cdot \text{index}(i, y^{sub}_{\text{rank}}) + m_{com} \cdot \text{index}(i, y^{com}_{\text{rank}}), \tag{42}$$

where $\text{index}(i, \cdot)$ returns the rank position of item i within a given list. This personalized fusion strategy allows the agent to balance substitute-based and complement-based reasoning in accordance with the inferred user intent, without incurring additional LLM inference overhead. The output list $\hat{\mathbf{r}}^{dual}$ is further fused with the item list from general reranking:

$$\hat{\mathbf{r}}^{final} = f_{\text{fuse}}((1, y^{dual}_{\text{rank}}), (m_{reg}, y^{reg}_{\text{rank}})), \tag{43}$$

where y^{dual}_{rank} denotes the item list sorted by ranking scores in $\hat{\mathbf{r}}^{dual}$. The final recommendation output consists of the top-k' items ranked by their scores in $\hat{\mathbf{r}}^{final}$. By incorporating the broader user preference profile as a form of regularization in the reranking process, the agent achieves a more robust recommendation strategy, particularly when relational cues among items are weak or absent.

4.7. Details of Prompts. The detailed contents of prompts used in AgentDR are shown from Table 6.2 to Table 6.8.

[**Instruction**]

Summarize the user's preference based on the historical items this user purchased under *Electronics* on *Amazon*.

[**Historical Items**]

{item_descriptions}

TABLE 6.2. Prompts in AgentDR for $\text{LLM}_{\text{prof}}(\cdot)$.

[**Instruction**]

According to the historical items purchased by a user, generate c *substitutes* of these items under *Electronics* on *Amazon*.

[**Historical Items**]

{item_descriptions}

The output must be one list of item titles in length of c, separated by lines.

TABLE 6.3. Prompts in AgentDR for $\text{LLM}^{sub}_{\text{gen}}(\cdot)$.

[Instruction]

Under *Electronics* on *Amazon*, according to the descriptions of items in three groups A, B and C, evaluate which group the target item is most relevant to.

[Group A]

{item_descriptions}

[Group B]

{item_descriptions}

[Group C]

{item_descriptions}

[Target Item]

{target_item_description}

The output must be one single character in {A, B, C} denoting the most relevant group.

TABLE 6.4. Prompts in AgentDR for $\text{LLM}_{\text{cpr}}(\cdot)$.

[Instruction]

Given the two groups of items under *Electronics* on *Amazon*, evaluate which group is more relevant to the target item.

[Group 1]

{item_descriptions}

[Group 2]

{item_descriptions}

[Target Item]

{target_item_description}

The output must be one single number in {1, 2} denoting the more relevant group.

TABLE 6.5. Prompts in AgentDR for $\text{LLM}_{\text{dcm}}(\cdot)$.

[Instruction]

According to the historical items purchased by a user under *Electronics* on *Amazon*, evaluate if this user exhibits clear substitute/complement patterns or not.

[Historical Items]

{item_descriptions}

The output must be one single word in {Yes, No}.

TABLE 6.6. Prompts in AgentDR for $\text{LLM}^{reg}_{\text{dcm}}(\cdot)$.

[Instruction]

According to the user profile, rank top-k' items this user may prefer from the candidate item list, from higher to lower probability.

[User Profile]

{user_profile}

[Candidate Item List in format of (ID, description)]

{(item_IDs, item_descriptions)}

The output must be a list of candidate item IDs with length of k', with items separated by lines.

TABLE 6.7. Prompts in AgentDR for $\text{LLM}^{reg}_{\text{rank}}(\cdot)$.

[Instruction]

Rank top-k' items from the candidate item list based on their similarity to the target item list, from higher to lower similarity.

[Target Item List ordered by priority]

{item_descriptions}

[Candidate Item List in format of (ID, description)]

{(item_IDs, item_descriptions)}

The output must be a list of candidate item IDs with length of k', with items separated by lines.

TABLE 6.8. Prompts in AgentDR for $\text{LLM}^{sub}_{\text{rank}}(\cdot)$ and $\text{LLM}^{com}_{\text{rank}}(\cdot)$.

5. Experiment

Through our extensive empirical analysis, we aim to address the following research questions.

- **RQ1:** How does AgentDR perform in full-ranking recommendation compared to other baselines?
- **RQ2:** Does incorporating LLM-based reasoning modules improve the semantic alignment of recommended items with user intent?
- **RQ3:** How do different ranking aggregation strategies affect the performance of AgentDR?
- **RQ4:** What is the individual contribution of each LLM-based module in AgentDR?

5.1. Experiment Settings.

5.1.1. *Datasets.* We conduct experiments on three datasets: Instacart [30], Electronics and Sports [2]. Electronics and Sports are two sub-datasets from public XMarket dataset [2]. The statistics are summarized in Table 6.9.

Dataset	Instacart	Electronics	Sports
User #	6,987	4,966	10,873
Item #	2,108	14,635	4,306
Interaction #	5,105,339	91,819	55,990
Density	34.663%	0.126%	0.120%

TABLE 6.9. Statistics of datasets for AgentDR.

5.1.2. *Tool selection.* In this work, three pretrained recommendation models are used as tools within AgentDR: **(1) LightGCN** [18], **(2) SASRec** [31], and **(3) SimpleX** [43]. The simplicity of these tools allows us to validate the effectiveness of our framework without relying on complex architectures, aligning with our goal of maintaining a model-agnostic design.

5.1.3. *Baselines.* Besides the individual recommendation tools, we compare AgentDR with eight other baselines. First, other representative MF-based, diffusion-based, and sequential models are used as tradition recommendation baselines:

(4) ENMF [9] introduces mathematical optimization that enable efficient full-data MF without negative sampling.

(5) DiffRec [72] is a diffusion-based recommendation model that generates user interactions in a denoising manner.

(6) FEARec [12] enhances self-attention with frequency-aware mechanisms to capture high-frequency signals and periodic patterns for sequential recommendation.

Both tools and recommendation baselines are pretrained on the first $(n-k)$ items of each user's interaction sequence $\mathbf{s}_{[:-k]}$, following the standard training protocol of sequential recommendation for fair performance comparison. Following the previous works [97, 82], we also compare AgentDR with language-based approaches:

(7) BM25 [55] ranks candidates according to their textual similarity with user historical interactions.

(8) LLMRank [21] uses the LLM as a zero-shot ranker, considering user sequential interaction histories as conditions.

In addition to these language-based baselines, we compare AgentDR against recommendation tools integrated with Retrieval-Augmented Generation (RAG). Specifically, the LLM is prompted to generate the top k' items from the 50 candidates retrieved by each tool, conditioned on the user's behavior sequence. The same hallucination filtering mechanism from Section 4.6 is applied for fair comparison. These baselines are represented as **(9)-(11)** $\mathbf{RAG}_{G/S/M}$ for LightGCN, SASRec and SimpleX, respectively. Since most prior LLM-agent

works [97, 73, 26, 96, 67, 82] are unable to rank all items in datasets, we exclude them from the comparison table instead of reporting trivially low full-ranking performance.

5.1.4. *Evaluation Metrics.* Recall@{10,20} and NDCG@{10,20} are adopted for ranking performance evaluation. Additionally, we propose a novel metric that measures the semantic vicinity of the predicted item list to the ground-truth item in the following Section 5.3.

5.1.5. *Implementation Details.* Since most item titles in the three grocery datasets used in experiments are self-explainable, we use titles as textual descriptions of items and remove all the items without titles. For each dataset, users' interactions are chronologically organized based on timestamps to construct user behavior sequences. Since our framework delegates full-ranking sequential recommendation tasks to existing models, we follow the same data preprocessing strategy in previous works [18, 31, 43] to remove cold-start users and items. The recommendation tools are pretrained on the first $(n-k)$ items of each user's interaction sequence $\mathbf{s}_{[:-k]}$, following the standard training protocol in recommendation where models learn from early interactions and are evaluated on their ability to predict future behaviors. This setup ensures consistency across tools and enables fair performance comparison between our framework and other recommendation baselines. A grid search is applied for searching learning rates α, β and γ in {1e-3, 5e-3, 1e-2, 5e-2, 1e-1}. To mitigate overfitting, each learning rate is multiplied by a decay factor of 0.8 after every optimization epoch. The number of historical items c used for substitute and complement generation is set to 10. Following the training scheme in sequential recommendation, the number of target items k is set to 1 during optimization. Since the evaluation metrics consider at most top 20 items, the number of reranking candidates k' is fixed at 20. All experiments are conducted using the Phi-4 language model, deployed locally with vLLM on eight Tesla V100 GPUs, each with 32 GB of VRAM. The temperature is set to 0 to ensure deterministic outputs. Aside from the complexity of the recommendation tools described in Section 3.2, the quantized version of a single Phi-4 model requires less than 12 GB of VRAM. Following prior LLM-agent works [97, 67, 26], we train the recommendation tools on the entire training set, and randomly sample 160 agents for optimization on each dataset.

5.2. Cost Analysis. The total LLM API calls in AgentDR consist of three parts: generation, agent optimization, and reranking. Generation (Eq. (28), Eq. (29) and Eq. (30)) and reranking (Eq. (37), Eq. (39), and Eq. (40)) contain 6 calls per user. Each epoch in agent optimization contains 3 calls (Eq. (31), Eq. (35) and Eq. (36)) per user. Hence, the total number of calls is $6|\mathcal{U}| + 3|\mathcal{U}|T_{\text{opt}}$, where $|\mathcal{U}|$ is the number of users and T_{opt} is the number of optimization epochs. With all the agent memory updated, AgentDR takes less than 1 second to generate the final ranking list for each user. We acknowledge that LLM-based agents generally have higher inference latency than embedding-based recommendation models, but they are still practical for offline ranking or secondary-stage recommendation pipelines.

5.3. Semantic Vicinity Evaluation: VDCG. Unlike ID-based recommenders, language-based methods prioritize semantic similarity over behavioral patterns. As a result, the item lists they generate may better reflect a user's interests, even if the exact target items are absent. For example, for a user interested in a black baseball helmet, a list of baseball helmets in other colors is clearly more relevant than a list of bicycle helmets, despite both yielding zero recall and NDCG. To better capture the semantic vicinity to the user intent, we propose a new LLM-based evaluation metric, Vicinity-DCG (VDCG), which jointly assesses the semantic alignment and ranking correctness of the predicted item list.

[Instruction]

Given the same category, rate how well each item from the recommended list matches the target item based on relevance, usefulness, and user interest.

[Category]

{item_category}

[Target Item]

{item_description}

[Recommended List]

{item_descriptions}

The output must be a list of integers. Each score must be between 0 (very unrelated) and 9 (exact match).

TABLE 6.10. Prompts in AgentDR for VDCG evaluation.

Specifically, we prompt the LLM to rate how well each item in the recommended list aligns with the ground-truth item in terms of relevance, usefulness, and user interest. The rating scale ranges from 0 to 9, where 0 indicates an entirely unrelated item, and 9 represents a perfect semantic match between the item descriptions. The prompts for VDCG evaluation are shown in Table 6.10. These ratings are treated as relevance scores to compute the DCG [5] for the list. For normalization, we divide by a customized ideal DCG, calculated from a synthetic ideal list of scores: $[p_{\max}, p_{\max}/2, \ldots, p_{\max}/2^l]$, where $p_{\max}$ is the maximum relevance score minus 1, and l is the length of the list. This ideal score distribution reflects the empirical tendency of the LLM to assign higher scores to only a few semantic-related items, following an approximately exponential decay. VDCG thus captures both the semantic vicinity and ranking quality of the predicted list with respect to the user's true intent.

5.4. RQ1: Ranking Performance Comparison. Performance comparison between AgentDR and other baselines are shown in Table 6.11. The best and second-best performances are in boldface and underlined. We have the following observations:

Dataset	Instacart				Electronics				Sports			
Metric	R@10	R@20	N@10	N@20	R@10	R@20	N@10	N@20	R@10	R@20	N@10	N@20
LightGCN	0.0500	0.1188	0.0221	0.0398	0.1188	0.1313	0.0734	0.0765	0.1438	0.1625	0.0997	0.1040
SASRec	0.0875	0.1500	0.0395	0.0552	0.1188	0.1313	0.0635	0.0667	0.1000	0.1438	0.0554	0.0667
SimpleX	0.0313	0.0500	0.0180	0.0226	0.1000	0.1063	0.0761	0.0776	0.1375	0.1563	0.0951	0.0999
ENMF	0.1125	0.1750	0.0711	0.0797	0.1250	0.1312	0.0881	0.0897	0.1375	0.1438	0.1172	0.1187
DiffRec	0.1188	0.1563	0.0773	0.0806	0.1562	0.1688	0.0971	0.0981	0.1125	0.1188	0.0875	0.0890
FEARec	0.1063	0.1875	0.0639	0.0845	0.1250	0.1625	0.0627	0.0726	0.1313	0.1563	0.0732	0.0797
BM25	0.0625	0.0875	0.0310	0.0370	0.1000	0.1313	0.0432	0.0514	0.1188	0.1250	0.0536	0.0552
LLMRank	0.0750	0.1000	0.0373	0.0436	0.1000	0.1250	0.0503	0.0565	0.0938	0.1063	0.0458	0.0488
RAG_G	0.1125	0.1563	0.0480	0.0594	0.1125	0.1438	0.0715	0.0748	0.1125	0.1188	0.0750	0.0768
RAG_S	0.0688	0.1063	0.0312	0.0405	0.1063	0.1188	0.0575	0.0605	0.1375	0.1625	0.0852	0.0913
RAG_M	0.0750	0.1125	0.0335	0.0427	0.0875	0.1250	0.0517	0.0613	0.1125	0.1375	0.0823	0.0871
AgentDR	**0.1563**	**0.2000**	**0.0796**	**0.0907**	**0.1688**	**0.2000**	**0.1180**	**0.1260**	**0.1750**	**0.2063**	**0.1247**	**0.1326**
Improv.	31.57%	6.67%	2.98%	7.34%	8.07%	18.48%	21.52%	28.44%	21.67%	26.95%	6.40%	11.71%

TABLE 6.11. Overall performance comparison of AgentDR with baselines.

- AgentDR exhibits superior performance across all datasets. Even compared with more advanced and complex recommendation methods such as DiffRec, it has up to 31.6% improvement in recall and up to 28.4% improvement in NDCG. When compared to its own base recommendation tools such as SASRec, AgentDR consistently achieves at least 33.3% improvement across all datasets. Since AgentDR's performance is inherently dependent on the underlying tools, we expect greater performance gains when integrated with more advanced recommenders. We leave the exploration of stronger underlying tools in future works, as any resulting performance gain would not stem from our agent design and thus falls outside the scope of this study.
- Language-only methods, such as BM25 and LLMRank, underperform compared to all recommendation baselines in most scenarios. This performance gap arises from their inability to effectively capture collaborative filtering signals or sequential behavioral patterns, both of which are essential for modeling personalized user preferences in large-scale item spaces. Lacking the ability to leverage historical interaction structures, these methods rely solely on surface-level text matching or general world knowledge, which limits their performance in full-ranking recommendation.
- Compared to AgentDR, RAG-based methods such as RAG_S, fail to consistently improve recommendation performance on all the datasets through the world knowledge of LLMs.

This is due to two main reasons. First, relying on a single recommender as the retrieval model leads the LLM to generate top items from a potentially biased candidate set, even if the set is larger than that used in AgentDR. Second, the behavior sequences fed to the LLM consist of item descriptions. These descriptions usually contain noises, such as abbreviations or symbols, that carry little to no preference-relevant information. In contrast, AgentDR reranks a smaller but higher-confidence candidate set aggregated from multiple recommendation tools and reasons over a summarized user profile with inferred intent, enabling more comprehensive and nuanced recommendations.

5.5. RQ2: VDCG Comparison. The VDCG comparison is presented in Table 6.12. Compared to the ranking-based evaluation results in Table 6.11, RAG-based methods demonstrate a much more pronounced advantage in VDCG over the base recommenders. Given their overall lower recall, this advantage primarily stems from the stronger semantic vicinity of their recommended item lists to user interests. This demonstrates that the world knowledge embedded in LLMs can be effectively harnessed to generate semantically meaningful recommendations aligned with user intent, which is especially valuable when the target item is difficult to predict from historical behavior. Similarly, language-only methods such as BM25 and LLMRank achieve competitive VDCG scores despite poor recall and NDCG performance, especially in the Electronics and Sports where item descriptions tend to be longer and more semantically distinguishable. In contrast, the relatively low VDCG scores of interaction-based recommenders in these datasets reveal a limitation not captured by standard recall or NDCG metrics. In real-world e-commercial services, a lack of semantic relevance in recommended lists may undermine user trust more severely than occasional misses. Addressing this gap, AgentDR actively reasons over implicit item-item relations, resulting in the highest overall semantic and ranking alignment, as reflected by its VDCG.
In Figure 6.3, we further examine the contribution of the LLM-based reasoning modules to VDCG by progressively removing them from the ranking refinement and agent optimization stages. The VDCG gap between AgentDR with and without the reranking modules highlights their importance in enhancing the semantic alignment between recommended item lists

Dataset	Instacart		Electronics		Sports	
Metric	V@5	V@10	V@5	V@10	V@5	V@10
BM25	0.2409	0.3220	0.2547	0.3188	0.5030	0.6075
LLMRank	0.2529	0.3224	0.2180	0.2932	0.4351	0.5341
LightGCN	0.3144	0.4039	0.1736	0.2344	0.4541	0.5702
SASRec	0.3143	0.4037	0.1748	0.2377	0.3898	0.4847
SimpleX	0.3091	0.3984	0.1735	0.2341	0.4551	0.5884
RAG_G	0.3363	0.4537	0.2656	0.3338	0.4119	0.5220
RAG_S	0.3300	0.4496	0.2569	0.3257	**0.5095**	**0.6094**
RAG_M	0.3306	0.4460	0.2100	0.2874	0.4670	0.5768
AgentDR	**0.4683**	**0.5743**	**0.3334**	**0.3846**	0.5085	0.5743
Improv.	39.25%	26.53%	25.90%	15.21%	-0.20%	-5.76%

TABLE 6.12. VDCG comparison of AgentDR with other baselines.

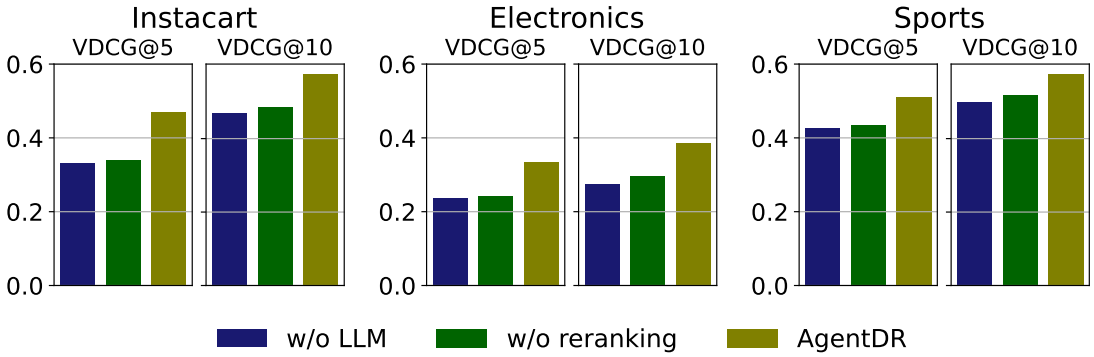


FIGURE 6.3. VDCG of AgentDR with and without LLM-based modules.

and user preferences. Notably, tool comparison consistently improves VDCG, as it increases the weight of the recommendation tool that produces the most semantically relevant item list during agent optimization.

5.6. RQ3: Aggregation Strategy Analysis.

5.6.1. *Setup.* The ranking comparison in Eq. (33) is a simple quantitative reflection mechanism to adjust RecTool weights for personalized tool suitability. This component is flexible and can be replaced with alternative learning-based ensemble methods. For instance, we can train a learnable vector $\mathbf{v} \in \mathbb{R}^{1\times|\mathcal{T}|}$ as adaptive tool weights for all users. This learnable vector is then list-wise multiplied by the ranking score lists $\hat{\mathbf{r}}_{\mathcal{T}} \in \mathbb{R}^{|\mathcal{T}|\times|\mathcal{I}|}$ of each user to obtain the aggregated ranking score list $\hat{\mathbf{r}}' \in \mathbb{R}^{1\times|\mathcal{I}|}$. Supervised training is performed using ground-truth target items in training set, with a hinge loss as the objective. To integrate

Dataset	Instacart		Electronics		Sports	
Metric	N@10	N@20	N@10	N@20	N@10	N@20
Ensemble: RC	0.0428	0.0633	0.0765	0.0874	0.0859	0.1009
+ AgentDR	0.0701	0.0825	0.1149	0.1227	0.1221	0.1286
Ensemble: LR	0.0401	0.0638	0.0862	0.0893	0.0950	0.1060
+ AgentDR	**0.0796**	**0.0907**	0.1136	0.1200	0.1246	0.1297
Ensemble: MLP	0.0397	0.0635	0.0754	0.0861	0.1009	0.1056
+ AgentDR	0.0747	0.0858	0.1071	0.1166	0.1247	0.1328
Ensemble: Att	0.0473	0.0692	0.0787	0.0878	0.1000	0.1078
+ AgentDR	0.0765	0.0844	**0.1180**	**0.1260**	**0.1247**	**0.1326**

TABLE 6.13. Ensemble methods with LLM reasoning modules from AgentDR.

this with the original framework, we normalize this $\hat{\mathbf{r}}'$ and $\hat{\mathbf{r}}$ from Eq. (34) and add them as an updated $\hat{\mathbf{r}}$ for the following reranking.

5.6.2. *Results.* Table 6.13 reports the performance of various ensemble strategies, both with and without all LLM-based reasoning modules from AgentDR. The original ranking comparison mechanism and the alternative linear model described above are denoted as *RC* and *LR*, respectively. Additionally, we explore MLP with and without attention mechanisms to capture inter-tool dependencies, as alternatives to the original ranking comparison. These variants are denoted as *Att* and *MLP* in the table. The results demonstrate that the proposed reasoning modules consistently provide significant benefits across different ensemble strategies. Although AgentDR equipped with the simplest ranking comparison mechanism already surpasses most baselines in Table 6.11, substituting it with more expressive ensemble methods yields up to a 13.56% improvement in NDCG. This performance gain comes at the cost of increased computational complexity during backpropagation. Among the evaluated methods, the MLP with an attention mechanism achieves the best overall performance when integrated into AgentDR, which has the the highest number of learnable parameters. We leave the exploration of potential improvement from integrating more advanced ensemble methods in the future work.

5.7. RQ4: Ablation Study.

5.7.1. *Tool Comparison and Dual S&C Reranking.* We investigate the contribution of the proposed tool comparison and dual S&C reranking modules, denoted as *T* and *Dual*

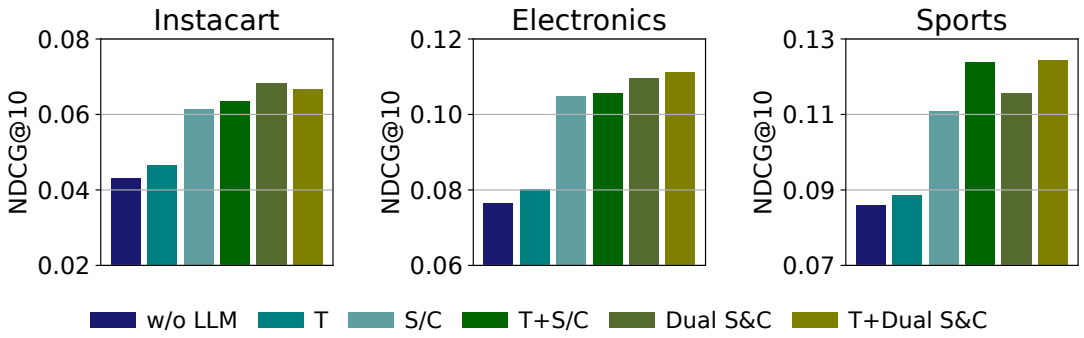


FIGURE 6.4. Ablation study on comparison and reranking modules in AgentDR.

S&C, respetively. We also explore the effect of replacing dual S&C by a mutually exclusive substitute or complement reranking based on intent memory of each user, denoted as *S/C*. The general reranking module is excluded from this study. The results are shown in Figure 6.4. We observe that tool comparison improves performance in most settings, while it may degrade performance on Instacart. In this dataset, the semantic differences between the grocery food lists recommended by different tools are less pronounced, providing weaker signals for LLM-based tool selection. Moreover, the dual S&C reranking consistently outperforms reranking based on either substitutes or complements alone across all settings, highlighting the advantage of a more comprehensive reranking strategy.

5.7.2. *General Reranking.* The general reranking module in Section 4.5.3 refines recommendation lists based on summarized user preferences, independent of substitution and complementarity signals. In Table 6.14, we compare the performance of AgentDR with and without this module under different reranking settings. The results show that general reranking consistently improves performance in most cases, and all best results across the three datasets are achieved with this module enabled, confirming the importance of general preference signals in complementing intent-specific reasoning. Since the corresponding intent identification module in Eq. (36) accounts for one-third of all inference calls during agent optimization, it can be optionally deactivated to save computational resources in this flexible agent framework, without sacrificing the general preference signals. In such cases, m_{reg} is set as a constant regularization coefficient shared across all users in the dataset.

Dataset	Instacart		Electronics		Sports	
Metric	N@10	N@20	N@10	N@20	N@10	N@20
S/C	0.0614	0.0740	0.1047	0.1076	0.1107	0.1174
+ General	0.0676	0.0817	0.1026	0.1120	0.1109	0.1190
T+S/C	0.0634	0.0794	0.1054	0.1136	0.1238	0.1320
+ General	0.0626	0.0800	0.1055	0.1149	**0.1244**	**0.1323**
Dual S&C	0.0682	0.0805	0.1095	0.1174	0.1155	0.1203
+ General	0.0684	0.0824	0.1050	0.1109	0.1190	0.1238
T+Dual S&C	0.0666	0.0793	0.1112	0.1211	0.1240	0.1307
+ General	**0.0701**	**0.0825**	**0.1149**	**0.1227**	0.1221	0.1286

TABLE 6.14. Performance of AgentDR with and without general reranking.

6. Chapter Summary

In this chapter, we propose a novel LLM-based agent framework, AgentDR, for full-ranking recommendation. It leverages the reasoning capabilities of LLMs to delegate the full-ranking task to multiple recommendation tools in a personalized manner. Furthermore, the commonsense world knowledge of the LLM is utilized to infer user intent regarding substitutes and complements, which is a challenging aspect for traditional models to capture effectively for recommendation enhancement. Extensive experiments are conducted with a proposed LLM-based evaluation metric VDCG to evaluate both semantic relevance and ordering correctness of recommendation results. We extend this tool learning perspective in PRTA [88], where a reflection mechanism lets the agent assess which recommendation tools serve each user best from user profiles and ranked candidates. Similar to ERLSC, a limitation of this framework is that substitution and complementarity relationships are more prevalent in domains like groceries than in others such as movies or music. The reasoning modules also depend on the particular LLM queried, and the evaluation protocol replays historical interactions offline, which cannot observe how tool selection would drift as a catalog evolves or how the agent would behave when its available tools disagree.

CHAPTER 7

Conclusion

This dissertation investigates how to enhance personalized RecSys by coupling graph learning with the semantic knowledge and the reasoning ability of language models. Traditional RecSys rely mainly on user-item interaction data, which is sparse and carries little information about what the items themselves mean. The five frameworks presented here address this gap from different directions, first by enriching the relational structure that a recommendation model can see, then by injecting semantic guidance into that structure, and finally by moving language model reasoning to the decision stage.

1. Summary of Contributions

- We develop a unified multitask pretraining framework UPRTH for recommendation based on task hypergraphs. It models diverse auxiliary tasks as hyperedge prediction problems and employs a transitional attention mechanism to selectively transfer useful knowledge, enabling effective integration of heterogeneous relational signals and alleviating data sparsity.
- We introduce an instruction-based hypergraph pretraining framework IHP to incorporate semantic task guidance into hypergraphs for recommendation. By encoding textual instructions with a pretrained language model and injecting them into hypergraph convolution, IHP aligns structural relations with semantic task descriptions and improves cross-domain knowledge transfer in recommendation.
- We propose a graph-language alignment framework GLTA to integrate semantic knowledge from LLMs into recommendation models. It aligns graph-based user and item representations with token embeddings from pretrained language models, allowing recommendation

models to inherit semantic knowledge while preserving the efficiency and determinism of ID-based item prediction.

- We design an LLM-guided relational modeling method ERLSC to capture substitute and complementary item relations. It leverages LLMs to infer functional relations between items and integrates these signals into hypergraph convolution through adaptive hyperedge weighting, enabling recommendation models to emphasize semantically meaningful item groups.
- We propose an LLM-agent framework for dynamic recommendation AgentDR. It treats the LLM as a high-level decision maker that coordinates multiple recommendation tools such as graph-based CF, and performs reasoning over item relations to refine candidate items, improving recommendation quality while maintaining scalability.

2. Cross-Chapter Synthesis and Findings

Although the five frameworks were developed and evaluated in distinct research settings, reading their findings together reveals several broader conclusions that no single chapter establishes on its own.

The first observation concerns the modeling interface. Hypergraphs recur in Chapter 2, Chapter 3, and Chapter 5, and in each case they carry a different kind of relation. A hyperedge is a pretext task in the first, a task shared between two domains in the second, and an item group induced by co-occurrence in the third. The three frameworks retain a common node-hyperedge aggregation backbone while adapting it to different structural, semantic, and task-specific signals, which suggests that the hypergraph is acting less as a particular model and more as a general interface through which heterogeneous and higher-order relations can be attached to a recommendation model without redesigning it. The same interface also turns out to be a convenient place to deposit knowledge that comes from outside the interaction data, since a hyperedge weight is a natural carrier for a semantic judgment.

The second observation concerns the role that the language model plays. Chapter 2 transfers knowledge purely structurally, by learning which auxiliary tasks are relevant to recommendation. Chapter 3 shows that describing those tasks in text and encoding the descriptions with a PLM supplies information that structure alone does not carry, which matters most when two domains share users but no items. Chapter 4 moves the same idea to a finer granularity by aligning representations with individual language tokens rather than with one vector per task. Chapter 5 changes the kind of knowledge rather than its granularity, asking an LLM about item function instead of about task semantics. Chapter 6 changes the timing, asking the model to decide rather than to encode. The progression from structural transfer to semantic guidance and then to explicit reasoning is therefore not a single axis of increasing model size, but a sequence of choices about what is asked of the language model and when.

The third observation concerns cost. Coupling becomes tighter from Chapter 3 to Chapter 4, and the computational burden rises with it, because token alignment requires gradients to pass through the language model while prompt encoding requires only a forward pass that can be cached. Chapter 5 and Chapter 6 deliberately reverse this direction. Offline relation inference pays for the language model once and reuses its output as a fixed structural signal, and inference-time reranking touches only a bounded candidate list. Both remain useful without backpropagating through a model whose size keeps growing, which is the practical reason the tightest coupling is not always the right target.

The fourth observation concerns grounding. Every framework in this dissertation returns item identifiers that exist in the catalog, and each achieves this differently. GLTA predicts through logits matching over the aligned token space instead of generating item names, and AgentDR delegates full-catalog ranking to conventional retrievers and confines the language model to reordering what those retrievers return. This is what allows the agent to reason without hallucinating items, and it suggests that grounding is better treated as an architectural constraint than as a post hoc filter.

Taken together, these observations indicate that no single coupling strategy dominates. Tight representation-level coupling is preferable when the recommendation model is trained from scratch and the semantic content of items is rich enough to justify the training cost. Offline relation inference is preferable when the useful knowledge is relational and stable, since it can be computed once and reused. Decision-level reasoning is preferable when several retrieval signals are available and the difficulty lies in choosing among them for a particular user. Because the five studies use different datasets, different baselines, and different evaluation protocols, their numbers are not directly comparable, and the conclusion drawn here is about where each strategy applies rather than about which one scores higher.

3. Future Directions

The five frameworks sit within a longer line of work for which I am also the primary author, and which shaped the questions this dissertation asks. Ranking-based group identification required item interactions and group participation to be modeled within a single social tripartite graph, which made clear that useful relational evidence often connects more than two entities at once and motivated the hypergraph formulation used throughout [89]. Transitional hypergraph convolution then let preferences learned over one type of entity serve as prior knowledge for another, and it reappears as the transitional attention layer of Chapter 2 [85]. Prompt-based continual learning showed that a compact prompt can retain task-specific information while a user model is updated, a role that plain-text instructions play again in Chapter 3 [87]. Personalized tool learning showed that an agent can reflect on its own ranking outcomes to decide which recommendation model to trust for a given user, a mechanism that Chapter 6 carries over to substitute and complement intent [88]. Each thread began with a structural question and arrived at a semantic one, which is the same movement the chapters here follow, and the directions below continue it.

One direction concerns the robustness of signals derived from language models. A judgment about item function is stored as a hyperedge weight in Chapter 5 and acted upon at inference time in Chapter 6, and in both cases a wrong judgment is treated exactly like a correct one.

Separating the contribution of the relation type from the contribution of the model that supplied it, and characterizing how performance degrades as annotation quality falls, would show how much of the observed gain transfers across language models.

A second direction concerns cost. Token alignment, offline annotation, and inference-time reranking incur costs of different kinds, respectively in training compute, in one-time annotation, and in per-request latency, and this dissertation compares them qualitatively rather than on a common scale. A systematic accounting would make the choice among coupling strategies easier to justify for a given deployment.

A third direction concerns evaluation. The protocols used throughout are static and offline, replaying historical interactions and measuring ranking quality against held-out items. This suits the models studied here but only partially suits an agent whose tools and prompts could change between sessions. Offline replay cannot observe how a user would respond to an explanation, how tool selection would drift as a catalog evolves, or how an agent would behave when its available tools disagree.

The final direction is one this dissertation can only begin. Existing RecSys operate within a single platform or dataset, whereas a user-owned agent could aggregate information across multiple online services and reason about the most suitable choices for the user. Two obstacles have kept this out of reach. Technically, earlier models lacked the capability to integrate and reason over heterogeneous information sources at scale. From an ecosystem perspective, modern RecSys are largely developed by individual service providers whose objectives center on business metrics such as user engagement and conversion, which leaves platform-independent recommendation poorly aligned with existing incentives.

Recent advances in LLMs may begin to change the technical side of this situation. The ability to reason over diverse information sources makes it possible to build agents that assist users across platforms, and the design of Chapter 6 is a step in that direction, since an agent that coordinates external retrieval tools does not need to own the data those tools index. At the same time, the cost of developing and deploying such agents continues to

decrease, and the tools for building them are becoming more accessible, which suggests that users may eventually be able to build or customize their own recommendation agents.

Realizing this would require progress on the concerns raised above. Cross-platform aggregation raises questions of data access and privacy that a single-platform evaluation never encounters, since an agent acting on behalf of a user must read from services that were not designed to share. The dependence of LLM-derived signals on the querying model becomes more consequential when the agent, rather than the platform, is the party the user trusts. Static offline evaluation is least adequate exactly in this setting, so protocols that measure how such agents behave over time, and how much control they actually return to users, would need to be developed alongside the methods themselves.

If these problems can be addressed, user-owned recommendation agents may represent a meaningful step toward giving users greater control over how information is filtered and recommended, and toward mitigating practical concerns associated with platform-centric RecSys, such as data privacy risks and information cocoons. It is our hope that the ideas explored in this dissertation will inspire future research toward RecSys that not only understand user preferences, but also reason about information in a way that better serves users in an increasingly complex digital ecosystem.

APPENDIX A

ACM Author's Reuse Rights

Authors can reuse any portion of their own work in a new work of their own (and no fee is expected) as long as a citation and DOI pointer to the Version of Record in the ACM Digital Library are included.
Authors can include partial or complete papers of their own (and no fee is expected) in a dissertation as long as citations and DOI pointers to the Versions of Record in the ACM Digital Library are included. Authors can use any portion of their own work in presentations and in the classroom (and no fee is expected).[1]

[1] https://authors.acm.org/author-resources/author-rights

APPENDIX B

Use of Generative AI and Language Models

Pretrained and generative language models serve as computational components and sources of semantic knowledge in this dissertation. Chapter 2 does not use a language model in its proposed method. Chapters 3 through 6 incorporate language models in different roles, as described below.

In Chapter 3, PLMs are used as text encoders. Node descriptions and textual task instructions are encoded into semantic representations that guide prompt-based hypergraph pretraining. The experiments compare several pretrained text encoders, with MiniLM used as the sentence encoder in the reported IHP framework.

In Chapter 4, a quantized LLaMA-2-7B model is used as a frozen language-model backbone. Its pretrained token space supports graph-language token alignment, and the model is used to generate profile and prediction tokens from item descriptions. The resulting token representations are incorporated into ID-grounded item prediction through the GLTA and GLLM components described in that chapter.

In Chapter 5, Phi-4 is queried to assess mutually exclusive substitute and complement relations between item pairs. These assessments are used to calculate relation-composition statistics and initialize hyperedge weights in ERLSC. The language-model outputs are therefore treated as model-generated semantic signals within the proposed recommendation method.

In Chapter 6, Phi-4 is also used as the language-model component of AgentDR. It is prompted to summarize user profiles, generate substitute and complement candidates, compare recommendation tools, infer user intent, rerank catalog-grounded candidate items, and assess semantic vicinity for the VDCG evaluation. Conventional recommendation tools remain responsible for scalable full-catalog retrieval, while the language model operates on the bounded inputs described in the chapter.

During preparation of this dissertation, Claude Opus 5 was used solely to assist with language editing and LaTeX formatting adjustments. It was not used to generate experimental data or results, alter the research methodology, invent citations, or determine the scientific conclusions of the dissertation. I reviewed and confirmed every AI-assisted modification and take full responsibility for the final text, analysis, and formatting.

BIBLIOGRAPHY

[1] Cristian Bodnar et al. "Neural sheaf diffusion: a topological perspective on heterophily and oversmoothing in GNNs". In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. NIPS '22. New Orleans, LA, USA: Curran Associates Inc., 2022.

[2] Hamed R. Bonab et al. "Cross-Market Product Recommendation". In: *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management, Virtual Event, Queensland, Australia, November 1 - 5, 2021*. Ed. by Gianluca Demartini et al. ACM, 2021, pp. 110–119.

[3] Tom B. Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. Ed. by Hugo Larochelle et al. 2020.

[4] Jiajun Bu et al. "Music Recommendation by Unified Hypergraph: Combining Social Media Information and Music Content". In: *Proceedings of the 18th ACM International Conference on Multimedia*. MM '10. Firenze, Italy: Association for Computing Machinery, 2010, pp. 391–400.

[5] Chris Burges et al. "Learning to rank using gradient descent". In: *Proceedings of the 22nd International Conference on Machine Learning*. ICML '05. Bonn, Germany: Association for Computing Machinery, 2005, pp. 89–96.

[6] Yuwei Cao et al. "Aligning Large Language Models with Recommendation Knowledge". In: *Findings of the Association for Computational Linguistics: NAACL 2024*. Mexico City, Mexico: Association for Computational Linguistics, June 2024, pp. 1051–1066.

[7] Yuwei Cao et al. "Multi-task Item-attribute Graph Pre-training for Strict Cold-start Item Recommendation". In: *Proceedings of the 17th ACM Conference on Recommender Systems, RecSys 2023, Singapore, Singapore, September 18-22, 2023*. Ed. by Jie Zhang et al. ACM, 2023, pp. 322–333.

[8] Haw-Shiuan Chang, Nikhil Agarwal, and Andrew McCallum. "To Copy, or not to Copy; That is a Critical Issue of the Output Softmax Layer in Neural Sequential Recommenders". In: *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. WSDM '24. Merida, Mexico: Association for Computing Machinery, 2024, pp. 67–76.

[9] Chong Chen et al. "Efficient Neural Matrix Factorization without Sampling for Recommendation". In: *ACM Trans. Inf. Syst.* 38.2 (2020), 14:1–14:28.

[10] Tong Chen et al. "Try This Instead: Personalized and Interpretable Substitute Recommendation". In: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '20. Virtual Event, China: Association for Computing Machinery, 2020, pp. 891–900.

[11] Zeyu Cui et al. "M6-Rec: Generative Pretrained Language Models are Open-Ended Recommender Systems". In: *CoRR* abs/2205.08084 (2022).

[12] Xinyu Du et al. "Frequency Enhanced Hybrid Attention Network for Sequential Recommendation". In: *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2023, Taipei, Taiwan, July 23-27, 2023*. Ed. by Hsin-Hsi Chen et al. ACM, 2023, pp. 78–88.

[13] Yifan Feng et al. "Hypergraph Neural Networks". In: *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*. AAAI Press, 2019, pp. 3558–3565.

[14] Pengyu Gao et al. "An Efficient Context-Dependent Memory Framework for LLM-Centric Agents". In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*. Ed. by Weizhu Chen et al. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 1055–1069.

[15] Yue Gao et al. "Hypergraph Learning: Methods and Practices". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.5 (2022), pp. 2548–2566.

[16] Shijie Geng et al. "Recommendation as Language Processing (RLP): A Unified Pretrain, Personalized Prompt & Predict Paradigm (P5)". In: *Proceedings of the 16th ACM Conference on Recommender Systems*. RecSys '22. Seattle, WA, USA: Association for Computing Machinery, 2022, pp. 299–315.

[17] William L. Hamilton, Rex Ying, and Jure Leskovec. "Inductive representation learning on large graphs". In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Long Beach, California, USA: Curran Associates Inc., 2017, pp. 1025–1035.

[18] Xiangnan He et al. "LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation". In: *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. ACM, 2020, pp. 639–648.

[19] Zhongmou He et al. "LinkGPT: Leveraging Large Language Models for Enhanced Link Prediction in Text-Attributed Graphs". In: *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. CIKM '25. Seoul, Republic of Korea: Association for Computing Machinery, 2025, pp. 843–853.

[20] Kasra Hosseini et al. "Retrieve, Annotate, Evaluate, Repeat: Leveraging Multimodal LLMs for Large-Scale Product Retrieval Evaluation". In: *Advances in Information Retrieval: 47th European Conference on Information Retrieval, ECIR 2025, Lucca, Italy, April 6–10, 2025, Proceedings, Part I*. Lucca, Italy: Springer-Verlag, 2025, pp. 149–163.

[21] Yupeng Hou et al. "Large Language Models are Zero-Shot Rankers for Recommender Systems". In: *Advances in Information Retrieval - 46th European Conference on Information Retrieval, ECIR 2024, Glasgow, UK, March 24-28, 2024, Proceedings, Part II*. Ed. by Nazli Goharian et al. Vol. 14609. Lecture Notes in Computer Science. Springer, 2024, pp. 364–381.

[22] Zhenyu Hou et al. "GraphMAE: Self-Supervised Masked Graph Autoencoders". In: *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data*

Mining, Washington, DC, USA, August 14 - 18, 2022. Ed. by Aidong Zhang and Huzefa Rangwala. ACM, 2022, pp. 594–604.

[23] Weihua Hu et al. "Strategies for Pre-training Graph Neural Networks". In: *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.

[24] Ziniu Hu et al. "GPT-GNN: Generative Pre-Training of Graph Neural Networks". In: *KDD '20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, CA, USA, August 23-27, 2020*. Ed. by Rajesh Gupta et al. ACM, 2020, pp. 1857–1867.

[25] Lei Huang et al. "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions". In: *ACM Trans. Inf. Syst.* 43.2 (2025), 42:1–42:55.

[26] Xu Huang et al. "Recommender AI Agent: Integrating Large Language Models for Interactive Recommendations". In: *ACM Trans. Inf. Syst.* (Apr. 2025). ISSN: 1046-8188.

[27] Shuyi Ji et al. "Dual Channel Hypergraph Collaborative Filtering". In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '20. Virtual Event, CA, USA: Association for Computing Machinery, 2020, pp. 2020–2029.

[28] Tong Jian et al. "Multi-task GNN for Substitute Identification". In: *Companion of The Web Conference 2022, Virtual Event / Lyon, France, April 25 - 29, 2022*. Ed. by Frédérique Laforest et al. ACM, 2022, pp. 228–231.

[29] Xunqiang Jiang et al. "Pre-training on Large-Scale Heterogeneous Graph". In: *KDD '21: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, Singapore, August 14-18, 2021*. Ed. by Feida Zhu, Beng Chin Ooi, and Chunyan Miao. ACM, 2021, pp. 756–766.

[30] Kaggle. *Kaggle Datasets by Yasser H.* Accessed: 2026-01-25. 2022.

[31] Wang-Cheng Kang and Julian J. McAuley. "Self-Attentive Sequential Recommendation". In: *IEEE International Conference on Data Mining, ICDM 2018, Singapore, November 17-20, 2018*. IEEE Computer Society, 2018, pp. 197–206.

[32] Nicolas Keriven. "Not too little, not too much: a theoretical analysis of graph (over)smoothing". In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. NIPS '22. New Orleans, LA, USA: Curran Associates Inc., 2022.

[33] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *Proceedings of the 3rd International Conference on Learning Representations*. 2015.

[34] Thomas N Kipf and Max Welling. "Semi-supervised classification with graph convolutional networks". In: 2016.

[35] Hang Li et al. "Knowledge Tagging with Large Language Model Based Multi-Agent System". In: *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*. Ed. by Toby Walsh, Julie Shah, and Zico Kolter. AAAI Press, 2025, pp. 28775–28782.

[36] Li Tao Li et al. "Dynamic Neural Control Flow Execution: an Agent-Based Deep Equilibrium Approach for Binary Vulnerability Detection". In: *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM*

2024, Boise, ID, USA, October 21-25, 2024. Ed. by Edoardo Serra and Francesca Spezzano. ACM, 2024, pp. 1215–1225.

[37] Junyang Lin et al. "M6: Multi-Modality-to-Multi-Modality Multitask Mega-transformer for Unified Pretraining". In: *KDD '21: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, Singapore, August 14-18, 2021*. Ed. by Feida Zhu, Beng Chin Ooi, and Chunyan Miao. ACM, 2021, pp. 3251–3261.

[38] Pengfei Liu et al. "Pre-Train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing". In: *ACM Comput. Surv.* 55.9 (Jan. 2023). ISSN: 0360-0300.

[39] Yiding Liu et al. "Decoupled Graph Convolution Network for Inferring Substitutable and Complementary Items". In: *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. CIKM '20. Virtual Event, Ireland: Association for Computing Machinery, 2020, pp. 2621–2628.

[40] Zemin Liu et al. "GraphPrompt: Unifying Pre-Training and Downstream Tasks for Graph Neural Networks". In: *Proceedings of the ACM Web Conference 2023*. WWW '23. Austin, TX, USA: Association for Computing Machinery, 2023, pp. 417–428.

[41] Donald Loveland et al. "On the Role of Weight Decay in Collaborative Filtering: A Popularity Perspective". In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*. KDD '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1975–1986.

[42] Yuanfu Lu et al. "Learning to Pre-train Graph Neural Networks". In: *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*. AAAI Press, 2021, pp. 4276–4284.

[43] Kelong Mao et al. "SimpleX: A Simple and Strong Baseline for Collaborative Filtering". In: *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management, Virtual Event, Queensland, Australia, November 1 - 5, 2021*. Ed. by Gianluca Demartini et al. ACM, 2021, pp. 1243–1252.

[44] Kelong Mao et al. "UltraGCN: Ultra Simplification of Graph Convolutional Networks for Recommendation". In: *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. CIKM '21. Virtual Event, Queensland, Australia: Association for Computing Machinery, 2021, pp. 1253–1262.

[45] Favour Nerrise et al. "GraFS: An Integrated GNN-LLM Approach for Inferring Best Functional Substitute Products". In: *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. CIKM '25. Seoul, Republic of Korea: Association for Computing Machinery, 2025, pp. 5047–5051.

[46] Jianmo Ni, Jiacheng Li, and Julian J. McAuley. "Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. Association for Computational Linguistics, 2019, pp. 188–197.

[47] Jianmo Ni et al. "Large Dual Encoders Are Generalizable Retrievers". In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Ed.

by Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 9844–9855.

[48] Mark O'Neill et al. "Condensing steam: Distilling the diversity of gamer behavior". In: *Proceedings of the 2016 internet measurement conference*. 2016, pp. 81–95.

[49] Haoyu Pang et al. "PNMTA: A Pretrained Network Modulation and Task Adaptation Approach for User Cold-Start Recommendation". In: *Proceedings of the ACM Web Conference 2022*. WWW '22. Virtual Event, Lyon, France: Association for Computing Machinery, 2022, pp. 348–359.

[50] Rastislav Papso. "Complementary Product Recommendation for Long-tail Products". In: *Proceedings of the 17th ACM Conference on Recommender Systems*. RecSys '23. Singapore, Singapore: Association for Computing Machinery, 2023, pp. 1305–1311.

[51] Jiezhong Qiu et al. "GCC: Graph Contrastive Coding for Graph Neural Network Pre-Training". In: *KDD '20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, CA, USA, August 23-27, 2020*. Ed. by Rajesh Gupta et al. ACM, 2020, pp. 1150–1160.

[52] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*. Ed. by Kentaro Inui et al. Association for Computational Linguistics, 2019, pp. 3980–3990.

[53] Xubin Ren et al. "Representation Learning with Large Language Models for Recommendation". In: *Proceedings of the ACM on Web Conference*. ACM, 2024, pp. 3464–3475.

[54] Steffen Rendle et al. "BPR: Bayesian personalized ranking from implicit feedback". In: *arXiv preprint arXiv:1205.2618* (2012).

[55] Stephen E. Robertson and Hugo Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Found. Trends Inf. Retr.* 3.4 (2009), pp. 333–389.

[56] Yedan Shen et al. "Alleviating LLM-based Generative Retrieval Hallucination in Alipay Search". In: *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '25. Padua, Italy: Association for Computing Machinery, 2025, pp. 4294–4298.

[57] Yuchen Shi et al. "AgentRE: An Agent-Based Framework for Navigating Complex Information Landscapes in Relation Extraction". In: *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM 2024, Boise, ID, USA, October 21-25, 2024*. Ed. by Edoardo Serra and Francesca Spezzano. ACM, 2024, pp. 2045–2055.

[58] Noah Shinn et al. "Reflexion: language agents with verbal reinforcement learning". In: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. Ed. by Alice Oh et al. 2023.

[59] Hongjin Su et al. "One Embedder, Any Task: Instruction-Finetuned Text Embeddings". In: *Findings of the Association for Computational Linguistics: ACL 2023*. Ed. by Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki. Toronto, Canada: Association for Computational Linguistics, July 2023, pp. 1102–1121.

[60] Kai Sugahara, Chihiro Yamasaki, and Kazushi Okamoto. "Is It Really Complementary? Revisiting Behavior-based Labels for Complementary Recommendation". In: *Proceedings of the 18th ACM Conference on Recommender Systems*. RecSys '24. Bari, Italy: Association for Computing Machinery, 2024, pp. 1091–1095.

[61] Mingchen Sun et al. "GPPT: Graph Pre-training and Prompt Tuning to Generalize Graph Neural Networks". In: *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*. Ed. by Aidong Zhang and Huzefa Rangwala. ACM, 2022, pp. 1717–1727.

[62] Xiangguo Sun et al. "All in One: Multi-Task Prompting for Graph Neural Networks". In: *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. KDD '23. ¡conf-loc¿, ¡city¿Long Beach¡/city¿, ¡state¿CA¡/state¿, ¡country¿USA¡/country¿, ¡/conf-loc¿: Association for Computing Machinery, 2023, pp. 2120–2131.

[63] Jiabin Tang et al. "GraphGPT: Graph Instruction Tuning for Large Language Models". In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2024, pp. 491–500.

[64] Ashish Vaswani et al. "Attention is All you Need". In: *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*. Ed. by Isabelle Guyon et al. 2017, pp. 5998–6008.

[65] Mengting Wan et al. "Fine-Grained Spoiler Detection from Large-Scale Review Corpora". In: *Proceedings of the 57th Conference of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019, pp. 2605–2610.

[66] Chenyang Wang et al. "Towards Representation Alignment and Uniformity in Collaborative Filtering". In: *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*. Ed. by Aidong Zhang and Huzefa Rangwala. ACM, 2022, pp. 1816–1825.

[67] Lei Wang et al. "User Behavior Simulation with Large Language Model-based Agents". In: *ACM Trans. Inf. Syst.* 43.2 (Jan. 2025). ISSN: 1046-8188.

[68] Liang Wang et al. "Text Embeddings by Weakly-Supervised Contrastive Pre-training". In: *CoRR* abs/2212.03533 (2022).

[69] Liyuan Wang et al. "AFEC: Active Forgetting of Negative Transfer in Continual Learning". In: *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*. Ed. by Marc'Aurelio Ranzato et al. 2021, pp. 22379–22391.

[70] Shoujin Wang et al. "Graph Learning based Recommender Systems: A Review". In: *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*. Ed. by Zhi-Hua Zhou. Survey Track. International Joint Conferences on Artificial Intelligence Organization, Aug. 2021, pp. 4644–4652.

[71] Wenhui Wang et al. "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers". In: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems*. 2020.

[72] Wenjie Wang et al. "Diffusion Recommender Model". In: *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information*

Retrieval, SIGIR 2023, Taipei, Taiwan, July 23-27, 2023. Ed. by Hsin-Hsi Chen et al. ACM, 2023, pp. 832–841.

[73] Yancheng Wang et al. "RecMind: Large Language Model Powered Agent For Recommendation". In: *Findings of the Association for Computational Linguistics: NAACL 2024*. Ed. by Kevin Duh, Helena Gomez, and Steven Bethard. Mexico City, Mexico: Association for Computational Linguistics, June 2024, pp. 4351–4364.

[74] Yaqing Wang et al. "Solving the sparsity problem in recommendations via cross-domain item embedding based on co-clustering". In: *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*. 2019, pp. 717–725.

[75] Wei Wei et al. "LLMRec: Large Language Models with Graph Augmentation for Recommendation". In: *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. ACM, 2024, pp. 806–815.

[76] Huizi Wu, Cong Geng, and Hui Fang. "Session-based recommendation by exploiting substitutable and complementary relationships from multi-behavior data". In: *Data Min. Knowl. Discov.* 38.3 (Dec. 2023), pp. 1193–1221. ISSN: 1384-5810.

[77] Jiancan Wu et al. "Self-Supervised Graph Learning for Recommendation". In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '21. Virtual Event, Canada: Association for Computing Machinery, 2021, pp. 726–735.

[78] Shirley Wu et al. "AvaTaR: Optimizing LLM Agents for Tool Usage via Contrastive Reasoning". In: *Advances in Neural Information Processing Systems*. Ed. by A. Globerson et al. Vol. 37. Curran Associates, Inc., 2024, pp. 25981–26010.

[79] Lianghao Xia et al. "Hypergraph Contrastive Collaborative Filtering". In: *SIGIR '22: The 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2022, pp. 70–79.

[80] Da Xu et al. "Product Knowledge Graph Embedding for E-commerce". In: *Proceedings of the 13th International Conference on Web Search and Data Mining*. WSDM '20. Houston, TX, USA: Association for Computing Machinery, 2020, pp. 672–680.

[81] Yu-Hao Xu et al. "A Recommendation Algorithm Based on a Self-supervised Learning Pretrain Transformer". In: *Neural Processing Letters* (Oct. 2022). ISSN: 1573-773X.

[82] Wujiang Xu et al. "iAgent: LLM Agent as a Shield between User and Recommender Systems". In: *Findings of the Association for Computational Linguistics: ACL 2025*. Ed. by Wanxiang Che et al. Vienna, Austria: Association for Computational Linguistics, July 2025, pp. 18056–18084.

[83] Junhan Yang et al. "GraphFormers: GNN-nested Transformers for Representation Learning on Textual Graph". In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato et al. Vol. 34. Curran Associates, Inc., 2021, pp. 28798–28810.

[84] Mingdai Yang et al. "AgentDR: Dynamic Recommendation with Implicit Item-Item Relations via LLM-based Agents". In: *Proceedings of the ACM Web Conference 2026*. WWW '26. United Arab Emirates: Association for Computing Machinery, 2026, pp. 6230–6241.

[85] Mingdai Yang et al. "Group Identification via Transitional Hypergraph Convolution with Cross-view Self-supervised Learning". In: *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. ACM, 2023, pp. 2969–2979.

[86] Mingdai Yang et al. "Instruction-based Hypergraph Pretraining". In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2024, pp. 501–511.

[87] Mingdai Yang et al. "PCL: Prompt-based Continual Learning for User Modeling in Recommender Systems". In: *Companion Proceedings of the ACM on Web Conference 2025, WWW 2025, Sydney, NSW, Australia, 28 April 2025 - 2 May 2025*. Ed. by Guodong Long et al. ACM, 2025, pp. 1475–1479.

[88] Mingdai Yang et al. "Personalized Recommendation Tool Learning via Autonomous Language Agents". In: *Proceedings of the 20th ACM Conference on Recommender Systems*. RecSys '26. To appear. 2026. arXiv: 2607.19739 [cs.IR].

[89] Mingdai Yang et al. "Ranking-based Group Identification via Factorized Attention on Social Tripartite Graph". In: *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*. ACM, 2023, pp. 769–777.

[90] Mingdai Yang et al. "Training Large Recommendation Models via Graph-Language Tokens Alignment". In: *Companion Proceedings of the ACM on Web Conference 2025, WWW 2025, Sydney, NSW, Australia, 28 April 2025 - 2 May 2025*. Ed. by Guodong Long et al. ACM, 2025, pp. 1470–1474.

[91] Mingdai Yang et al. "Unified Pretraining for Recommendation via Task Hypergraphs". In: *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. ACM, 2024, pp. 891–900.

[92] Wenting Ye et al. "A Transformer-Based Substitute Recommendation Model Incorporating Weakly Supervised Customer Behavior Data". In: *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '23. Taipei, Taiwan: Association for Computing Machinery, 2023, pp. 3325–3329.

[93] Chung-En Yu, Brian Jalaian, and Nathaniel Bastian. "Mitigating Large Vision-Language Model Hallucination at Post-hoc via Multi-agent System". In: *Proceedings of the AAAI Symposium Series* 4 (Nov. 2024), pp. 110–113.

[94] Junliang Yu et al. "Self-supervised multi-channel hypergraph convolutional network for social recommendation". In: *Proceedings of the web conference 2021*. 2021, pp. 413–424.

[95] Huanhuan Yuan, Jian Yang, and Jiajin Huang. "Improving Hypergraph Convolution Network Collaborative Filtering with Feature Crossing and Contrastive Learning". In: *Applied Intelligence* 52.9 (July 2022), pp. 10220–10233. ISSN: 0924-669X.

[96] An Zhang et al. "On Generative Agents in Recommendation". In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024, Washington DC, USA, July 14-18, 2024*. Ed. by Grace Hui Yang et al. ACM, 2024, pp. 1807–1817.

[97] Junjie Zhang et al. "AgentCF: Collaborative Learning with Autonomous Language Agents for Recommender Systems". In: *Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, May 13-17, 2024*. Ed. by Tat-Seng Chua et al. ACM, 2024, pp. 3679–3689.

[98] Wei Zhang et al. "Learning from Substitutable and Complementary Relations for Graph-based Sequential Product Recommendation". In: *ACM Trans. Inf. Syst.* 40.2 (2022), 26:1–26:28.

[99] Yuhui Zhang et al. "Language models as recommender systems: Evaluations and limitations". In: *NeurIPS 2021 Workshop on I (Still) Can't Believe It's Not Better*. 2021.

[100] Runhao Zhao et al. "Zero-shot Knowledge Graph Question Generation via Multi-agent LLMs and Small Models Synthesis". In: *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM 2024, Boise, ID, USA, October 21-25, 2024*. Ed. by Edoardo Serra and Francesca Spezzano. ACM, 2024, pp. 3341–3351.

[101] Wanjun Zhong et al. "MemoryBank: Enhancing Large Language Models with Long-Term Memory". In: *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*. Ed. by Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan. AAAI Press, 2024, pp. 19724–19731.

[102] Dengyong Zhou, Jiayuan Huang, and Bernhard Schölkopf. "Learning with Hypergraphs: Clustering, Classification, and Embedding". In: *Advances in Neural Information Processing Systems 19, Proceedings of the Twentieth Annual Conference on Neural Information Processing Systems, Vancouver, British Columbia, Canada, December 4-7, 2006*. Ed. by Bernhard Schölkopf, John C. Platt, and Thomas Hofmann. MIT Press, 2006, pp. 1601–1608.

[103] Fan Zhou and Chengtai Cao. "Overcoming Catastrophic Forgetting in Graph Neural Networks with Experience Replay". In: *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*. AAAI Press, 2021, pp. 4714–4722.

[104] Kun Zhou et al. "S3-Rec: Self-Supervised Learning for Sequential Recommendation with Mutual Information Maximization". In: *CIKM '20: The 29th ACM International Conference on Information and Knowledge Management, Virtual Event, Ireland, October 19-23, 2020*. Ed. by Mathieu d'Aquin et al. ACM, 2020, pp. 1893–1902.

[105] Jing Zhu et al. "Mosaic of Modalities: A Comprehensive Benchmark for Multimodal Graph Learning". In: *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*. June 2025, pp. 14215–14224.

Vita

Mingdai Yang

EDUCATION

Ph.D., 2026 Computer Science, University of Illinois Chicago, Chicago, Illinois.

M.S., 2020 Computer Science, The University of Chicago, Chicago, Illinois.

B.Eng., 2019 Computer Science and Engineering, The Chinese University of Hong Kong, Shenzhen, China.

RESEARCH EXPERIENCE

2021–2026 Research Assistant, Big Data and Social Computing Laboratory, Department of Computer Science, University of Illinois Chicago. Advisor: Prof. Philip S. Yu. Research on graph learning, hypergraph pretraining, and LLM-based agents for recommender systems.

2025 Applied Scientist Intern, Stores Economics and Science, Amazon, Palo Alto, California. Project on dynamic recommendation with implicit item-item relations via LLM-based agents.

2024 Applied Scientist Intern, Central Machine Learning, Amazon, Seattle, Washington. Projects on prompt-based continual learning for universal customer representation learning, and on integrating a graph neural network into a transformer-based sequential recommendation model.

2019–2021 Research Assistant, YLAB, The University of Chicago. Advisor: Prof. Yanjing Li. Simulation and evaluation of a hybrid optical-electrical deep neural network accelerator.

2018–2019 Research Assistant, Shenzhen Research Institute of Big Data, The Chinese University of Hong Kong, Shenzhen. Advisor: Prof. Xiaoguang Han. Depth inpainting for 3D point scene completion from a single depth image.

2017–2018 Research Assistant, Shenzhen Institute of Artificial Intelligence and Robotics for Society, The Chinese University of Hong Kong, Shenzhen. Advisor: Prof. Ning Ding. Virtual prototyping of a wheel-based cable inspection climbing robot.

TEACHING EXPERIENCE

2021–2024 Teaching Assistant, Department of Computer Science, University of Illinois Chicago, for CS 111 Program Design I, CS 112 Program Design I in the Context of Biological Problems, CS 151 Mathematical Foundations of Computing, and CS 362 Computer Design.

PUBLICATIONS

Yang, M., Choudhary, N., Du, J., Huang, E. W., Yu, P. S., Subbian, K., and Koutra, D.: AgentDR: Dynamic recommendation with implicit item-item relations via LLM-based agents. In *Proceedings of the ACM Web Conference 2026*, pages 6230–6241, 2026.

Yang, M., Liu, Z., Zhang, W., Wang, Y., Peng, H., and Yu, P. S.: Personalized recommendation tool learning via autonomous language agents. In *Proceedings of the 20th ACM Conference on Recommender Systems (RecSys '26)*, to appear, 2026.

Wang, C., **Yang, M.**, Liu, Z., Li, P., Pang, L., Wen, Q., and Yu, P. S.: AGP: Auto-guided prompt refinement for personalized reranking in recommender systems. In *Companion Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining*, pages 154–158, 2026.

Zhang, J., Peng, H., **Yang, M.**, and Yu, P. S.: Enhanced pre-training for recommendation via hypergraph structural entropy. *ACM Transactions on Information Systems*, 44(2):39:1–39:48, 2026.

Yang, M., Liu, Z., Yang, L., Liu, X., Wang, C., Peng, H., and Yu, P. S.: Training large recommendation models via graph-language tokens alignment. In *Companion Proceedings of the ACM Web Conference 2025*, pages 1470–1474, 2025.

Yang, M., Yang, F., Guo, Y., Xu, S., Zhou, T., Chen, Y., Shao, S., Liu, J., and Gao, Y.: PCL: Prompt-based continual learning for user modeling in recommender systems. In *Companion Proceedings of the ACM Web Conference 2025*, pages 1475–1479, 2025.

Yang, M., Liu, Z., Yang, L., Liu, X., Wang, C., Peng, H., and Yu, P. S.: Instruction-based hypergraph pretraining. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 501–511, 2024.

Yang, M., Liu, Z., Yang, L., Liu, X., Wang, C., Peng, H., and Yu, P. S.: Unified pretraining for recommendation via task hypergraphs. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pages 891–900, 2024.

Liu, X., Yang, L., Liu, Z., **Yang, M.**, Wang, C., Peng, H., and Yu, P. S.: Knowledge graph context-enhanced diversified recommendation. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pages 462–471, 2024.

Wang, C., Fan, Z., Yang, L., **Yang, M.**, Liu, X., Liu, Z., and Yu, P. S.: Pre-training with transferable attention for addressing market shifts in cross-market sequential recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2970–2979, 2024.

Wang, C., Yang, L., Liu, Z., Liu, X., **Yang, M.**, Liang, Y., and Yu, P. S.: Collaborative alignment for recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 2315–2325, 2024.

Yang, M., Liu, Z., Yang, L., Liu, X., Wang, C., Peng, H., and Yu, P. S.: Group identification via transitional hypergraph convolution with cross-view self-supervised learning. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pages 2969–2979, 2023.

Yang, M., Liu, Z., Yang, L., Liu, X., Wang, C., Peng, H., and Yu, P. S.: Ranking-based group identification via factorized attention on social tripartite graph. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*, pages 769–777, 2023.

Yang, L., Liu, Z., Wang, C., **Yang, M.**, Liu, X., Ma, J., and Yu, P. S.: Graph-based alignment and uniformity for recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pages 4395–4399, 2023.

Liu, X., Yang, L., Liu, Z., Li, X., **Yang, M.**, Wang, C., and Yu, P. S.: Group-aware interest disentangled dual-training for personalized recommendation. In *IEEE International Conference on Big Data*, pages 393–402, 2023.

Liu, X., Yang, L., Wang, C., **Yang, M.**, Liu, Z., and Yu, P. S.: Multi-view graph convolution for participant recommendation. In *IEEE International Conference on Big Data*, pages 5647–5656, 2023.

Yang, M., Jokar, M. R., Qiu, J., Lou, Q., Liu, Y., Udupa, A., Chong, F. T., Dallesasse, J. M., Feng, M., Goddard, L. L., Hu, X. S., and Li, Y.: A hybrid optical-electrical analog deep learning accelerator using incoherent optical signals. In *GLSVLSI '21: Great Lakes Symposium on VLSI 2021*, pages 271–276, 2021.

Han, X., Zhang, Z., Du, D., **Yang, M.**, Yu, J., Pan, P., Yang, X., Liu, L., Xiong, Z., and Cui, S.: Deep reinforcement learning of volume-guided progressive view inpainting for 3D point scene completion from a single depth image. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 234–243, 2019.

Zheng, M., **Yang, M.**, Yuan, X., and Ding, N.: A light-weight wheel-based cable inspection climbing robot: From simulation to reality. In *IEEE International Conference on Robotics and Biomimetics*, pages 1365–1370, 2018.